The Charlotte Operating System

An async-first, capability-based microkernel
for critical, high-performance software

2026-08-15

Scope and status

This manual describes both the implemented CharlotteOS prototype and its intended architecture. Unless a section explicitly says otherwise, statements about security, performance, transport independence, hardware support, or failure recovery are design goals rather than production guarantees. The implementation-status appendix records the repository's currently verified scope; source code and tests remain authoritative when the manual and implementation differ. Repository paths in this manual are clickable links to the corresponding file or directory on the public `main` branch; begin with the `docs/README.md` documentation index.

Contents

1 The Problem. 14

1.1 The operating system is the bottleneck 14

1.2 The retrofit tax, itemized 14

1.2.1 Async over blocking 14

1.2.2 Ambient authority 14

1.2.3 Hardware asynchrony disguised as software synchrony 14

1.2.4 Fault propagation across trust boundaries 15

1.3 Why Rust ownership is not enough 15

1.4 What needs to change 15

2 Design Philosophy. 17

2.1 The combined thesis 17

2.1.1 CharlotteOS — the protection, scheduling, and interrupt substrate. 17

2.1.2 Sitas — the shard-per-core execution discipline. 17

2.1.3 Xous — the isolated userspace service model 18

2.2 How they fit together. 18

2.3 The three kernel primitives 19

2.3.1 Capabilities: “May I?” 19

2.3.2 Endpoints: “With whom?”. 20

2.3.3 Memory Objects: “Where is the data, and who owns it?”. 20

2.4 An explicit design rule 20

2.5 What “asynchronous throughout” means 20

2.5.1 Blocking is still useful 21

2.5.2 Immediate operations remain immediate. 21

2.5.3 No asynchronous userspace upcalls 21

2.6 Why this matters for critical software 21

3 Vocabulary 23
3.1 Protection domain 23
3.2 Execution context 23
3.3 Logical processor (LP) 23
3.4 Sitas shard 24
3.5 Capability 24
3.6 Endpoint 24
3.7 Connection capability 24
3.8 Message 24
3.9 Reply token 25
3.10 Resource capability 25
3.11 Operation and completion queue 25
3.12 Notification 25
3.13 Rust Waker 25
3.14 Distinctions that matter 25
3.15 Quick reference: acronyms 26
4 Capabilities — Authority Without Ambiguity 27
4.1 What a capability is 27
4.2 Object types and their rights 27
4.3 Delegation and attenuation 28
4.4 Generation-safe lookup 28
4.5 Bootstrap — how capabilities enter a domain 28
4.6 Why capabilities matter for critical software 28
5 Endpoints — Communication as a First-Class Primitive 30
5.1 Server and connection model 30
5.2 Message classes 30
5.2.1 Scalar messages 30
5.2.2 Memory-bearing messages 30

5.3 Message envelope . . . 31
5.4 Call and reply . . . 31
5.4.1 Synchronous-looking futures . . . 31
5.4.2 Deferred replies . . . 32
5.4.3 Reply token semantics . . . 32
5.5 Connection delegation . . . 32
5.6 Service identity and discovery . . . 33
5.7 Two kernel data paths, not one . . . 33
5.8 Why endpoints matter for critical software . . . 34
6 Memory Objects — Ownership That Crosses Boundaries . . . 35
6.1 The problem with copying . . . 35
6.2 First-class memory objects . . . 35
6.3 Four transfer modes . . . 35
6.3.1 Copy . . . 35
6.3.2 Move . . . 36
6.3.3 BorrowRead . . . 36
6.3.4 BorrowWrite . . . 36
6.4 Why hardware enforcement matters . . . 36
6.5 Control plane versus data plane . . . 37
6.6 DMA as a third ownership dimension . . . 37
6.7 Why memory objects matter for critical software . . . 37
7 Completion Queues and Non-Lossy Results . . . 39
7.1 When to use a completion queue . . . 39
7.2 The CQ ring — a shared-memory completion buffer . . . 39
7.3 The non-lossy invariant . . . 40
7.4 Per-shard CQ partitioning . . . 40
7.5 Operation lifecycle . . . 40
7.6 Waiting on a CQ — `CQ_WAIT` . . . 41
7.7 LP affinity for CQ waits . . . 41

7.8 Operation IDs versus per-operation capabilities 42
7.9 Cancellation and buffer ownership 42
7.10 Why completion queues matter for critical software 42

8 Shards and Logical Processors 43
8.1 The logical processor (LP) — the hardware unit of scheduling 43
8.2 The shard — the userspace unit of ownership 43
8.3 The usual mapping: one shard per LP 43
8.4 `PerLp<T>` — interrupt-safe shared data 44
8.5 `ShardLocal<T>` — lock-free single-owner data 44
8.6 Why the distinction matters for critical software 45

9 The Sitas Executor 46
9.1 One executor per shard 46
9.2 The polling loop 46
9.3 The unified wait — three event sources, one wait point 47
9.3.1 Kernel completions (CQ entries) 47
9.3.2 Endpoint readiness (coalesced wake) 47
9.3.3 Device interrupts (coalesced wake) 47
9.4 Task wakeup — from CQ entry to future poll 47
9.5 Cross-shard messaging without the kernel 48
9.6 The ShardParker — blocking on native state 48
9.7 Why the sitas executor matters for critical software 49
9.8 A worked example: the mailbox index build 49
9.8.1 The workload 50
9.8.2 Kernel and userspace responsibilities 50
9.8.3 The mailbox path 50
9.8.4 Coordination and joining 51
9.8.5 Verification 51
9.8.6 Why this demo matters 51

10 Scheduling and Backpressure 53
10.1 The kernel scheduler 53
10.2 The block-yield-wake cycle 53
10.2.1 Block 53
10.2.2 Wake 53
10.2.3 Resume 54
10.3 Wake coalescing 54
10.4 The quantum timer 54
10.5 The deferred reaper 54
10.6 Load rebalancing 55
10.7 Backpressure and queue bounds 55
10.8 Why scheduling and backpressure matter for critical software 56
10.9 Scheduler observability 56
11 Service Architecture 58
11.1 Protection domains as failure boundaries 58
11.2 The supervisor — domain lifecycle manager 58
11.3 The name service — discovery without the kernel 59
11.4 Service restart and recovery 59
11.5 The overall system topology 60
11.6 Why service architecture matters for critical software 60
12 Userspace Drivers 61
12.1 The problem with kernel drivers 61
12.2 The DriverGrant 61
12.3 MMIO delegation 61
12.4 Interrupt delivery 62
12.5 The reference UART driver 62
12.6 DMA and the IOMMU 63
12.7 Driver service layering 63
12.8 Why userspace drivers matter for critical software 63

13 Networking 65
13.1 Why not sockets? 65
13.2 The native protocol stack 65
13.2.1 Ethernet — generic frame transport 66
13.2.2 Reliable message layer 66
13.2.3 RPC layer 66
13.2.4 Distributed objects 67
13.3 TCP/IP as a compatibility service 67
13.4 Transport independence 68
13.5 Zero-copy networking 68
13.6 Why the networking architecture matters for critical software 69
14 Consensus and Replication 70
14.1 Raft as a capability service 70
14.2 Intended properties relative to socket-based Raft 71
14.2.1 Authority, not addresses 71
14.2.2 Transport transparency 71
14.2.3 Zero-copy log replication 71
14.2.4 Capability-based security 71
14.2.5 Failure isolation 71
14.3 Persistence and launch configuration 72
14.4 Cluster bootstrap — the seed problem 72
14.4.1 Static seeds (current, simplest) 72
14.4.2 Pre-shared cluster capability (small kernel change) 72
14.4.3 Ethernet broadcast discovery (zero-configuration) 73
14.4.4 Distributed name service (implemented) 73
14.5 The generic StateMachine trait 73
14.6 Relationship to the name service 74
14.7 Election timers as first-class completions 74
14.8 Why consensus as a service matters for critical software 75

15 Persistent Storage 76
15.1 The storage stack 76
15.2 NVMe block driver 76
15.2.1 Initialisation 76
15.2.2 I/O path 77
15.2.3 Lessons from bringup 77
15.3 Block device protocol 77
15.4 Object store 78
15.4.1 On-disk layout 78
15.4.2 Current durability boundary 78
15.4.3 Operations 79
15.4.4 The initial service-artifact store 79
15.5 Raft persistence 79
15.6 Native filesystem 80
15.6.1 Directory encoding 80
15.6.2 Path resolution 80
15.6.3 Host inspection 80
15.7 Why this matters for critical software 81
16 Live Service Upgrade 82
16.1 The four primitives 82
16.2 The handoff protocol 82
16.3 Executable source 83
16.4 What the supervisor prototype provides 83
16.5 What the service must implement 84
16.6 What clients see 84
16.7 Restart-with-state vs. zero-downtime 85
16.8 Relationship to crash recovery 85
16.9 Why this matters for critical software 85

17 Server-Class Cluster Vision 86
17.1 The vision: interchangeable compute 86
17.2 Implemented foundation 86
17.3 Target node model and current bootstrap 87
17.4 Implemented deployment pipeline and production target 88
17.5 Placement: affinity, observation, migration 88
17.5.1 Declared policy: implemented contract 88
17.5.2 Observation. 89
17.5.3 Demonstrated assignment handoff and future state transfer 89
17.6 Remaining implementation boundary 89
17.7 Demonstrated two-node deployment slice 90
17.7.1 The replicated deployment manifest. 90
17.7.2 Cross-node hosting 90
17.7.3 The agent 91
17.7.4 clusterctl and the admin console 91
17.7.5 Naming 91
17.7.6 Cryptography: real signatures, honest boundaries 91
17.7.7 What the slice proves 92
17.8 Related work 92
17.8.1 Historical distributed operating systems 92
17.8.2 Cluster managers and placement products. 93
17.8.3 Trust, signing, and bootstrap 93
17.8.4 Membership and artifact stores 93
18 Programming Model 95
18.1 Invariants. 95
18.1.1 Only the owning LP mutates its state 95
18.1.2 Messages are owned values 95
18.1.3 References to shard-local state must not escape 95
18.1.4 Work stays on its assigned LP 95
18.1.5 Observability returns owned snapshots 95

18.2 Kernel-side primitives . . . 95
18.2.1 PerLp<T> . . . 95
18.2.2 ShardLocal<T> . . . 96
18.2.3 ShardMailbox<M> . . . 96
18.2.4 IPI closures . . . 96
18.2.5 Completion API . . . 96
18.2.6 CQ ring . . . 97
18.3 Userspace programming model . . . 97
18.3.1 The launch contract . . . 97
18.3.2 Resource ownership in Rust . . . 98
18.3.3 Bootstrap authority . . . 98
18.3.4 Caller identity is kernel authority . . . 99
18.3.5 Backpressure is normal . . . 99
18.3.6 Sitas integration . . . 99
18.4 Rule-of-thumb guidance . . . 99
18.5 The syscall ABI . . . 99
18.6 Why the programming model matters for critical software . . . 100
19 Development Workflow . . . 101
19.1 Prerequisites . . . 101
19.2 Build targets . . . 101
19.2.1 Kernel build . . . 101
19.2.2 Userspace service programs . . . 102
19.3 Booting in QEMU . . . 102
19.3.1 AArch64 . . . 102
19.3.2 x86-64 . . . 102
19.3.3 Expected output . . . 103
19.4 Self-tests . . . 103
19.5 The async-syscall demo . . . 104
19.6 CI pipeline . . . 104
19.7 Codebase reading guide . . . 104

19.8 Troubleshooting 105

20 Formal Safeguards with TLA+ 106

20.1 Why model functionality as a state machine? 106

20.2 The safeguarding loop 107

20.3 What is modeled 108

20.4 Model-driven findings 109

20.4.1 Authorization is controlled capability issuance 110

20.4.2 Two-phase service publication 110

20.4.3 Reliable-message session identity 110

20.4.4 Restart-safe Raft admission 111

20.4.5 Retained regression patterns 111

20.5 Concrete conformance and atomicity 112

20.6 Validation 113

20.6.1 Running TLC 113

20.6.2 Implementation validation 114

20.7 What a successful check means 114

20.8 Workflow for changes 115

A Glossary 116

B Implementation Status 118

B.1 Implementation phases 118

B.2 Success criteria 119

B.3 Verified scope 119

B.4 Known limitations 120

C Lessons Learned 121

C.1 Bugs found on hardware 121

C.1.1 Panic handler recursed through heap-dependent logging 121

C.1.2 Blocked threads lost their execution context 121

C.1.3 RwLock held-across-call deadlocks 121

C.1.4 Quantum timer grew without bound . . . 121
C.1.5 A thread cannot free its own kernel stack . . . 121
C.1.6 LA57 paging-mode mismatch . . . 122
C.1.7 ASID-0 collision . . . 122
C.1.8 x86-64 trampoline halted on thread return . . . 122
C.1.9 Hypervisors that strip TTBR0 ASIDs break TLB isolation . . . 122
C.2 Engineering lessons . . . 122

D Research Lineages and Their Afterlives . . . 123
D.1 Capability kernels and confined Unix . . . 123
D.1.1 CharlotteOS inheritance . . . 123
D.2 IPC, typed channels, and ownership . . . 124
D.2.1 CharlotteOS inheritance . . . 124
D.3 Multicore systems, runtimes, and fast I/O . . . 124
D.3.1 CharlotteOS inheritance . . . 125
D.4 Service recovery . . . 125
D.5 From distributed operating systems to cluster managers . . . 125
D.5.1 CharlotteOS inheritance . . . 126
D.6 The synthesis and the rejected inheritances . . . 126

E Architectural Redirections . . . 127
E.1 Retained mechanisms . . . 127
E.2 What to reframe . . . 127
E.3 Replacements adopted . . . 128
E.3.1 Stop expanding raw mailbox syscalls . . . 128
E.3.2 Do not allocate one completion capability per service request . . . 128
E.3.3 Separate readiness, notification, and completion . . . 128
E.3.4 Replace arbitrary cross-LP closures . . . 128
E.3.5 Do not equate shard wake with IPI delivery . . . 128
E.3.6 Introduce memory objects before rich IPC payloads . . . 128
E.3.7 Treat service discovery as userspace policy . . . 128

1 The Problem

1.1 The operating system is the bottleneck

Critical software must be both **correct** and **fast**. Modern systems often pursue both goals by adding layers to operating systems whose basic abstractions were designed for timesharing.

This manual calls the resulting complexity and performance cost the **retrofit tax**: a synchronous, ambient-authority, monolithic system must be made to behave as though it were asynchronous, least-privilege, and decomposed.

1.2 The retrofit tax, itemized

1.2.1 Async over blocking

Many operating systems expose synchronous calls: `read()` may block the calling thread until data arrives. Applications use `epoll`, `kqueue`, or `io_uring` for concurrency without dedicating one thread per operation. These interfaces are effective, but applications may still need adapters when a kernel or library operation blocks. Runtimes then translate blocking work into readiness or completion notifications.

This translation layer brings with it:

- Async-signal-safety problems — the kernel may deliver a completion while userspace holds a lock, requiring elaborate signal-handler discipline or a dedicated thread pool.
- Thread-pool overhead — many async runtimes spawn a pool of OS threads to convert blocking syscalls into futures, undoing the resource benefits of asynchronous execution.
- Two concurrency models — the application uses async/await; the kernel uses threads and blocking. The mismatch forces every boundary crossing to bridge the two models.

1.2.2 Ambient authority

A conventional OS process has access to the filesystem namespace, the network, and other global resources by default. Security is subtractive: you start with everything and try to take things away (chroot, seccomp, namespaces). This is brittle because it requires the developer to enumerate what *should not* be accessible — an unbounded problem. In practice, most programs run with far more authority than they need, and exploits exploit that gap.

1.2.3 Hardware asynchrony disguised as software synchrony

Hardware is inherently asynchronous with respect to the CPU. A disk controller signals completion via an interrupt. A network card delivers packets on its own schedule. A timer fires after a programmed interval. Yet the operating system interface presents these as synchronous calls: `read()` returns when the data is ready, blocking the calling thread in the meantime.

A thread blocked on I/O cannot do other work. High-concurrency servers compensate with thread pools; real-time systems may use carefully tuned polling loops.

1.2.4 Fault propagation across trust boundaries

A bug in a device driver can crash a monolithic kernel. A memory corruption in one application can, in a system without hardware-enforced isolation, corrupt another. Conventional microkernels solve this with address-space isolation, but at the cost of IPC overhead that often drives designers back toward monolithic architectures for performance.

A common compromise is to isolate untrusted code in virtual machines while keeping trusted code in the kernel. Kernel defects can still cross that boundary.

1.3 Why Rust ownership is not enough

Safe Rust prevents many data races, use-after-free errors, and null-pointer dereferences. Unsafe code, hardware interfaces, and logic errors remain. This is a substantial improvement for systems programming, but Rust programs run on operating systems that do not share Rust's ownership discipline.

- A Rust program cannot protect itself from a C driver that corrupts kernel memory. Isolation must be enforced by the hardware (MMU, IOMMU), not by the language.
- A Rust `Mutex<Vec<T>>` crossing a syscall boundary is still a shared-memory lock with all the contention, priority inversion, and deadlock risks that implies. The kernel does not know the lock exists.
- Rust's `async`/`await` compiles to a state machine driven by a userspace executor, but the executor must still interact with the kernel's blocking I/O model. The `Future` trait and the kernel's wait queue speak different languages.

Rust prevents some bug classes within an address space. The OS still defines the rules between address spaces. If authority is ambient until restricted, the language's guarantees remain local.

1.4 What needs to change

We need an operating system whose foundational abstractions align with the demands of critical, performant software:

1. Asynchronous by construction — operations that may wait for external progress should have submission/completion semantics, not blocking-call semantics. No subsystem should force a thread to dedicate itself to one operation.
2. Capability-based, not ambient, authority — a process should be able to perform an operation only if it holds a capability authorizing it. Capabilities are unforgeable and must be explicitly granted.
3. Isolation by default — every protection domain should run in its own address space. Communication crosses a kernel-mediated boundary. A fault in one domain should not propagate to another.
4. Ownership that crosses boundaries — data movement between domains should transfer ownership, not copy bytes. The kernel should move page mappings, so the sender loses access and the receiver gains it.

5. Bounded resource consumption — every queue, table, and buffer should be bounded. Backpressure should propagate explicitly. No unbounded allocation should be a correctness requirement.
6. The OS and the language runtime should agree — the kernel's model of concurrency (asynchronous, completion-driven) should match the language's model of concurrency (futures, wakers). No translation layer should be needed.

CharlotteOS is built around these six principles. The rest of this manual describes the architecture, programming model, implementation status, and evidence for its safety claims.

2 Design Philosophy

CharlotteOS combines ideas from three projects around one thesis.

2.1 The combined thesis

> **Hardware is asynchronous; the operating system and the userspace runtime should be too.**

Three systems approach the same design space from different directions. Rather than collapsing their abstractions into one universal mechanism, the architecture composes them at the right boundaries.

2.1.1 CharlotteOS — the protection, scheduling, and interrupt substrate

CharlotteOS contributes the mechanisms that *must* be privileged:

- Address-space isolation through page tables and the MMU.
- Kernel scheduling of threads across logical processors.
- LP-local state and interrupt routing.
- Page-table manipulation and physical-memory management.
- MMIO and DMA authority delegation.
- Per-domain capability tables.
- Event observation and thread wakeup.
- Syscall dispatch and asynchronous completion delivery.

Its design rule is: **the kernel provides the minimum set of primitives from which everything else can be composed.** If a facility can be built in userspace without compromising isolation, it belongs in userspace.

2.1.2 Sitas — the shard-per-core execution discipline

Sitas contributes an execution discipline — a set of rules about how state is owned and mutated:

> Only the owning shard mutates shard-local state. Cross-shard interaction occurs through bounded typed messages carrying owned values.

Sitas provides:

- One executor per shard — cooperative task scheduling within a shard; preemption between shards.
- Futures as userspace state machines — every operation that may wait is a `Future`, driven by the shard's executor.
- Bounded backpressure — every queue has fixed capacity; senders must handle `WouldBlock`.

- Shard-local ownership — mutable state belongs to exactly one shard; no locks, no atomics for shard-internal data.
- Explicit placement — where a task runs is a deliberate choice, not an accident of scheduling.
- Typed command/reply APIs — what a shard receives is a specific Rust type, not a generic byte buffer.
- Structured shutdown and cancellation — tasks can be cancelled, and cancellation has defined ownership semantics.
- Observability through owned snapshots — diagnostics capture state by value, never by borrowed reference into runtime internals.

The sitas discipline is enforced by Rust's type system within a single protection domain. CharlotteOS extends it across domains through the kernel.

2.1.3 Xous — the isolated userspace service model

Xous, a microkernel OS for Precursor and Betrusted hardware, contributes the protection-domain service model:

- Services are userspace servers — drivers, filesystems, and protocol stacks run in their own protection domains, not in the kernel.
- Clients connect to servers and receive connection identifiers — a connection *is* authority; possessing one means you may communicate with that server.
- IPC is a kernel primitive — communication is not a convention layered on shared memory; it is mediated and validated by the kernel.
- Scalar messages are distinct from memory-bearing messages — small control-plane operations (opcodes, handles, status codes) travel in fixed-size messages; bulk data travels as memory-object attachments.
- Memory IPC distinguishes ownership transfer from borrowing — move transfers ownership permanently. Borrow grants temporary access, revoked on reply or cancellation.
- A userspace name service maps human-readable names to opaque server identities — the kernel knows nothing about strings.

CharlotteOS adopts Xous's userspace-server model and combines it with the sitas execution discipline and an async-native kernel. The detailed co-design is maintained in `docs/architecture/sitas-xous.md`.

2.2 How they fit together

The three systems do not collapse into one. They compose at well-defined boundaries:

Layer	Provided by
Shard-local async execution	sitas
Cross-domain protection and IPC	CharlotteOS kernel (Xous-inspired)
Completion delivery for kernel/device ops	CharlotteOS kernel
Service discovery and policy	Userspace (name service, supervisor)

The layers are:

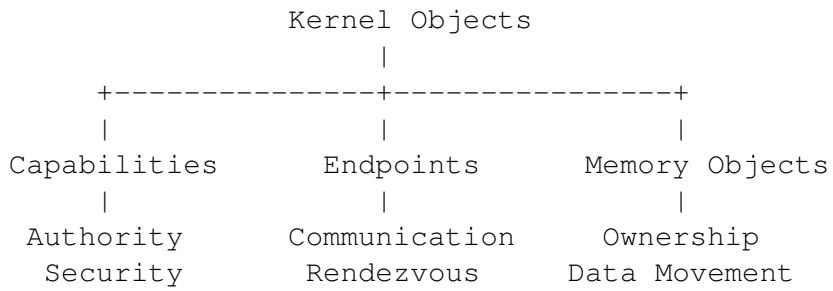
```
CharlotteOS kernel
    protection domains
    threads and LP scheduling
    capabilities and endpoint connections
    interrupt routing
    memory objects and page mappings
    operation submission and completion queues

Userspace service process
    one or more externally visible IPC endpoints
    protocol-specific request/reply handling
    one sitas executor per assigned shard
    shard-local state
    typed in-process commands
    driver or service implementation

Client process
    typed protocol library
    endpoint connection capabilities
    futures backed by IPC replies or completion records
    owned buffers whose transfer mode is explicit
```

2.3 The three kernel primitives

Three orthogonal kernel abstractions form the foundation:



Every other kernel facility — completion queues, reply tokens, device grants, DMA mappings — is a **composition** of these three primitives.

2.3.1 Capabilities: “May I?”

A capability answers one question: **“May I perform this operation?”** It represents authority, not work. Examples include endpoint capabilities, connection capabilities, memory-object capabilities, interrupt capabilities, DMA-domain capabilities, and timer capabilities.

2.3.2 Endpoints: "With whom?"

An endpoint answers: **"Who am I communicating with?"** It provides bounded rendezvous between protection domains. Endpoints know nothing about packet buffers, futures, DMA, or ownership. They carry requests and replies.

2.3.3 Memory Objects: "Where is the data, and who owns it?"

A memory object answers: **"Where is the data, and who currently owns it?"** It represents page-backed storage whose ownership can move independently of communication. Its responsibilities include page ownership, mappings, transfer modes, DMA state, and lifetime.

2.4 An explicit design rule

> **When adding a new subsystem, first ask whether it can be expressed using capabilities, endpoints, and memory objects. New kernel primitives should be introduced only when they cannot be naturally composed from the existing ones.**

This rule limits the kernel surface and keeps the ABI and trust model reviewable.

2.5 What "asynchronous throughout" means

"Asynchronous throughout" means:

1. Operations that may wait for external progress have submission/completion semantics.
2. No subsystem boundary forces the caller to dedicate a thread to waiting for one operation.
3. A userspace shard can wait on one aggregated kernel source (the CQ).
4. Completions are retained until observed.
5. Buffers and capabilities have explicit ownership during an operation.
6. Cancellation and teardown have defined state transitions.
7. Bounded queues preserve backpressure across layers.
8. Service implementations can suspend one task while continuing other work on the shard.

This is not the same as saying:

- Every syscall is asynchronous.
- Every call returns a newly allocated completion capability.
- No thread ever blocks.
- Every wake sends an IPI.
- All waitable objects have identical semantics.
- Userspace code is interrupted by arbitrary asynchronous callbacks.

2.5.1 Blocking is still useful

When no sitas task is runnable, the shard's execution context should block in one kernel wait operation:

```
ready tasks exist
    -> poll tasks

no ready tasks
    -> wait on CQ / endpoint readiness / deadline

event arrives
    -> execution context becomes runnable
    -> drain events
    -> wake relevant futures
```

> **The shard parks directly on native completion and message state; no helper-thread pool is required merely to convert blocking kernel interfaces into futures.**

2.5.2 Immediate operations remain immediate

Operations that cannot meaningfully block — capability duplication, metadata queries, nonblocking CQ drain, closing a quiescent object — complete synchronously. The ABI does not force asynchronous ceremony onto inherently synchronous work.

2.5.3 No asynchronous userspace upcalls

Completions do not inject callbacks into userspace at arbitrary instructions. The path is:

```
hardware interrupt
    -> kernel records result or notification
    -> blocked execution context becomes runnable
    -> normal return to userspace
    -> sitas executor drains CQ/endpoints
    -> executor wakes relevant Rust tasks
```

The kernel may preempt threads, but it does not inject arbitrary task callbacks into userspace. Task scheduling therefore remains cooperative and reviewable.

2.6 Why this matters for critical software

Every design decision in CharlotteOS traces back to one or more of the six principles from Chapter 1:

- Capabilities replace ambient authority — a compromised service cannot access resources it was never granted. The set of things a process can do is enumerable and auditable by construction.

- Endpoints make communication authenticated — a client that holds a connection capability has proven (at grant time) that it is authorized to communicate. The kernel enforces this on every message.
- Memory objects constrain stale cross-domain access — ownership moves through the kernel and is enforced by page tables. Correctness still depends on MMU, TLB, and DMA handling.
- Composition limits the privileged surface — most system functionality lives in userspace, where faults can be contained to a service domain.
- The async model reduces adaptation — the kernel and runtime share submission, completion, and wake semantics.
- Bounded ingress makes pressure visible — endpoint and hardware queues apply explicit backpressure. Chapter 10 records the remaining unbounded internal backlog.

3 Vocabulary

This chapter defines the manual's CharlotteOS-specific terms.

3.1 Protection domain

A **protection domain** is an address space and authority boundary. It owns:

- A capability table.
- Memory mappings.
- One or more execution contexts (threads).
- Resource accounting.
- Failure and teardown state.

A *process* is the concrete implementation of a protection domain, but architectural discussion prefers the semantic term.

3.2 Execution context

An **execution context** is a kernel-scheduled thread. It may have:

- LP affinity.
- Priority or scheduling class.
- A current protection domain.
- A wait state.
- An associated userspace executor.

3.3 Logical processor (LP)

A **logical processor** is CharlotteOS's representation of a schedulable hardware execution context — typically one CPU core (or one hyperthread).

An LP is **not** a shard (in the sitas sense). A typical mapping is:

```
one sitas shard
    <-> one LP-affine userspace thread
    <-> normally one logical CPU
```

The architecture retains the distinction so that CPU hotplug, oversubscription, migration, asymmetric CPUs, and multiple sharded processes remain possible.

3.4 Sitas shard

A **shard** is a userspace ownership and execution domain:

- One executor runs its tasks.
- At most one task at a time mutates shard-owned state.
- Cross-shard access occurs through typed messages carrying owned values.
- Placement is explicit but is not the shard's identity.

3.5 Capability

A **capability** is an unforgeable reference to a kernel object, carrying rights. It answers: "May I perform this operation?"

Capabilities are scoped to a protection domain's capability table. A process cannot fabricate a capability; it can only receive one through the bootstrap contract or through IPC.

3.6 Endpoint

An **endpoint** is a bounded server-side IPC receive object. It has:

- An owning protection domain.
- A queue capacity.
- A protocol/interface identity.
- Receive rights.
- Lifecycle and closure state.

3.7 Connection capability

A **connection capability** is a client-side authority to send or call an endpoint. Possession of a service name is not authority. A name service resolves a name and conditionally returns a connection capability.

3.8 Message

A **message** is a kernel-validated IPC envelope consisting of:

- Interface or protocol identifier.
- Opcode.
- Flags.
- Inline scalar words.
- Zero or more memory-object attachments.
- Zero or more delegated capability attachments.

- Optional reply authority.

The kernel understands the envelope structure, not arbitrary Rust types.

3.9 Reply token

A **reply token** is one-shot authority to complete a blocking or deferred call. It is:

- Created by the kernel.
- Bound to the caller's pending call.
- Consumable exactly once.
- Invalidated by cancellation, caller death, or endpoint teardown.
- Optionally delegable when explicitly allowed.

3.10 Resource capability

A **resource capability** authorizes operations on a kernel-managed object such as a memory object, endpoint, connection, interrupt, MMIO region, DMA domain, device, completion queue, or timer.

3.11 Operation and completion queue

An **operation** is submitted work whose progress may depend on the kernel, hardware, or a privileged service.

A **completion queue (CQ)** is the aggregated, per-shard or per-process destination for operation results. It is waitable (with `CQ_WAIT`) and has non-lossy terminal completion semantics.

3.12 Notification

A **notification** means: "State may have changed; inspect the corresponding object or queue." It may be coalesced. It is not itself a result.

3.13 Rust Waker

A Rust `Waker` means: "Poll this userspace future again." It is a userspace scheduling hint. It is **not** a kernel completion record and should not cross the ABI directly.

3.14 Distinctions that matter

Several concepts sound similar but must be kept separate. Confusing them leads to architectural mistakes.

Concept	Is	Is not
Capability	Unforgeable authority to act	An event, a file descriptor, a handle
Endpoint	Server-side bounded receive queue	A socket, a pipe, a channel
Completion	A result of an async operation	A Waker, a notification
Waker	A hint to repoll a future	A kernel completion record
Shard	A userspace ownership domain	An LP, a core, a thread
LP	A schedulable hardware context	A shard
Notification	“Inspect state”	A result, a data payload
IPC call	Cross-domain service invocation	A kernel operation submission

3.15 Quick reference: acronyms

ASID Address Space Identifier. 0 = kernel; >0 = user address space.
CQ Completion Queue. Shared-memory ring buffer for async operation results.
EL0 Exception Level 0 (AArch64). Userspace execution.
EL1 Exception Level 1 (AArch64). Kernel execution.
HHDM Higher Half Direct Mapping. Linear mapping of all physical memory into kernel virtual address space (provided by the Limine bootloader).
IPI Inter-Processor Interrupt. Signal from one LP to another.
LP Logical Processor. One schedulable hardware execution context.
MMU Memory Management Unit. Hardware that enforces page-table permissions.
IOMMU / SMMU I/O Memory Management Unit. Enforces DMA access permissions from devices, analogous to the MMU for CPU access.

4 Capabilities — Authority Without Ambiguity

A capability is the answer to one question: **"May I perform this operation?"** It is authority, not work. CharlotteOS capabilities are unforgeable, delegable, and attenuable.

4.1 What a capability is

A capability is a per-address-space handle that names a kernel object and carries a rights mask. The kernel maintains a **capability table** for each protection domain. An entry in this table is the only way a userspace process can reference a kernel object.

Capabilities have four key properties:

- Unforgeable — a process cannot create a capability through computation. Capabilities enter a domain only through the bootstrap contract or through IPC delegation.
- Scoped — a capability is valid only in the domain that holds it. Capability index 3 in domain A refers to a different object (or nothing) than capability index 3 in domain B.
- Righted — a capability carries a rights mask specifying which operations are permitted. An endpoint capability might carry only `SEND | CALL` rights; it does not grant `RECEIVE`.
- Revocable — when a kernel object is destroyed, all capabilities referencing it become invalid. Subsequent use returns an error, not undefined behavior.

4.2 Object types and their rights

CharlotteOS defines a fixed set of kernel object types, each with its own rights model. A capability carries both a type tag and a rights mask.

Type	Example rights	What it authorizes
Endpoint	RECEIVE, MINT_CONNECTION	Server-side IPC receive queue
Connection	SEND, CALL, DELEGATE	Client-side authority to invoke a service
ReplyToken	COMPLETE, DELEGATE	One-shot completion of a pending call
CompletionQueue	SUBMIT, WAIT, DRAIN	Submission to / waiting on a CQ
MemoryObject	MAP, UNMAP, MOVE, LEND, COPY	Page-backed memory ownership and access
Interrupt	ACK, MASK, BIND_TO_CQ	Hardware interrupt handling
MmioRegion	MAP_DEVICE	Device register access (MMIO)
DmaDomain	PIN, UNPIN	DMA buffer management

Device	RESET, CONFIGURE	Device lifecycle management
Timer	ARM, CANCEL, BIND_TO_CQ	Timer operations

4.3 Delegation and attenuation

Capabilities can be **delegated** to another protection domain through IPC. When delegating, the sender may **attenuate** the rights: the receiver gets a subset of the rights the sender held. A service manager that holds a connection with `SEND | CALL | DELEGATE` rights can delegate a client-facing connection with only `SEND | CALL`, keeping `DELEGATE` for itself.

Delegation happens in the kernel, mediated by the IPC path. The sender specifies the target domain, the capability to delegate, and the reduced rights mask. The kernel validates that the sender actually holds the capability with at least the rights being transferred.

4.4 Generation-safe lookup

Capability tables use generation counters to prevent use-after-free through stale indices. When a capability is revoked, the table slot is freed and its generation is incremented. Any attempt to use an old index with a stale generation returns `InvalidCapability` rather than accessing a different object that happened to be allocated at the same index.

4.5 Bootstrap — how capabilities enter a domain

A newly created protection domain starts with an empty capability table. It receives its initial capabilities through the **config-page contract**: a mapped page that the supervisor populates with typed entries before the domain begins execution. The bootstrap contract delivers at minimum:

- A connection to the name service (for service lookup).
- For driver domains: device capabilities (MmioRegion, Interrupt).
- For service domains: an endpoint on which to listen.
- A default CQ handle for submission and waiting.

After bootstrap, additional capabilities arrive through IPC: the name service delegates service connections; the supervisor delegates device grants; other services delegate whatever the protocol authorizes.

4.6 Why capabilities matter for critical software

1. Least privilege is architectural, not advisory — a process starts with nothing. Every capability it holds was explicitly granted by a trusted component (the supervisor, the name service, or another service under a protocol). There is no ambient authority to strip away; the default is zero.

2. Kernel-mediated authority is enumerable — a process's capability table records the kernel objects it may use. This makes authority easier to audit, but does not by itself account for device side effects, protocol state, or implementation defects.
3. Delegation is mediated — a process cannot grant authority it does not hold; it cannot grant rights beyond what it possesses; and every delegation crosses the kernel, which enforces both constraints.
4. Teardown revokes the destroyed domain's capabilities and closes its endpoint relationships. Existing tests cover stale connections and pending-call reconciliation; this is not a claim of system-wide atomic revocation for every future object type or external device effect.
5. Capability tables are bounded — a process cannot create unlimited capabilities to exhaust kernel memory. Submission backpressure (`WouldBlock`) applies when a table is full, forcing the application to drain completions before submitting more work.

Capabilities give endpoints and memory objects explicit authority instead of making them ambient resources addressed only by an index or address.

5 Endpoints — Communication as a First-Class Primitive

An endpoint answers: **"Who am I communicating with?"** It is CharlotteOS's primary primitive for calls between protection domains.

5.1 Server and connection model

An **endpoint** is a bounded server-side IPC receive object. A service creates one and receives messages on it. A **connection capability** is a client-side authority to send to that endpoint. Connections are minted from the endpoint (typically by the name service) and delegated to clients.

```
Service creates endpoint:   endpoint_create(interface="ns", capacity=16)
Name service mints conn:    connection_mint(endpoint, SEND | CALL)
Client receives conn:       name_service.lookup("ns") -> ConnectionCap
```

Ordinary clients do not mint arbitrary connections. The name service or a policy service normally delegates them after an access check.

5.2 Message classes

The ABI distinguishes two message classes:

5.2.1 Scalar messages

A scalar message carries an opcode, one 64-bit argument word, and optionally a reply token. It fits in registers. It is suitable for:

- Opcodes and request types.
- Capability handles.
- Status values and queue indices.
- Compact control-plane operations where the payload is a single word.

Scalar messages require no memory-object allocation, page-table change, or TLB invalidation.

5.2.2 Memory-bearing messages

A message can carry one or more **memory-object attachments** with explicit transfer modes (see Chapter 6). These are suitable for:

- Structured requests and responses.

- Strings, names, and configuration structures.
- Packet payloads and block I/O buffers.
- Larger replies that exceed register width.

The kernel validates each attachment: the sender must hold the referenced memory object with sufficient rights for the declared transfer mode. The receiver gets a new capability to the memory (or a temporary mapping, for borrows).

5.3 Message envelope

The kernel validates a fixed-width message envelope. It does not understand Rust enums, `serde` formats, or protocol-specific types:

```
struct MessageHeader {
    interface: u128,       // protocol identity
    version: u32,
    opcode: u32,
    flags: u32,
    inline_len: u16,
    segment_count: u16,
    capability_count: u16,
    reserved: u16,
    user_tag: u64,
}
```

Constraints on the ABI:

- Fixed-width fields — no `usize`, no pointer-sized values.
- Explicit alignment — no compiler-dependent layout.
- Checked offsets and lengths — the kernel rejects out-of-bounds.
- Explicit protocol versioning — forward compatibility is negotiated.
- Maximum attachment counts — bounded kernel work per message.
- No arbitrary userspace pointers as durable message identity.

5.4 Call and reply

The most common IPC pattern is **call/reply**: a client sends a request and expects a response.

5.4.1 Synchronous-looking futures

At the Rust level, a client writes:

```
let response = network_service.connect(address).await?;
```

Underneath, the runtime:

1. Sends a call message through the connection capability.
2. The kernel creates a one-shot **reply token** bound to this call.
3. The caller task becomes pending (the `Future` returns `Poll::Pending`).
4. The caller shard continues running other tasks — no thread is dedicated to this request.
5. The server receives the message on its endpoint.
6. The server replies (immediately or later).
7. Reply arrival wakes the client future, which returns `Poll::Ready(response)`.

5.4.2 Deferred replies

A server may retain a reply token and complete it later — while continuing to serve other requests on the same shard. This supports device-driven operations:

```
async fn handle_read(pending: PendingRequest) {
    // Submit a DMA read, retaining the reply token.
    // Continue serving other clients while the read is in flight.
    let buffer = device.read(address, len).await?;
    pending.reply_with_buffer(buffer);
}
```

The shard's executor polls other tasks while the read completes.

5.4.3 Reply token semantics

A reply token is:

- Created by the kernel when a call message is enqueued.
- Bound to the pending call — it completes exactly that caller's future.
- Consumable exactly once — a second reply attempt returns an error.
- Invalidated by cancellation — if the caller times out or aborts, the reply token becomes inert.
- Invalidated by endpoint teardown — if the server dies, all outstanding reply tokens are invalidated, and all waiting callers receive `ServiceRestarted`.
- Optionally delegable — a server may forward a reply token to another service, allowing a chain of services to contribute to a single reply.

5.5 Connection delegation

A connection capability can be delegated to another domain with attenuated rights. This is the mechanism by which the userspace name service returns service-level connections:

```
// Name service:
let full_connection = name_service.lookup("driver.nic.v1")?;
```

```
// full_connection has SEND | CALL | DELEGATE rights

// Delegate to a client with reduced rights:
connection_delegate(full_connection, client_domain, SEND | CALL)?;
// client_domain now holds a connection with SEND | CALL only
```

The kernel mediates. The name service never sees raw endpoint identities; it holds connection capabilities, which it re-delegates. Each delegation step can only reduce rights.

5.6 Service identity and discovery

The kernel uses opaque endpoint identities. A userspace **name service** maps human-readable names to re-delegable connection capabilities:

- Registration — a service registers as `"driver.nic.v1"` with a specific endpoint. The name service stores the endpoint's connection.
- Generation tracking — each registration bumps a generation counter. Stale client connections bound to the previous generation fail with `ServiceRestarted`.
- Access-key gating — a registration may specify a non-zero access key. Lookups without the matching key are denied — policy enforced by a userspace service, not the kernel.
- Lookup — a client asks for `"driver.nic.v1"` and receives a connection capability (or an error). It never receives an address, a port number, or a raw handle.

The name service runs in userspace. The kernel provides the endpoint and connection primitives; userspace provides the naming policy.

5.7 Two kernel data paths, not one

A central distinction is that endpoint IPC is for **cross-domain service calls**. Completion queues are for **kernel and device operations**. They are not the same mechanism.

Endpoint IPC path	Completion queue path
Application → network service	Timer expiry
Network service → NIC driver	DMA completion
Application → filesystem service	IRQ-derived device completion
Service → name service	Waiting for thread/process exit

A userspace driver may compose both: receive an endpoint message (data path), then submit a hardware operation and await a CQ entry (control path). The endpoint message and the hardware completion are related *in the service's logic*, but they are different kernel abstractions with different semantics.

5.8 Why endpoints matter for critical software

1. Communication carries authority — a client that holds a connection capability was authorized (at grant time, by the name service or policy service) to communicate with the target. Every message is authorized by possession of a kernel-mediated capability rather than by an address or UID alone.
2. Scalar and memory messages are distinct — the kernel does not copy arbitrary data between address spaces by default. Small control messages stay small. Bulk data moves as memory objects (Chapter 6) with explicit ownership semantics.
3. Deferred replies enable efficient concurrency — a server is never forced to block waiting for one slow client. It retains reply tokens for in-flight work and continues serving other requests. The shard's executor multiplexes naturally.
4. Restart errors are defined — stale connections return typed errors such as `EndpointClosed` and `ServiceRestarted`. A replacement cannot inherit the old instance's reply tokens.
5. Name service is policy, not mechanism — the kernel provides endpoints and connections. Userspace decides who can register what name, who can look up what service, and what access keys are required. The kernel remains small; policy is replaceable without a kernel rebuild.

6 Memory Objects — Ownership That Crosses Boundaries

A memory object answers: **"Where is the data, and who owns it?"** It lets page-backed data cross protection-domain boundaries with explicit ownership and, for move and borrow modes, without copying the payload.

6.1 The problem with copying

In a conventional OS, moving data between processes typically means copying: the kernel reads from the sender's address space and writes to the receiver's. For large payloads (packet buffers, file blocks, IPC messages), this copy dominates the cost of the operation.

Unrestricted shared memory avoids the copy but permits two processes to modify the same pages. Correctness then depends on synchronization rather than page-table exclusivity.

CharlotteOS adopts a third approach: **ownership transfer through page-table manipulation.** When a memory object moves from one domain to another, the kernel changes the page-table mappings. The sender loses access; the receiver gains it. No bytes are copied. The MMU enforces the ownership.

6.2 First-class memory objects

A memory object is a dedicated kernel object representing page-backed storage with explicit ownership and capability semantics. The kernel manages:

- Physical frames — the backing storage.
- Mappings — which domains have virtual addresses pointing to the pages, and with what permissions.
- Ownership — which domain currently holds the master capability.
- Transfer state — whether the object is idle, in transit, borrowed, or undergoing DMA.
- Lifetime — when the object is freed, all mappings are revoked and all capabilities to it are invalidated.

Userspace never exchanges raw pointers across protection domains. Instead, IPC messages attach **memory-object capabilities** with declared transfer modes.

6.3 Four transfer modes

CharlotteOS defines four transfer modes, enforced by the MMU:

6.3.1 Copy

The receiver gets a byte-for-byte duplicate. The sender retains the original, unmodified.

- **Use when:** the data is small, the sender needs to keep using it, or the receiver is not trusted with the original.
- **Kernel work:** allocate new physical frames, copy the contents, map them into the receiver's address space.
- **Semantics:** simplest; both domains can read their respective copies independently.

6.3.2 Move

Ownership transfers to the receiver. The sender loses all mappings and all capability rights. The receiver becomes the new owner, responsible for subsequent transfer or destruction.

- **Use when:** the sender is finished with the data and wants to hand it off without copying.
- **Kernel work:** validate sender ownership, prepare receiver mappings, remove sender mappings, perform TLB invalidation, commit ownership, publish the IPC message.
- **Semantics:** physical pages never move. Mappings change. After the move, any attempt by the sender to access the (now unmapped) virtual addresses faults.

6.3.3 BorrowRead

The receiver gets a temporary read-only mapping. The sender retains ownership. The borrow is revoked when the receiver replies (or the call is cancelled, or the server dies).

- **Use when:** the receiver needs to inspect data but must not modify it.
- **Kernel work:** map the pages into the receiver's address space with read-only permissions. Track the borrow so it can be revoked.
- **Semantics:** writable aliases in the receiver are forbidden by the page-table permissions. The receiver cannot write even through unsafe code. Revocation at reply time unmaps the pages.

6.3.4 BorrowWrite

The receiver gets a temporary exclusive writable mapping. The sender's access is revoked for the duration of the borrow. When the receiver replies, sender access is restored.

- **Use when:** the receiver needs to write into a buffer the sender will later read (e.g., a read-into-user-buffer pattern). This is *mutable lending*.
- **Kernel work:** map the pages writable into the receiver, temporarily revoke (unmap or read-protect) sender access, track the borrow for reply-time revocation.
- **Semantics:** the hardware enforces mutual exclusion. The sender cannot access the pages while they are lent out. Unsafe Rust can violate this — the MMU will fault.

6.4 Why hardware enforcement matters

The transfer modes are enforced by page tables, not by Rust's type system:

- A C program running in a protection domain cannot violate the constraints any more than a Rust program can.
- A buffer overrun cannot read pages that are not mapped.
- An unsafe block cannot write to a `BorrowRead` buffer.
- A use-after-drop cannot resurrect a moved memory object.

Rust's ownership complements the kernel enforcement — it catches errors within a domain at compile time. But the kernel is the ultimate guarantor for cross-domain safety, so non-Rust or buggy-Rust processes are still contained by page tables and capability checks.

6.5 Control plane versus data plane

Endpoint IPC belongs to the **control plane**: it configures connections, carries opcodes, sets up device queues.

Memory objects belong to the **data plane**: they carry the actual bytes (packets, blocks, buffers, strings).

For networking specifically:

- Control plane — endpoint IPC configures NIC queues, sets MTU, registers buffer pools.
- Data plane — descriptor rings coordinate producer/consumer state; packet payloads move as memory objects.

This separation avoids turning every packet into a copied IPC payload while preserving strict ownership.

6.6 DMA as a third ownership dimension

When a device performs DMA, a memory object enters a third ownership state:

- CPU ownership — the CPU reads and writes the pages normally.
- DMA ownership — the device is reading or writing the pages; the CPU must not access them.
- Rust/API ownership — which domain holds the capability.

The AArch64 SMMUv3 path tracks an explicit DMA pin per mapped memory object. The kernel validates access direction, installs only the object's frames in a device-private translation context, and releases the pin only after a synchronous IOTLB invalidation. Drivers receive IOVAs rather than physical addresses. CPU mappings are not automatically revoked at the raw syscall layer, so coherent descriptor rings remain an explicit driver protocol. Ordinary Rust applications should instead use `catten_rt::owned`: calling `unmap` on `MappedMemory` consumes the CPU mapping, and only the resulting `OwnedMemory` can create a `DmaTransfer`. The memory cannot be mapped or exposed as a Rust slice again until the transfer's `finish` method has synchronously removed the IOMMU mapping.

6.7 Why memory objects matter for critical software

1. Page transfer without payload copying — for aligned, page-backed objects, ownership can move by changing mappings rather than copying bytes. Whether this is faster depends on payload size and

mapping/TLB costs.

2. Cross-domain stale access is constrained by mappings — when a memory object is moved, the sender's mappings are removed. Any later access through those mappings faults. When a borrow is revoked, the borrower's mappings are removed. This does not rule out bugs in unsafe code, page-table maintenance, or DMA.
3. Borrows have defined lifetimes — a borrow is always tied to an IPC call. When the call completes (reply, cancellation, timeout, or server death), the borrow ends. There is no "I forgot to return it."
4. DMA ownership is tracked on the reference platform — the AArch64 SMMUv3 path pins mapped memory and restricts each requester to its DMA domain. The ownership-aware Rust API prevents ordinary CPU mappings from overlapping a transfer; raw coherent-ring implementations remain responsible for volatile/atomic access and the device protocol. Other IOMMU platforms remain unsupported.
5. Hardware enforcement, not language enforcement — the page tables are the ultimate arbiter. No amount of unsafe code, C FFI, or compiler bug should access CPU pages that are not mapped, assuming correct MMU setup and TLB maintenance. The reference SMMUv3 path provides the corresponding DMA boundary for explicitly mapped objects.

7 Completion Queues and Non-Lossy Results

A completion queue (CQ) carries asynchronous kernel-operation results to userspace. Its central safety rule is that a full userspace ring must not silently discard a terminal result.

7.1 When to use a completion queue

Use submission/completion when the operation belongs to the **kernel or a hardware-facing mechanism**, not to another userspace service:

- Timer expiry.
- DMA completion (disk read finished, packet transmitted).
- IRQ-derived device completion (data available on UART).
- Asynchronous page operations.
- Waiting for thread or process exit.
- Kernel-managed asynchronous object work.

Use endpoint IPC (Chapter 5) when one protection domain invokes another. Confusing these two paths leads to architectural mistakes: a service call is not a kernel operation, and a reply token is not a completion record.

7.2 The CQ ring — a shared-memory completion buffer

A completion queue is a **shared-memory ring buffer** between the kernel (producer) and userspace (consumer). A single 4 KiB page contains a header and a circular array of fixed-size entries.

Each entry is 32 bytes:

```
struct CompletionEntry {
    operation: u64,    // operation identifier (e.g., timer id)
    cookie: u64,       // opaque value set by the submitter
    status: u32,       // completion status code
    flags: u32,        // per-entry flags
    result: i64,       // operation result (e.g., bytes read)
}
```

The ring page is mapped read-write in userspace because the consumer advances its tail; the kernel accesses the same frame through its privileged mapping. The ABI assigns each side its own fields:

- Userspace drains entries without syscalls — advancing the consumer tail is a store to a shared page; reading entries is a load. No kernel crossing needed for the common fast path.
- The kernel publishes entries with a release fence — the consumer sees coherent data without additional synchronization.
- Overflow is detected, not hidden — an overflow counter in the header lets userspace detect when entries were produced faster than consumed.

7.3 The non-lossy invariant

> **A terminal operation result must never be silently lost because the userspace CQ ring is full.**

This is an architectural invariant, so when the ring is full, the kernel has several options:

- Retain overflow in a per-domain kernel backlog — entries that cannot fit in the ring wait in a kernel-side buffer. When userspace drains the ring, the backlog drains into it.
- Stop accepting new submissions — the kernel may return `WouldBlock` on `submit`, applying back-pressure at the submission point rather than at the completion point.
- Propagate `WouldBlock` — the CQ's wait operation can indicate that the ring is full and must be drained.

The overflow counter is pressure telemetry: it tells the consumer that it is falling behind.

In a critical system, a lost completion for a disk write or a timer expiry is a correctness bug with potentially severe consequences. CharlotteOS treats each completion as a datum that must be delivered at least once.

7.4 Per-shard CQ partitioning

Each address space owns a set of completion queues, keyed by `CqId`. Queue 0 is the default. For a service with multiple sitas shards, **per-shard CQ rings** enable targeted wakes:

- Shard A's executor waits on CQ 0.
- Shard B's executor waits on CQ 1.
- A completion destined for Shard A is posted to CQ 0.
- Only Shard A's executor is woken; Shard B is undisturbed.

This prevents cross-shard completion stealing (a pathological behavior where one busy shard drains entries intended for another) and keeps wakeup latency proportional to the number of entries, not the number of shards.

7.5 Operation lifecycle

Every asynchronous operation transitions through a defined state machine:

```
Created
    -> Submitted
    -> Accepted | Rejected
    -> InFlight
    -> Completed | Failed | Cancelled
    -> ResultObserved
    -> ResourcesReclaimed
```

Each transition is explicit and observable:

Created The operation descriptor exists in userspace but has not been submitted to the kernel.
Submitted `submit()` has been called. The kernel has the operation record but has not yet validated it.
Accepted / Rejected The kernel has validated the operation. It may be rejected immediately (invalid parameters, insufficient rights, queue full) with a synchronous error.
InFlight The operation is being processed. The kernel or device owns any associated buffers.
Completed / Failed / Cancelled A terminal state. The result is posted to the CQ. Buffer ownership returns to userspace.
ResultObserved The consumer has read the CQ entry.
ResourcesReclaimed The operation's capability-table slot has been freed (via `close`).

7.6 Waiting on a CQ — `CQ_WAIT`

When a shard's executor has no ready tasks, it calls `CQ_WAIT` on its assigned queue. The kernel blocks the calling thread until one of three things happens:

1. A completion entry is posted to the CQ.
2. An explicit `CQ_WAKE` arrives (from another shard or the kernel).
3. A deadline elapses (timeout variant: `CQ_WAIT_TIMEOUT`).

The wait is race-free. The sequence is:

1. Inspect the CQ ring — if completions are pending, return immediately.
2. Register a waiter against the CQ's generation counter.
3. Re-inspect the CQ ring — if completions arrived during registration, unregister and return.
4. Block the thread — it will be woken when the generation counter changes (a new entry is posted or a `CQ_WAKE` bumps it).

This is the **single wait point** for the entire shard. Three event sources converge on it:

- Kernel completions (CQ entries from `submit` operations).
- Endpoint readiness (a coalesced wake when messages arrive on a bound endpoint).
- Device interrupts (a coalesced wake from a bound interrupt object).

The shard does not need separate wait operations for timers, IPC messages, and device events. One aggregated wait covers all of them.

7.7 LP affinity for CQ waits

A thread that blocks in `CQ_WAIT` is assigned an affinity LP. When woken, the scheduler returns it to the same LP rather than scanning for the least-loaded one. This has three benefits:

1. Timer events stay on the same LP's queue, eliminating cross-LP migration races (observer-not-found when a completion is posted on a different LP than the one where the waiter was registered).
2. Cache warmth is preserved — the waking thread finds its data in the LP's local caches.
3. Shard-to-LP affinity is maintained — the Sitas shard that blocked on CQ 0 on LP 2 wakes up on LP 2, where its shard-local state lives.

7.8 Operation IDs versus per-operation capabilities

An earlier experimental design returned a `CompletionCap` for every submission. The final ABI uses **operation IDs** for the common path:

- `submit()` returns an `OperationId` (a lightweight identifier).
- The result arrives in the CQ with the same operation ID.
- No capability-table slot is consumed for common operations.
- A first-class operation capability is created only when independent waiting, cancellation, delegation, inspection, or policy requires it.

This avoids capability-table pressure for high-rate operations (timer events, network packet completions) while preserving the capability model for operations that need first-class authority.

7.9 Cancellation and buffer ownership

When a userspace task drops a completion handle or explicitly cancels an in-flight operation:

1. The kernel marks the operation as `Cancelling` but **retains any transferred buffer.** The buffer may still be the target of DMA or kernel access.
2. A terminal `Cancelled` completion is eventually posted to the CQ, returning buffer ownership to userspace.
3. On domain teardown (process exit), outstanding buffers may be leaked rather than freed while the kernel or a device may still touch them. Reclamation goes through explicit operation and device teardown.

Under this contract, a buffer is retained while the kernel or device may still reach it. The safe Rust wrapper consumes the owned value on submission and returns it on terminal completion.

7.10 Why completion queues matter for critical software

1. Non-lossy completions are a correctness requirement — a disk-write confirmation, a timer expiry, or a sensor reading is a fact and losing it is a bug. The backlog retains terminal results when the ring is full.
2. The single aggregated wait eliminates a class of race conditions — a shard that must wait on "timer or IPC message or device interrupt" in a conventional OS must manage three separate wait objects, with potential lost-wake races between checking one and arming another. One CQ means one atomic check-and-arm sequence.
3. Per-shard CQ rings enable predictable latency — no shard wakes another shard's executor to deliver a completion. Wakes are targeted, bounded, and proportional to the actual work.
4. The CQ ring is zero-syscall in the common case — the fast path — drain entries, process results, loop — requires no kernel transitions. Only when the ring is empty and the shard must block does a syscall occur.
5. Cancellation is defined, not best-effort — every cancellation path ends in a terminal completion. There is no "the buffer might or might not be freed" ambiguity. The ownership contract is explicit and testable.

8 Shards and Logical Processors

CharlotteOS separates the hardware execution context (logical processor, or LP) from the userspace ownership domain (shard).

8.1 The logical processor (LP) — the hardware unit of scheduling

An LP is CharlotteOS's representation of a schedulable hardware execution context. In most configurations, one LP equals one CPU core (or one hyperthread). The kernel shards its own state per-LP, meaning each LP has:

- Its own run queue of ready threads.
- Its own timer queue and quantum timer.
- Its own scheduler instance (e.g., `RoundRobin`).
- Its own IPI command queues for receiving work from other LPs.
- Per-LP data structures accessible through `PerLp<T>`.

An LP is a **place**: threads are pinned to an LP at creation and, by default, stay there. Migration is opt-in and restricted.

8.2 The shard — the userspace unit of ownership

A shard is a userspace ownership and execution domain. The rules of the shard are the Sitas discipline:

1. Only the owning shard mutates shard-local state — there is no lock on shard-local data because there is no concurrent access. The executor runs one task at a time on the shard.
2. Cross-shard interaction goes through bounded typed messages — a value that moves from shard A to shard B is sent through a typed channel (`ShardSender<M>`). The sender loses the value; the receiver gains it.
3. Placement is explicit — code that needs to be shard-local is accessed through `ShardLocal<T>`, which verifies at runtime that the calling thread belongs to the owning LP.
4. Observability returns owned snapshots — diagnostics and introspection capture state by value. No borrowed reference into shard internals escapes the shard.

8.3 The usual mapping: one shard ↔ one LP

In the typical configuration:

```
one sitas shard
    <-> one LP-affine userspace thread
    <-> normally one logical CPU
```

The userspace thread and its executor are pinned to one LP. `ShardLocal<T>` state is valid only while that LP's owning thread runs.

The architecture retains the distinction (shard $\neq$ LP) so that more complex configurations remain possible:

- Multiple sharded processes — one LP might host shards from multiple protection domains, time-sliced by the kernel scheduler.
- CPU hotplug — an LP can be taken offline; its shards can be migrated (when migration-safe) or torn down.
- Oversubscription — a test can run N shards on $M < N$ LPs to exercise migration and scheduling paths.

8.4 `PerLp<T>` — interrupt-safe shared data

For kernel data that must be accessible from multiple LPs (including from interrupt context), CharlotteOS uses `PerLp<T>`: a boxed slice of per-LP cells, each wrapped in a lock. Access is:

```
per_lp_data.with(|lp_data| { lp_data.do_something(); });
```

The `with` closure acquires the lock for the current LP, calls the closure, and releases the lock. This is interrupt-safe (the lock masks interrupts during the critical section on architectures where that is necessary).

Use `PerLp<T>` for data that is:

- Accessed from interrupt handlers (timers, device IRQs).
- Modified by an IPI from another LP.
- Part of the scheduler, memory allocator, or other kernel subsystem that must be LP-safe.

8.5 `ShardLocal<T>` — lock-free single-owner data

For kernel data that is accessed only from one LP and never from interrupt context, CharlotteOS provides `ShardLocal<T>`: a lock-free per-LP container using `UnsafeCell<T>` with runtime owner-check and borrow-flag guard. Access is:

```
shard_local.with(|val| { *val = 42; });
```

The `with` closure:

1. Checks that the calling thread belongs to the owning LP (panics if not).
2. Checks that the borrow flag is clear (panics on re-entrant access).
3. Sets the borrow flag.
4. Calls the closure with `&mut T`.
5. Clears the borrow flag.

The closure must not:

- Store the `&mut T` reference anywhere (it must not escape).
- Send it to another thread or LP.
- Cross an `.await` point.
- Panic or unwind (the borrow flag would remain set, deadlocking future access).

8.6 Why the distinction matters for critical software

1. Single-owner mutation — the intended shard path does not access shard-local state concurrently. Runtime owner and borrow checks complement Rust's type rules.
2. Placement is deliberate — sending work to a specific LP is an explicit operation (`try_run_on_lp`, `ShardSender::try_send`) and it is not at the mercy of a global work-stealing scheduler. This makes it possible to reason about locality, cache behavior, and latency.
3. The two container types force a design choice — when you reach for shared data, you must decide: is this accessed from interrupt context? Use `PerLp<T>` with locking. Is it thread-only? Use `ShardLocal<T>` without locking. The choice is explicit.
4. Cross-shard communication is typed and bounded — A `ShardSender<M>` carries a specific Rust type (not `Box<dyn Any>`, not a byte buffer). The receiver knows what it will get. The channel has fixed capacity; senders handle backpressure. There is no unbounded message queue that can exhaust memory.

9 The Sitas Executor

The sitas executor drives futures on one shard and parks on the shard's completion queue when no task is ready.

9.1 One executor per shard

Each sitas shard has:

- One executor — a cooperative scheduler that polls ready futures within a budget.
- One ready queue — tasks whose futures returned `Poll::Pending` and whose wakers have since been invoked.
- One timer wheel — futures awaiting deadlines.
- One CQ partition — the completion queue on which this shard waits (`CQ_WAIT`).
- Shard-owned service state — mutable data that only this executor touches.

The executor runs on one LP-affine thread. When that thread is scheduled onto its LP by the kernel, the executor runs. When the thread blocks (no ready tasks), the LP is free to run another domain's thread.

9.2 The polling loop

The executor's main loop is:

```
loop {
    // 1. Poll ready tasks within a budget.
    while budget_remaining > 0 && ready_tasks.not_empty() {
        let task = ready_tasks.pop();
        match task.future.poll(task.waker) {
            Poll::Ready(output) => { /* task done */ }
            Poll::Pending => { /* will be re-awoken by waker */ }
        }
        budget_remaining -= 1;
    }

    // 2. If tasks remain but budget exhausted, re-queue and yield.
    if ready_tasks.not_empty() {
        ready_tasks.requeue_for_next_tick();
    }

    // 3. No ready tasks? Block in the unified wait.
    if ready_tasks.is_empty() && timer_wheel.is_empty() {
        reactor.wait_for_events(); // -> CQ_WAIT
    }
}
```

The budget prevents one task (or a tight loop of always-ready tasks) from starving other tasks on the shard. If the budget is exhausted with work remaining, the executor re-queues the remaining tasks and yields (either to other threads on the same LP via the kernel scheduler, or to idle).

9.3 The unified wait — three event sources, one wait point

When the executor has no ready tasks and no pending timers, it calls `CQ_WAIT` on the shard's completion queue. This is the **only** blocking point in the shard's execution.

Three event sources converge on this one wait point:

9.3.1 Kernel completions (CQ entries)

When a submitted operation completes (timer expired, DMA finished, device interrupt processed), the kernel posts a `CompletionEntry` to the shard's CQ ring and wakes the blocked thread. The executor drains the ring:

```
fn drain_cq(&mut self) {
    while let Some(entry) = self.cq_ring.read() {
        let waker = self.waker_registry.take(entry.user_data);
        if let Some(waker) = waker {
            waker.wake(); // task becomes ready
        }
    }
}
```

9.3.2 Endpoint readiness (coalesced wake)

An endpoint can be bound to a CQ. When a message arrives on the endpoint, the kernel posts a coalesced wake to the bound CQ. The executor, on waking, calls the endpoint's receive path to drain the pending messages.

Coalescing means: if 10 messages arrive before the shard wakes, the kernel delivers one wake, not 10. The executor drains all 10 messages in one batch.

9.3.3 Device interrupts (coalesced wake)

A device interrupt object can be bound to a CQ. When the interrupt fires, the kernel posts a coalesced wake. The executor, on waking, calls the device's interrupt handler (or the driver's polling function).

9.4 Task wakeup — from CQ entry to future poll

When the executor drains a CQ entry, it looks up the registered waker for that entry's `user_data` cookie. The cookie was set by the task when it submitted the operation. The waker marks the task ready:

```
// Inside a task's Future::poll:
fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<u64> {
    match self.cap.poll() {
        Ok(Some(completed)) => Poll::Ready(completed.result),
        Ok(None) => {
            // Register waker so we are notified when this completes.
            self.cap.register_waker(cx.waker().clone());
            Poll::Pending
        }
        Err(e) => Poll::Ready(/* error */),
    }
}
```

The waker is a `ShardWaker` — when invoked, it sets the task's ready flag and signals the reactor (if the reactor is currently blocked) to re-enter the polling loop. The reactor then polls the task, which finds its completion ready.

9.5 Cross-shard messaging without the kernel

Within one protection domain, tasks on different shards communicate through sitas typed mailboxes — not through kernel endpoint IPC:

```
// Shard A: send work to Shard B.
let sender: ShardSender<Command> = shard_b.mailbox();
sender.try_send(Command::Process { data: owned_buffer })?;

// Shard B: receive work.
let receiver: ShardReceiver<Command> = shard_b.receiver();
loop {
    let cmd = receiver.recv().await?;
    match cmd {
        Command::Process { data } => { /* handle */ }
    }
}
```

The `ShardSender/ShardReceiver` pair is a bounded, typed, MPSC channel within userspace. Sending does not cross the kernel boundary. The receiving task registers its waker on the channel; when a message arrives, the waker fires and the task re-enters the executor's polling loop.

This is **internal** cross-shard communication (same protection domain, different shards). External communication (different protection domains) uses endpoint IPC (Chapter 5).

9.6 The ShardParker — blocking on native state

When the executor has no ready tasks, it must block its OS thread. The mechanism is the `ShardParker` trait:

```
trait ShardParker {
    fn park(&self);     // Block the current thread.
    fn unpark(&self);  // Wake the blocked thread (from another shard).
}
```

On CharlotteOS, the implementation is:

```
fn park(&self) { syscall::cq_wait(self.cq_id); }
fn unpark(&self) { syscall::cq_wake(self.cq_id, self.target_lp); }
```

The parker maps directly to `CQ_WAIT` and `CQ_WAKE`. There is no intermediate thread pool, condition variable, or busy-loop. The thread parks on native completion and message state.

This direct mapping is the basis of the "no retrofit tax" claim. The runtime does not need a thread pool to translate blocking calls into futures because `CQ_WAIT` covers the shard's asynchronous sources.

9.7 Why the sitas executor matters for critical software

1. Cooperative scheduling means no data races on shard-local state — within a shard, only one task runs at a time. The executor polls tasks sequentially. There is no preemption within the shard. This means shard-local mutable state never needs locks.
2. The single wait point eliminates a class of race conditions — the executor does not wait on "timer or CQ or endpoint" through three separate blocking calls with three separate check-and-arm sequences. One CQ, one wait, one drain loop. The kernel guarantees the wake is not lost between checking and arming.
3. No thread-pool overhead — the executor uses exactly one kernel thread per shard. When that thread blocks, it blocks on the CQ — the same object that will deliver its wake. There is no need for a pool of threads to convert blocking syscalls into futures.
4. Budgeting prevents starvation — the polling budget ensures that a shard with many ready tasks makes progress on all of them, not just the one that finishes fastest. This prevents priority-inversion-like behavior within a shard.
5. Internal messaging avoids unnecessary kernel crossings — shards within the same process communicate through typed channels in userspace. Only cross-process IPC crosses the kernel boundary. This keeps the fast path fast and the trusted computing base small.

9.8 A worked example: the mailbox index build

The mailbox index build is an end-to-end sitas demo at EL0. It is the `no_std` counterpart of the host-side `sharded_index_mailbox` example: scanners route generated records to logical assembler work units through typed mailboxes; assemblers sort the records; and a coordinator merges and verifies the runs.

The demo is implemented by `sitas_core::mailbox_index` and runs inside `catten-user`, the EL0 sitas image that the kernel's `el0_sitas` self-test loads and verifies.

9.8.1 The workload

The demo uses 1024 synthetically generated records over two shard threads and three logical assembler work units. A record's key is a pure function of its index (a SplitMix64 avalanche over `seed` XOR `index`), so any thread can re-derive any key without storing the data set. An index entry is a `(key, offset)` pair.

- Scanners — two producer threads, one per shard. Each generates its contiguous partition of records, routes every key to an assembler with a key hash, batches the entries, and sends the batches as owned messages.
- Assemblers — three logical work units. Two are placed on shards 0 and 1; the third is deliberately placed on shard 0 so that logical naming and physical placement are visibly separate. Each assembler receives its entries, sorts them into a run, and hands the completed run to the coordinator as an owned value.
- Coordinator — the main thread. It spawns the shard threads, waits for all three runs, k-way merges them, verifies the final index, and writes the verified record count to the status page.

9.8.2 Kernel and userspace responsibilities

The demo separates kernel and userspace responsibilities.

The kernel supplies:

- An address space with mapped user pages (code, heap, CQ ring, status page) and nothing else: no file, network, or mailbox syscalls are used anywhere in the demo.
- `spawn_thread` via SVC, which pins each shard thread to its LP.
- The completion queue: a ring mapped read/write into user memory, with `CQ_WAIT`/`CQ_WAKE` as the only blocking primitive.

Userspace (the sitas `no_std` lane) supplies:

- One `ShardExecutor` per shard, polling its futures and blocking in the unified wait (Section 9.3).
- The typed mailboxes (`ShardSender`/`ShardReceiver`) that carry work between shards without ever crossing the kernel boundary (Section 9.5).
- All of the logic: routing, batching, backpressure, sorting, the merge, and the verification.

The kernel never sees a message, key, or run. It sees threads being spawned and blocking on the completion queue. The division is explicit: the kernel provides authority isolation, thread placement, and wakeups; userspace owns the workload state and message discipline.

9.8.3 The mailbox path

The typed channel carries `IndexMessage`:

```
enum IndexMessage {
    Entries { from: ShardId, batch: Vec<IndexEntry> },
    ProducerDone { from: ShardId },
}
```

Senders are cloned into every scanner; each receiver moves into its assembler shard, and assemblers are started before scanners so receivers exist before producers begin. The `no_std` channel exposes a non-blocking `try_send`; a producer whose mailbox is full parks briefly on the runtime's `ShardParker` and retries, rather than spinning. The assembler task awaits `receiver.recv()`; the await parks the shard in its reactor wait, and the sending side's channel waker re-queues exactly that task and releases the reactor — the same channel-waker-to-reactor chain as the sharded KV service (Section 9.5).

9.8.4 Coordination and joining

The demo coordinates like the sharded KV requester: the coordinator parks and polls the result mailbox until all three runs arrive, then verifies. A parked coordinator consumes no CPU, and a stolen or coalesced wake costs at most one park interval.

Joining shard threads is also real on CharlotteOS now. A shard thread terminates itself with `thread_exit` once its closure completes (a user thread has no "returned" trap, so the trampoline must ask the kernel to reap it), and the kernel's `ObserveThreadExit` syscall registers a completion capability as an exit-observer of that thread: when the kernel reaps the thread, the capability fires through the ordinary completion ABI. The backend's `CharlotteJoinHandle` wraps this — `join` blocks in the kernel's completion wait and then reads the closure's return value from a per-spawn result slot, and `is_finished` polls the capability. The `catten-user` join probe exercises it: it spawns two shard threads that return 40 and 41, joins both, and writes the sum (81) to the status page. The coordinator of the mailbox demo itself does not join because work completion (all runs arrived) already implies the shards are done — but a caller that needs "the thread really exited" has the kernel-backed primitive.

9.8.5 Verification

Verification checks the product, not the implementation path:

1. Every run is internally sorted.
2. A k-way merge over the runs is globally sorted and contains all 1024 records.
3. Every entry's key equals the deterministic key of its offset, and the offsets form a complete bijection of the record range (checked with an XOR invariant).
4. Exactly one run arrives per work unit.

On success the demo writes 1024 (the number of records) to status-page offset 1 and the join probe writes 81 to offset 2, next to the `basic_kv` result at offset 0. The kernel's deferred verifier polls all three words and reports:

```
[sitas] SUCCESS: basic_kv total_len=3,
mailbox index verified 1024 records,
join probe sum 81
```

A failed demo writes the failure sentinel `0xBADC0DE` instead, which the verifier turns into a failed self-test.

9.8.6 Why this demo matters

The mailbox index build shows the architecture working as designed: mutable state is owned, work is transferred as owned values, shards communicate without locks or kernel crossings, and the kernel stays

a thin substrate. Because the demo is written against the `no_std` `ShardRuntime`, the identical code path runs on a Unix host through `sitas-unix` with a plain `cargo test` and on CharlotteOS through the kernel syscall backend: the runtime is not rewritten or emulated for either side. That is the Rust-affine version of the old division of labour: the kernel contributes the primitives that only it can provide, and the language contributes the discipline that userspace is trusted to enforce.

10 Scheduling and Backpressure

The kernel schedules threads through a block–yield–wake cycle. Bounded ingress propagates pressure, although not every internal queue has a hard memory bound.

10.1 The kernel scheduler

CharlotteOS uses a pluggable scheduler architecture. The system scheduler (`SYSTEM_SCHEDULER`) delegates to per-LP `LpScheduler` instances. The reference implementation is `RoundRobin`:

- A FIFO run queue per LP.
- A per-LP quantum timer (10 ms by default) that drives preemption.
- Operations: `spawn_thread`, `block_thread`, `submit_ready_thread`, `yield_lp`, `abort`.

The scheduler is pluggable: a different scheduling policy (EDF, CFS, fixed priority) can be provided by implementing the `LpScheduler` trait. The rest of the kernel interacts with the scheduler only through the abstract interface.

10.2 The block-yield-wake cycle

This cycle underlies asynchronous operations. Every async primitive in CharlotteOS — `sleep()`, `wait()`, `CQ_WAIT` — is implemented through this cycle.

10.2.1 Block

A thread blocks by calling `block_thread(tid, &observable)`:

1. The kernel marks the thread `Blocked` in the scheduler.
2. The thread's `Waker` is registered as an observer on the observable event (a completion, a timer, an endpoint, or a CQ).
3. The thread is removed from the LP's run queue.
4. `yield_lp()` saves the thread's register state and stack pointer to its `ThreadContext`.
5. The scheduler selects the next ready thread (if any) and restores its context. The blocked thread is suspended at the point of the yield.

10.2.2 Wake

An event fires (timer expired, completion posted, message arrived):

1. The event's observable fires, calling `Waker::notify(tid)`.
2. `notify` calls `submit_ready_thread(tid)`, which marks the thread `Ready` and adds it to its affinity LP's run queue.

3. If the target LP is idle, a reschedule IPI is sent to wake it. If the target LP is already running, no IPI is needed — the thread will be picked up at the next scheduling point (quantum expiry or voluntary yield).

10.2.3 Resume

At the next scheduling opportunity on the target LP:

1. The scheduler selects the woken thread from the run queue.
2. The thread's saved `ThreadContext` (registers, stack pointer) is restored.
3. Execution resumes immediately after the `yield_lp()` call that originally blocked the thread.
4. From the thread's perspective, `block_thread` returned normally and the event has fired.

10.3 Wake coalescing

Wakes are **idempotent**: calling `submit_ready_thread` on an already-`Ready` or `Running` thread is a no-op. This enables coalescing:

- If 10 messages arrive on an endpoint before the shard wakes, the kernel posts one wake, not 10.
- If 3 timer events fire before the LP processes the timer queue, the kernel delivers them in one batch during the next `process_events`.
- The unified shard wait (Chapter 9) aggregates multiple wake sources onto a single thread. Re-waking that thread is harmless.

The rule is: **enqueue before signaling.** The work is placed in a queue (CQ ring, endpoint queue, timer queue) first; then a wake is delivered. The woken thread finds the work when it drains the queue.

10.4 The quantum timer

Each LP's `RoundRobin` scheduler arms a periodic quantum timer (10 ms default). When it fires:

1. The timer IRQ handler sets a context-switch-pending flag.
2. At the next `yield_lp()` (or at the IRQ handler's tail), the scheduler checks the flag.
3. If set and another thread is runnable, the current thread's context is saved and the next thread's is restored.

An `armed` flag ensures exactly one quantum event is in flight at any time, preventing unbounded timer-queue growth from manual yields.

10.5 The deferred reaper

A thread cannot free its own kernel stack (it is executing on it). When a thread exits:

1. `abort()` moves the dying thread from the thread table to a `DEAD_THREADS` list. The thread is extracted from the table without being dropped.
2. The scheduler performs a context switch to the next thread.
3. **After** the switch (on the new thread's stack), `reap_dead_threads()` drops the dead threads, freeing their stacks and other resources.

Deferred reclamation is required because threads own their stacks.

10.6 Load rebalancing

By default, threads are pinned to their affinity LP. Migration is opt-in:

- A thread must be marked `migration_safe`.
- Migration is rejected while the thread holds timer events, endpoint waiters, CQ registrations, or other LP-affine ownership constraints.
- Rebalancing uses a **sustained-imbalance window**: per-LP load is sampled over a runtime-adjustable interval rather than reacting to a single instantaneous queue-depth observation.

This conservative approach preserves cache warmth and eliminates migration races (observer-not-found when a completion is posted on a different LP than the one where the waiter was registered). Stable affinity is the correctness default; migration is an opt-in optimization.

10.7 Backpressure and queue bounds

Backpressure is a design principle. Endpoint and hardware rings are bounded, but not every internal queue currently has a hard memory bound. In particular, the non-lossy CQ overflow backlog may grow until its consumer drains it.

Layer	Bounded object	Backpressure signal
Application request	Service endpoint queue	`QueueFull`
Service shard mailbox	`ShardSender<M>`	`Err(m)` returned
Driver endpoint/ring	NIC descriptor ring	Descriptor exhaustion
Kernel submission	Capability table	`WouldBlock`
Hardware	DMA descriptor ring	Ring full

Each layer has an explicit backpressure policy:

```
enum SendPolicy {
    Try,               // Fail immediately if full.
    Wait,              // Wait for capacity.
    Deadline(Instant), // Wait until a deadline.
    // Merge a pending "data available" notification.
    DropIfCoalescible,
}
```

Rules:

- Terminal completions are never dropped — the CQ backlog guarantees this (Chapter 7).
- Coalescible notifications may be merged — two "data available" signals before the consumer wakes become one.
- Control messages normally fail or wait explicitly — a service registration request that cannot be queued fails with `QueueFull`.
- Data-plane producers should stop before unbounded allocation — a NIC driver should stop accepting TX packets when its descriptor ring is full.
- Emergency paths require reserved capacity — the bounded IPI queue reserves slots for mandatory kernel messages (TLB shootdowns, scheduler commands), ensuring they cannot be starved by user work.

10.8 Why scheduling and backpressure matter for critical software

1. The block-yield-wake cycle is exercised across subsystems — the same mechanism handles sleep, IPC wait, CQ wait, and timer wait. A bug fixed in one path fixes all paths. The cycle is exercised by the async demo and by self-tests that call `wait()`; this is test evidence, not a formal proof.
2. Idempotent wakes eliminate lost-wake races — re-waking an already-ready thread is a no-op. This means the kernel can be aggressive about delivering wakes without worrying about double-delivery.
3. Bounded ingress limits some exhaustion paths — endpoint and descriptor-ring capacities reject excess work with backpressure. Resource quotas and a hard bound for all completion backlog memory are not yet implemented, so this is not a general denial-of-service guarantee.
4. Stable LP affinity prevents migration races — completion delivery is LP-local. A waker registered on LP 2 is signaled on LP 2. There is no window where a completion is posted on LP 1 while the thread is migrating to LP 2 and the observer is not found.
5. The scheduling interface is pluggable — `RoundRobin` is the implemented policy. Other policies would require their own implementation and validation behind the same interface.

10.9 Scheduler observability

The prototype records integer running statistics for each thread at scheduler context switches. A snapshot includes the thread identity and generation, address-space owner, scheduler state, LP affinity, dispatch count, and the count, minimum, maximum, total, and sum of squares of completed on-CPU slices. The active slice start tick is reported separately. The counter frequency in the snapshot header permits conversion from ticks to time.

The accumulator uses integer arithmetic and reports saturation. Mean and sample variance are exported as exact rational components; standard deviation and coefficient of variation are calculated in userspace. This avoids using floating-point/SIMD state in the kernel. Accumulators are mergeable so future hot-path metrics can be gathered per LP rather than through a global lock.

The `THREAD_STATISTICS` syscall is address-space-scoped by default: callers see only their own threads and receive the snapshot as a memory-object capability. At boot the supervisor starts exactly one `observe` EL0 service and delegates a typed system-observer capability through its launch vector. Presenting that capability authorizes a machine-wide snapshot; a fabricated or wrong-type handle fails closed. The service

publishes snapshots through endpoint IPC and registers itself with the node name service. Machine-wide observation is therefore explicit authority rather than an ambient privilege.

An HTTP `GET /metrics` adapter is a reasonable userspace consumer of this IPC protocol, but it is not implemented yet. The present TCP/IP service does not supply the complete server-side bind/listen/accept path needed to host it. See `docs/reference/observability.md` for the wire format, security boundary, and suggested next metrics.

11 Service Architecture

Protection domains contain service faults and authority. Naming and protocol policy remain in userspace; privileged code manages lifecycle and resources.

11.1 Protection domains as failure boundaries

A protection domain (Chapter 3) is the unit of both authority and failure. When a service crashes:

- Its capability table is destroyed, invalidating its handles — client connections to the closed endpoint fail through the IPC lifecycle.
- Its memory mappings are removed — outstanding borrows are revoked; the kernel unmaps the pages from the (dead) borrower and restores the lender's access.
- Its threads are aborted — any thread blocked on a CQ or endpoint is woken with a terminal error.
- Its device grants are recovered — the supervisor regains ownership of any MMIO region and interrupt objects the domain held.

The MMU and capability checks are intended to contain the fault to that domain; kernel, hardware, and DMA defects remain outside this guarantee.

11.2 The supervisor — domain lifecycle manager

The **supervisor** is a kernel-side component that creates and destroys protection domains. It does not understand service protocols, but it does understand ELF images, memory mappings, and capability grants.

Its operations are:

Load Parse an ELF image, validate its segments, create a new address space, map `PT_LOAD` segments at their linked virtual addresses, set up a stack with a guard page.
Grant Populate the domain's config-page with the bootstrap capabilities it needs:

- A connection to the name service (for service lookup).
- For driver domains: an `MmioRegion` capability, an `Interrupt` capability.
- For service domains: an `Endpoint` capability on which to listen.
- A default CQ handle.

Spawn Create the initial thread with the domain's entry point and its affinity LP.
Monitor Register to be notified when the domain exits.
Teardown on exit, reclaim all capabilities, revoke mappings, reconcile outstanding operations, and make the address-space ID available for reuse.

Grants are minted kernel-side by the supervisor and never through a syscall. A driver does not ask the kernel for MMIO; the supervisor grants it at creation time based on the device topology it enumerated during boot.

11.3 The name service — discovery without the kernel

The kernel does not know about service names. A userspace **name service** maps human-readable names to connection capabilities:

Registration A service calls `register("driver.nic.v1", endpoint, access_key)` on the name service. The name service stores the endpoint's identity and opens a connection to it.
Generation tracking Each registration bumps a generation counter. If the service restarts and re-registers with the same name, the new generation replaces the old one. Existing connections bound to the old generation fail with `ServiceRestarted`.
Lookup A client calls `lookup("driver.nic.v1", access_key)`. The name service checks the access key (if the service registered one), then delegates a connection capability to the client — typically with `SEND | CALL` rights only.
Access-key gating A registration with a non-zero access key requires lookers to provide the matching key. This is userspace policy: the name service enforces it; the kernel does not participate.

The name service runs as a `no_std` EL0 service process. Boot starts exactly one node-local instance and the supervisor retains its registry endpoint. Each ordinary service or application receives an attenuated connection to that endpoint through its bootstrap page, so all processes on the node observe the same registry. Registering another name does not create another name-service process. A separate registry is appropriate only for an explicitly isolated namespace, such as a test or a future tenant domain.

The supervisor's endpoint handle remains kernel-private. Possessing a bootstrap connection permits the operations granted on that connection; it does not permit an application to mint arbitrary registry connections or inspect another process's capability table.

11.4 Service restart and recovery

When a service crashes and the supervisor restarts it:

1. Device reset — if the service was a driver, the supervisor resets the device before re-granting it to a new driver instance. The new driver sees a clean hardware state.
2. Generation increment — the name service bumps the service's generation. All old connections are invalidated.
3. Stale connections — clients connected to the old instance receive `ServiceRestarted` on their next call attempt. They re-lookup the service and receive a connection to the new instance.
4. Outstanding operation reconciliation — the supervisor reconciles any operations the old domain had submitted but not completed: cancellations are posted, borrowed memory is returned, DMA buffers are recovered.
5. Fresh bootstrap — the new domain receives a fresh config-page with freshly minted capabilities. It does not inherit the previous instance's capability table, reply tokens, or DMA ownership.

A restarted service receives fresh authority. Tests cover stale connections and operation reconciliation; the status appendix records the verified scope.

11.5 The overall system topology

At steady state, a CharlotteOS system consists of:

- The kernel — capabilities, endpoints, memory objects, CQs, scheduling, interrupt routing.
- The supervisor (kernel-side) — domain lifecycle, ELF loading, device enumeration.
- The node name service (one userspace instance) — the shared registry for service registration, lookup, access-key gating, and generation tracking.
- Service processes — one or more per logical function: NIC driver, TCP/IP stack, filesystem, Raft consensus, application servers.
- Client processes — application code that looks up services and invokes them through capabilities.

Every component beyond the kernel, supervisor, and name service is a replaceable userspace process. A different NIC driver, a different filesystem, a different network stack — these are configuration choices, not kernel modifications.

11.6 Why service architecture matters for critical software

1. Failure containment is architectural — a service fault closes its endpoints and affects clients through defined errors rather than direct memory corruption. Availability still depends on the failed service and its dependants.
2. Recovery follows a defined sequence — the restart sequence (device reset, generation bump, connection invalidation, operation reconciliation) is the same for every driver. A new driver implemented to the same endpoint protocol replaces the old one without any special-case recovery code.
3. Policy lives in userspace — the kernel enforces mechanism (capabilities, endpoints, memory objects). Userspace enforces policy (who can register what name, who can look up what service, how many restarts are allowed, what log level to use). The kernel stays small; policy is auditable and replaceable.
4. The bootstrap contract is explicit and minimal — every domain starts with exactly the capabilities it needs and no more. The config-page contract defines what a domain receives; there is no ambient authority to discover or exploit.

12 Userspace Drivers

CharlotteOS runs device drivers as userspace processes with delegated device resources and endpoint interfaces.

12.1 The problem with kernel drivers

In a monolithic kernel, a driver has access to everything: all physical memory, all interrupt vectors, and all kernel data structures. A driver bug — such as a buffer overflow, use-after-free, or race — can corrupt the kernel and crash the system.

The microkernel solution is to move drivers to userspace. But simply moving them is not enough: a userspace driver that can still map arbitrary physical memory or receive arbitrary interrupts is still dangerous. The driver must be **granted** only what it needs, and the grants must be mediated by the kernel.

12.2 The DriverGrant

When a driver domain is created, the supervisor delivers a `DriverGrant` through the config-page contract:

```
struct DriverGrant {
    device: DeviceCap,        // identity and lifecycle of the device
    mmio: Vec<MmioRegionCap>, // register windows, page-granular
    interrupts: Vec<InterruptCap>, // IRQ sources for this device
    // DMA translation, if an IOMMU is present.
    dma_domain: Option<DmaDomainCap>,
    // Endpoint on which to serve clients.
    bootstrap_endpoint: EndpointCap,
}
```

The driver receives **exactly** the MMIO register windows and interrupt vectors it needs. It cannot:

- Map arbitrary physical memory.
- Receive interrupts not assigned to its device.
- Access other devices' MMIO regions.
- Perform DMA without explicit pinning into its DMA domain.

12.3 MMIO delegation

An `MmioRegion` capability represents a page-granular device register window. The driver maps it into its address space as Device-nGnRnE memory (user-accessible, execute-never, strongly ordered on AArch64; uncacheable on x86-64).

The mapping is a kernel-mediated operation: the driver calls `mmio_map(region_cap, addr)`. The kernel validates the capability, creates the page-table entries with the correct memory attributes, and returns. The driver then performs **direct EL0 MMIO** — load and store instructions to the mapped virtual addresses, with no kernel involvement on the fast path.

The common MMIO access therefore avoids a syscall or context switch.

12.4 Interrupt delivery

An `Interrupt` capability represents a routed interrupt source. The driver binds it to its completion queue:

```
interrupt_bind(int_cap, cq_id);
```

When the interrupt fires:

1. The IRQ dispatcher reads the per-interrupt route table atomically.
2. It masks the source at the interrupt controller (GICv3 on AArch64, x2APIC on x86-64).
3. It atomically bumps a coalescing counter.
4. It pushes a wake onto a deferred queue. The actual `completion::wake` (which takes locks) is performed from thread context by `yield_lp` and the LP idle loop.
5. The driver's shard wakes.
6. The driver reads the device's status register (direct EL0 MMIO) to determine the interrupt cause.
7. The driver processes the interrupt and unmasks the source.

The interrupt path performs an atomic counter update and deferred-wake enqueue; thread context performs the remaining work.

12.5 The reference UART driver

The PL011 UART driver demonstrates the full lifecycle:

- Direct MMIO writes to the transmit FIFO — no syscalls.
- Interrupt-driven deferred reads — the driver retains a reply token for a pending read. When the RX interrupt fires, the driver reads the receive register and completes the deferred reply.
- Cooperative shutdown — on graceful exit, the driver unmaps its MMIO region and closes its interrupt.
- Crash recovery — if the driver terminates without releasing its device caps (panic, infinite loop, killed by supervisor), the teardown path:
  1. Reclaims the MMIO region (unmaps it, returns to the kernel).
  2. Unroutes the interrupt.
  3. Reconciles outstanding operations: any pending reply tokens are invalidated; any borrowed memory is returned to lenders.
  4. Resets the device hardware.

  A restarted instance with fresh grants resumes under a bumped generation.

12.6 DMA and the IOMMU

For DMA-capable devices (network cards, storage controllers), a userspace driver needs more: the ability to tell the device "read from physical address X" or "write to physical address Y." Without an IOMMU, this is a safety gap: a malicious or buggy driver could program the device to DMA into kernel memory or another domain's pages.

The implementation has two platform levels:

- Without IOMMU — the driver is trusted with DMA. It still receives only the MMIO and interrupt it needs, and it still runs in a separate address space, so a CPU-side bug cannot read kernel memory. But a DMA programming bug can violate memory isolation.
- With IOMMU/SMMU — the DMA domain capability binds the device to a specific IOMMU translation table. The driver can only DMA into pages explicitly pinned into its DMA domain. The kernel validates every pin/unpin through the capability. The IOMMU hardware enforces the rest — even a malicious driver cannot DMA outside its domain.

On the implemented AArch64 ACPI path, `dma_map(domain, memory, direction)` returns an IOVA and `dma_unmap(domain, iova)` synchronously invalidates the IOTLB before releasing the memory pin. QEMU's coherent SMMUv3 is boot-tested with the userspace NVMe driver. Each PCI requester receives a private stage-1 context containing only explicitly mapped memory objects and its MSI doorbell page. SMMU event-queue faults are handled by the kernel. Non-coherent SMMUs, ATS/PRI, multiple SMMUs, and non-AArch64 IOMMUs remain unsupported.

12.7 Driver service layering

Complex I/O stacks decompose into separate failure and authority domains:

```
Application
    -> socket endpoint
Network service
    -> packet endpoint
NIC driver
    -> MMIO / DMA / IRQ
Hardware
```

The NIC driver knows about hardware registers and descriptor rings, the network service knows about protocols and sockets, and the application knows about its business logic. A crash in any layer is contained: the NIC driver crashes, the network service sees `ServiceRestarted`, and reconnects to a fresh instance.

12.8 Why userspace drivers matter for critical software

1. CPU-side driver faults are contained — a buffer overflow in a NIC driver is confined by its address space, assuming correct kernel, MMU, and DMA isolation.
2. The privilege model is grant-by-grant — a driver receives exactly the resources it needs. There is no "driver has access to everything because it's in the kernel" default.

3. Direct MMIO keeps the fast path fast — reading a device register uses a direct load rather than a syscall or context switch.
4. Restart follows a defined path — the supervisor handles driver restart (device reset, grant re-issuance, generation bump). Driver authors do not write recovery code.
5. Hardware-isolated DMA is implemented on the reference AArch64 SMMUv3 platform — other IOMMU implementations and advanced SMMU facilities remain future work.

13 Networking

CharlotteOS treats networking as an extension of its message-passing and capability model rather than making TCP/IP the native interface for all communication.

The guiding principle is:

> **Remote IPC is local IPC with a different transport.**

13.1 Why not sockets?

The Berkeley socket API (`socket()`, `connect()`, `send()`, `recv()`) has served the Internet for decades. But it is not the right foundation for a capability-based microkernel:

- Sockets identify hosts, not services — `10.2.3.17:443` tells you where to connect, not what you are talking to. The mapping from address to service is out-of-band (DNS).
- Sockets are byte streams, not messages — message boundaries are lost. Every protocol invents its own framing on top of TCP's undifferentiated stream.
- Sockets carry ambient authority — any process can open a socket to any address. There is no capability check; authority comes from being able to reach the network, not from being granted a connection.
- Sockets couple applications to transports — code written against TCP cannot transparently use shared memory for local communication or RDMA for high-performance communication.

CharlotteOS instead uses layers with separate responsibilities.

The stack below is the target architecture. The current repository contains a userspace virtio-net driver, a frame demultiplexer ("frouter"), a reliable-message service with fragmentation, a distributed name service that replicates a `name → {node, generation}` catalog across two guests, a smoltcp TCP/IP service, and a remote-invocation path through the catalog. The NIC/TCP path is runtime-validated on macOS QEMU TCG (`-relmsg-test`, `-disco-test`, `-dns-test`, `-tcpip-test`), and remote scalar remote-call prototype is implemented (`dns::OP_CALL`); the general remote capability delegation is not.

13.2 The native protocol stack

```
Applications
        |
Capability Invocation
        |
Distributed Objects
        |
RPC
        |
Reliable Message Layer
        |
Ethernet Frame Transport
```

```
        |
VirtIO / NIC Driver
```

13.2.1 Ethernet — generic frame transport

The NIC driver exports raw Ethernet frames. It knows about MAC addresses, MTU, and frame CRCs. It knows **nothing** about IP, TCP, RPC, or services. Its endpoint protocol has two operations:

- `OP_SEND(frame_buffer)` — transmit a frame via a moved memory object.
- `OP_RECV` — deferred receive: the caller retains a reply token; the driver replies with a received frame buffer.

Ethernet is a generic frame transport that can carry IP, ARP, LLDP, custom protocols, or CharlotteOS native messages. The driver provides the frames and higher layers interpret them.

13.2.2 Reliable message layer

This layer provides sequenced, acknowledged, reliable delivery of **messages** (not streams) over the raw frame transport. It handles:

- Sequencing (each message has a monotonic sequence number).
- Acknowledgements (the receiver confirms delivery).
- Retransmission (unacknowledged messages are resent).
- Fragmentation (messages larger than MTU are split and reassembled).
- Batching (multiple messages in one frame, one ACK for multiple messages).
- Congestion control (bounded in-flight windows).

The abstraction exported is a **reliable message**, not a byte stream. This is the building block for RPC, consensus protocols, and distributed objects.

Fragmentation is implemented: a message larger than one frame's payload (≈1460 bytes in wire protocol v2) is split across frames that share the message's sequence number, each fragment carrying its byte offset and a `FLAG_FRAG`/ `FLAG_MORE` pair in the header's reserved bits. The receiver buffers fragments by offset and reassembles the message once the last fragment arrives. The application-level ceiling is `relmsg::MAX_MSG` (65535 bytes).

Wire protocol v2 carries a 64-bit sender-session identifier. Data and acknowledgement frames assert it with `FLAG_SYN`. Observing a new, non-retired session resets the per-peer sequence spaces (including an outstanding transmission), so one reliable-message service can restart while its peer remains running. A bounded retired-session window rejects delayed frames from prior instances. The receive queue and fragment count/bytes are also bounded; a full delivery queue withholds sequence advancement and its acknowledgement, applying retransmission-based backpressure.

13.2.3 RPC layer

The RPC layer provides request/reply semantics over the reliable message layer:

- Synchronous-looking futures — `service.call(opcode, args).await?`

- Asynchronous invocation — the `Future` is backed by a pending RPC call; the shard continues other work while waiting.
- Object references — the target architecture can represent delegated remote authority; general remote capability delegation is not implemented.
- Serialisation — typed protocol crates handle encoding/decoding between Rust types and the wire format.

The implemented DNS v3 remote-call path is narrower than this target model. It currently carries scalar opcode/argument calls. A request is identified by caller node, caller DNS-session generation, and call ID, and a reply is accepted only from the expected peer. The implementation bounds outstanding calls at 64, returns `ERR_UNCERTAIN` after five seconds (execution may already have occurred), and retains 128 completed results to suppress duplicate execution within that window. The replicated catalog assigns a monotonically advancing service generation; requests and replies carry it, and the target rejects stale generations before execution. Remote object-capability delegation remains future work.

Registration uses two replicated phases. A `prepare` command allocates the next generation but remains invisible. After the owner successfully publishes its node-local endpoint, a generation-fenced `activate` command makes the entry visible. Failed local publication therefore leaves an inactive tombstone rather than a remotely resolvable ghost. Automatic replicated unregistration when the hosted service dies remains future work, but explicit unregistration is implemented. Its replicated tombstone must match the owning node and distributed generation. Subsequent removal from the node-local name service must also match the local generation captured before the proposal, so a delayed operation cannot remove a replacement instance.

Lookup and call-routing decisions do not use follower-local catalog state. A follower sends a correlated query to the known leader, and the leader answers only when Graft's quorum-contact read barrier succeeds. Query failure returns a safely retryable `ERR_NOT_LEADER`; a timeout after remote execution may have begun remains `ERR_UNCERTAIN`. Raft time is supplied by a persistent detached-timer completion, preventing sustained endpoint traffic from freezing the lease clock. Catalog and status dumps are diagnostic local observations, not linearizable reads.

13.2.4 Distributed objects

At the top of the stack, applications interact with **capabilities** — the same abstraction they use for local IPC:

```
let payroll: Capability<PayrollService> = lookup("Payroll")?;
payroll.call("calculate_salary", employee).await?;
```

The intended API hides whether a service uses local endpoint IPC or remote RPC. The implemented remote path is limited to the scalar DNS v3 call contract described above.

13.3 TCP/IP as a compatibility service

TCP/IP is supported as a userspace compatibility service, not the native programming model. The frouter owns the NIC's deferred `OP_RECV` and routes IPv4 (`0x0800`) and ARP (`0x0806`) frames to the `tcpip` service's `OP_FRAME` ingress; a smoltcp `phy::Device` adapter pulls those frames and transmits via `OP_SEND`:

```
// smoltcp adapter (receive is fed by the frouter's OP_FRAME ingress):
fn receive(&mut self, now: Instant) -> Option<(IpPacket, ...)> {
    self.rx.pop_front() // frames queued by the tcpip service
}

fn transmit(&mut self, now: Instant) -> Option<...> {
    // Calls OP_SEND with a memory object.
}
```

The TCP/IP stack itself uses smoltcp 0.13. It is wrapped in a userspace service that registers as `"tcpip"` and serves clients through a socket-API protocol (`OP_SOCKET`, `OP_CONNECT`, `OP_BIND`/`OP_LISTEN`, `OP_ACCEPT`, `OP_SEND`, `OP_RECV`, `OP_CLOSE`). Legacy software that requires TCP/IP connects to this service.

13.4 Transport independence

The reliable-message interface is designed to be transport-agnostic. Candidate transports include:

- Ethernet — for inter-machine communication.
- Shared memory — for co-located processes on the same machine, where the kernel can set up shared memory regions instead of going through the NIC.
- Loopback — for same-machine same-stack communication.
- RDMA — for high-performance cluster interconnects.
- PCIe NTB — for backplane communication in multi-socket systems.

The intended RPC layer and everything above it should remain unchanged across transports. This is the same principle that makes local IPC transparent to the application when it moves from same-process to same-machine to different-machine.

13.5 Zero-copy networking

The intended network stack avoids copying whenever practical:

- Receive path — the NIC driver receives a buffer from hardware. That buffer (a memory object) is passed up the stack by transferring ownership — first to the message layer (for reassembly), then to the RPC layer (for deserialization), then to the application. No intermediate copies.
- Transmit path — the application creates a buffer. It is passed down by moving ownership. The NIC driver DMA's it directly from the application's pages.
- Buffer recycling — after the application consumes a received buffer, it returns the (now empty) memory object to the driver's RX pool. The driver reuses it for DMA and the cycle repeats. IOMMU-backed pinning is implemented on the reference AArch64 SMMUv3 path.

This is the memory-object transfer model (Chapter 6) applied to networking.

13.6 Why the networking architecture matters for critical software

1. Remote invocation follows the local call shape — the target API keeps transport choices out of application protocols. This permits:
   - Services can be relocated without application changes.
   - Testing can substitute a local mock for a remote service.
   - A common interface can reduce environment-specific application code, although the network boundary still introduces failures absent from local IPC.
2. Authority is not an address — an application invokes a service capability rather than relying on `host:port` reachability. Network access alone does not mint that capability.
3. Copy avoidance is a design goal — page ownership transfer can avoid intermediate payload copies. CPU mappings are MMU-enforced; DMA isolation is implemented for the reference AArch64 SMMUv3 path. No performance comparison with sockets has yet been established.
4. Message boundaries are preserved — the native abstraction is a message with identity, ownership, metadata, and boundaries. There is no need for an additional stream-framing layer. This can simplify protocol implementations, but does not eliminate parsing or framing defects.
5. TCP/IP is available but not mandatory — legacy software connects to the TCP/IP service. Native software uses the capability-based stack. The transition is incremental: a service can expose both a native endpoint and a TCP/IP socket through the compatibility service.

14 Consensus and Replication

CharlotteOS exposes distributed consensus as a capability service: a replicated state machine reached through the same endpoint IPC used by local services.

The test suite covers local two-node election, single-voter term recovery from NVMe, and cross-machine catalog replication and invocation through `dns`. A separate persistent-storage boot test exercises discovery-led admission through `JOIN`, joint consensus, and `FINALIZE`. The production DNS voter set remains statically launched, and power-loss behavior and generic `raft`-service client commands are not validated end to end.

The core follows the shared Graft model for normalized peer identity, voters and learners, joint-consensus quorums, replicated `JOIN`/`JOINT`/`FINALIZE` commands, leader no-op entries, current-term commit rules, linearizable-read contact quorums, and membership-bearing snapshots. These paths have focused regression tests. The `dns` service exposes client commands end to end through a concrete state machine; membership administration through the generic `raft` service is not yet exposed.

14.1 Raft as a capability service

The Raft consensus protocol provides a replicated, fault-tolerant state machine across a cluster of nodes. In CharlotteOS, each Raft node is an EL0 service process that:

- Registers with the name service as `"raft-<id>"`.
- Obtains connection capabilities to its peers through the name service (or through seed configuration at bootstrap).
- Communicates with peers through endpoint IPC — shared `raft.proto` VoteRequest, AppendEntries, and InstallSnapshot payloads travel in moved memory capabilities. The scalar argument carries the exact protobuf length.
- Uses completion-queue waits for time — a bounded `CQ_WAIT` drives election and heartbeat checks without a detached timer completion or a busy polling loop.
- Reserves a client-command endpoint operation — `OP_CLIENT_COMMAND` is defined and the core can submit, commit, and apply opaque commands. The `dns` service realizes that path with a concrete state machine; the generic EL0 service still does not.

The Raft core (`catten-graft`) is a generic, transport-agnostic library. It implements the `RaftTransport` trait, which abstracts peer communication behind a few methods:

```
trait RaftTransport {
    fn send_vote_request(&self, request: VoteRequest);
    fn send_append_entries(&self, request: AppendEntries);
    fn send_install_snapshot(&self, request: InstallSnapshot);
    fn poll_completions(&self) -> Vec<RpcCompletion>;
}
```

The CharlotteOS implementation of this trait uses endpoint IPC for same-machine peers and the reliable message layer for cross-machine peers (the `dns` transport); the consensus core does not change.

14.2 Intended properties relative to socket-based Raft

14.2.1 Authority, not addresses

Once the cluster is running, peers are identified by name service entries, not `IP:port`. A node that has joined the cluster receives connection caps for its peers through the name service:

```
let peer_1: ConnectionCap = name_service.lookup("raft-node-1")?;
let peer_2: ConnectionCap = name_service.lookup("raft-node-2")?;
```

No IP configuration. No DNS. The authority to communicate with a peer is the capability; without it, you cannot even address the peer.

14.2.2 Transport transparency

The transport is behind the `RaftTransport` trait. A connection capability routes through local IPC (same machine), Ethernet frames (same network), or a future RDMA transport (high-performance cluster) — the consensus core never changes. This means:

- Development and testing can run a multi-node Raft cluster on one machine with local IPC.
- Deployment can switch to cross-machine networking by changing how peers are resolved, not by changing the Raft code.
- The cluster can be heterogeneous: some peers communicate via shared memory, others via Ethernet.

14.2.3 Zero-copy log replication

Locally, log payloads can transfer via Move memory objects: the kernel flips page-table ownership instead of copying bytes. A follower that receives a Move attachment gets the exact same physical pages the leader held — no intermediate buffer, no copy.

The current local transport uses one moved page per RPC and batches the largest AppendEntries prefix whose protobuf encoding fits. This is a 4 KiB transport attachment limit, not a persistent-object or Raft-log limit. A network transport can use different framing without changing the protobuf contract.

14.2.4 Capability-based security

A client invokes the Raft service only if it possesses a connection cap. The service manager attenuates rights: a typical client receives only `CALL` for `OP_CLIENT_COMMAND`, not for `OP_ADD_SERVER` or `OP_REMOVE_SERVER`. There is no ambient authority, no IP-based ACL, and no "any process on the cluster network can propose commands."

14.2.5 Failure isolation

Each Raft node runs in a separate protection domain. A crash in one node does not affect the others. The storage integration test demonstrates recovery of term/vote state across explicit process teardown and

restart. Automatic supervisor restart, recovery of a replicated log within a live quorum, and rejoining a remote cluster remain future integration work. The generic service lifecycle separately demonstrates that stale connections fail and clients can re-lookup a replacement generation.

14.3 Persistence and launch configuration

The service supports three launch policies: in-memory storage, optional persistent storage with an explicit fallback, and required persistent storage. Required mode performs a waitable lookup of the object-store service rather than spinning during boot. Stable object identifiers derived from cluster and node identity hold term/vote state, log entries, and snapshot data; mutations are followed by an NVMe flush.

The node identity, initial seed list, election timeout, and storage policy remain in the launch manifest. They are deployment configuration owned by the supervisor. Active membership is replicated through the Raft log and stored in the snapshot envelope; after recovery, it supersedes an obsolete seed list. Peer discovery follows that active membership through waitable name-service lookups. The ordinary two-node local test remains in-memory so the generic Raft service is testable on a multicore machine without storage hardware. The distinct durability test verifies one node across process restart. The `dns` test separately replicates a name-service catalog across two computers.

14.4 Cluster bootstrap — the seed problem

A new node must contact at least one existing cluster member. CharlotteOS expresses the seed as a service name or capability rather than an IP address.

14.4.1 Static seeds (current, simplest)

Launch-manifest records carry the node and seed-peer service names:

```
node-id = "r4"
peer-id = "r1"
peer-id = "r2"
peer-id = "r3"
```

The node calls `OP_LOOKUP("raft-r1")` to obtain a connection cap. If the seed is on the same machine, the name service returns the cap directly. If remote, the distributed name service resolves it.

14.4.2 Pre-shared cluster capability (small kernel change)

A cluster administrator provisions a “cluster capability” — a connection to a known cluster member — and delivers it to new nodes via the config page. No name lookup required; the node already holds a first-class connection.

14.4.3 Ethernet broadcast discovery (zero-configuration)

On a single Ethernet segment, a joining node broadcasts a frame with a well-known EtherType carrying a "Raft peer discovery" payload. Existing members respond with their service names. The joining node looks them up through the name service to obtain attested connection caps. This works without any seed configuration.

14.4.4 Distributed name service (implemented)

The `dns` service realizes this end state: its `CatalogMachine` is a `StateMachine` implementation on the Raft substrate, and a new node needs only a single seed to discover all peers through the consistent name registry. There remains one process-facing name-service instance per machine; those instances apply the same committed registry state. The `ns` node-local registry and the `dns` cluster catalog remain distinct services.

14.5 The generic StateMachine trait

Raft replicates an opaque command log. The meaning of those commands is provided by a pluggable `StateMachine` trait:

```
trait StateMachine {
    fn apply(&mut self, command: Vec<u8>) -> Result<Vec<u8>, ApplyError>;
    fn snapshot(&self) -> Vec<u8>;
    fn restore(&mut self, snapshot: Vec<u8>);
}
```

Different services are different state machines on the same Raft substrate:

- Distributed name service — commands are "register(name, owner, route, generation)" and "unregister(name, owner, generation)". Unregistration is rejected unless both owner and generation still match the active entry. The owning DNS process watches the local endpoint through a waitable kernel completion. Endpoint death proposes this same fenced tombstone; a follower forwards an authenticated, idempotent request to its known leader and retries across leadership changes. A stale death notification therefore cannot remove a replacement generation. Scalar registration adopts an existing node-local service connection through a non-blocking local lookup. This lets DNS route to and observe the endpoint without granting ordinary lookup clients minting rights; names without a local endpoint remain catalog-only registrations. The state maps names to owning-node identities, routable service identities, generations, and policy metadata. Replicated, it makes service discovery available cluster-wide; this is the implemented `dns` service.
- Distributed lock service — commands are "acquire(lock_id)" and "release(lock_id)". The state is a set of held locks.
- Cluster configuration store — commands are arbitrary key-value writes. The state is a key-value map.

None of these state machines know about sockets, IP addresses, or TCP. They implement `apply()` and `snapshot()`. The Raft substrate handles replication, leader election, and fault tolerance.

Raft RPCs and DNS control messages use one serialized relmsg queue per peer. Because QEMU may acknowledge that queue slowly, consecutive queued AppendEntries requests and their serially ordered responses are coalesced, and heartbeats are generated only at a bounded fraction of the election timeout. Non-Raft control messages retain their relative FIFO order but pass queued, supersedable append traffic. A two-second stop-and-wait retry lease is shorter than the DNS remote-call deadline, preventing one lost heartbeat from holding the peer's only transmission slot long enough to starve service retraction, placement, or invocation traffic.

The DNS process also never waits synchronously for an arbitrary local service while running this reactor. Name lookup and local invocation are retained as bounded pending operations and polled on later turns. Consequently a failed service cannot stop Raft heartbeats or relmsg receive draining. A timeout before execution is reported as a retryable lookup failure; after invocation has been submitted it is reported as an uncertain outcome. Duplicate remote calls are suppressed both while pending and in the acknowledged-result window.

14.6 Relationship to the name service

The node-local `ns` registry and replicated `dns` catalog are separate services. DNS applies committed catalog state and resolves remote routes; NS holds machine-local endpoint capabilities:

- Cluster registrations are submitted through the Raft leader and applied to each DNS replica; local endpoint publication remains an NS operation.
- Service generations are replicated consistently. A restart on node 3 cannot create a split-brain where node 1 sees generation 5 and node 2 sees generation 3.
- Replicated policy metadata and tombstones propagate through the log; local delegation still depends on machine-local authority.

Kernel capability identifiers and endpoint objects are machine-local and are not Raft state. For a local registration, lookup delegates a connection to the local endpoint. For a remote registration, the local name-service instance must return a local proxy connection whose transport reaches the owning node. Raft replicates the metadata needed to choose that route, not the capability object itself.

14.7 Election timers as first-class completions

In a traditional Raft implementation, election timeouts are implemented with `sleep()` or a polling loop:

```
// Traditional: busy wait or sleep
while !timeout_elapsed() {
    sleep_ms(10); // or just spin
}
```

In CharlotteOS, they use the completion system:

```
// CharlotteOS:
let timer_id = submit_timer(timeout_duration)?;
let result = wait_on_cq(timer_id).await?;
// Timer fired; start election.
```

The timer is a kernel-managed operation. The shard parks on its CQ. When the timer fires, the executor wakes, drains the CQ entry, and resumes the election task. No polling loop, no thread dedicated to sleeping.

14.8 Why consensus as a service matters for critical software

1. Consistency primitives can be exposed as services — a replicated service implements `StateMachine` and uses the shared Raft, transport, timer, and storage interfaces.
2. Capabilities, not IP addresses, identify peers — the cluster configuration is a set of capabilities, not a list of `host:port` pairs. A network reconfiguration (new NIC, new subnet, move to RDMA) need not change the replicated peer identities.
3. Timers are first-class and reliable — an election timeout that is lost because the polling loop missed its window is a correctness bug (it can cause spurious leader changes or extended unavailability). Kernel-managed timers use the non-lossy CQ delivery path exercised by the local-election test.
4. Failure isolation can extend to the consensus layer — a Raft node is a userspace process. It can crash, restart, and recover without affecting the kernel or other Raft nodes. The supervisor handles the lifecycle; the Raft protocol handles the consensus.
5. A distributed name service is implemented — the `dns` service runs the `CatalogMachine` state machine on the Raft substrate; all other services that need cluster-wide consistency can use the same `StateMachine` pattern and shared consensus implementation.

15 Persistent Storage

CharlotteOS treats storage as a layered stack of userspace services, each speaking a well-defined protocol through endpoint IPC. The block driver, object store, and filesystem services run in userspace. The kernel still participates through capability, memory-mapping, physical-address, interrupt, and PCI/MSI-X setup paths.

15.1 The storage stack

```
Filesystem ("fs")           ← charlotte-protocol-fs
    |
Object store ("obj")        ← charlotte-protocol-objstore
    |
Block driver ("blk0")       ← charlotte-protocol-block
    |
NVMe controller             ← MMIO + DMA + MSI-X (granted at boot)
```

15.2 NVMe block driver

The NVMe driver is a userspace EL0 prototype implementing the subset needed for QEMU bring-up: controller and namespace identification, queue creation, and basic read, write, and flush operations. It is not a claim of NVMe 1.4 compliance. It receives an `MmioRegion` capability covering the controller's BAR0 register window, an `Interrupt` capability for I/O completion signalling, and a `DmaDomain` capability bound to the controller's PCI requester ID. Queue and data memory are mapped with `dma_map`; the driver receives contiguous IOVAs rather than physical addresses. The kernel pins the backing frames and removes each translation with a synchronous SMMU invalidation before unpinning.

15.2.1 Initialisation

The driver follows the standard NVMe initialisation sequence:

1. Controller reset — disable CC.EN, wait for CSTS.RDY = 0.
2. Admin queue setup — allocate physically contiguous memory for the Admin Submission Queue (ASQ, 32 entries) and Admin Completion Queue (ACQ, 32 entries). Write AQA, ASQ, and ACQ registers. Enable the controller and wait for CSTS.RDY = 1.
3. Identify controller — submit an Identify command (CNS=1) through the admin SQ. Read capabilities, version, and supported features.
4. Set Features (Number of Queues) — declare how many I/O queue pairs the driver intends to use. QEMU's NVMe controller requires this before accepting I/O queue creation commands.
5. Identify namespace — submit an Identify command (CNS=0, NSID=1) to read the namespace size (NSZE) and the active LBA format (FLBAS), which determines the block size.
6. Create I/O Completion Queue — allocate memory, submit a Create I/O CQ admin command. The CDW11 register must set bit 0 (PC = Physically Contiguous) to 1 — this is required by the NVMe specification and enforced by QEMU's device model.

7. Create I/O Submission Queue — allocate memory, submit a Create I/O SQ admin command, associating it with the I/O CQ created in the previous step.

After initialisation the driver registers `"blk0"` with the name service and enters its main loop, blocking on `cq_wait` for endpoint messages and I/O completions.

15.2.2 I/O path

OP_READ receives a BorrowWrite memory attachment from the caller. The driver maps it into the controller's DMA domain, builds a PRP chain from the returned IOVA range, and submits an NVM Read. On completion it unmaps the DMA translation, replies, and lets the kernel revoke the borrow and return the filled buffer to the caller.

OP_WRITE receives a Move memory attachment, maps it for DMA, builds the PRP chain, and submits an NVM Write. On completion the driver removes the mapping and replies; the moved buffer is consumed.

OP_FLUSH submits an NVM Flush command to ensure all previously written data reaches durable media before replying.

15.2.3 Lessons from bringup

The NVMe driver surfaced several non-obvious requirements:

- CDW11 bit 0 (PC) must be 1 — QEMU's NVMe device model requires physically contiguous queues. Without this bit, the Create I/O CQ command returns status 0x4002 regardless of all other parameters.
- SQE layout is exacting — the Command Identifier occupies bits 31:16 of DW0, the opcode occupies bits 7:0. The NSID shares DW0's high bits with DW0 in the first u64. PRP1 is at byte offset 24, CDW10-11 at byte offset 40. Getting any of these wrong produces silent failures.
- Doorbell stride must be derived from CAP.DSTRD — on QEMU this is 0 (stride = 4 bytes), but the architecture supports strides up to $4 \ll 15$ bytes.
- MMIO mapping must cover the doorbell region — the NVMe controller registers occupy offsets 0x0000–0x003F within BAR0, but doorbell registers start at offset 0x1000. A single-page MMIO grant is insufficient — at least two pages are needed.
- On TCG, `cq_wait` is required for VM exits — a pure spin-loop prevents QEMU from processing I/O because the guest never yields. The driver must block on `cq_wait` to let the host process NVMe commands.

15.3 Block device protocol

The block protocol (`charlotte-protocol-block`) is a minimal `no_std` crate defining the endpoint IPC interface between block device consumers and drivers:

Opcode	Name	Description
1	OP_INFO	Query block size and total block count. Scalar call/reply.

2	OP_READ	Read blocks into a BorrowWrite memory object.
3	OP_WRITE	Write blocks from a Move memory object.
4	OP_FLUSH	Flush write cache to durable media.
5	OP_TRIM	Discard a block range (optional).

The protocol is device-independent: consumers speak `OP_READ`/`OP_WRITE` without knowing the underlying hardware. Only the NVMe implementation is currently present; AHCI, virtio-blk, ramdisk, and network block drivers would require separate implementations.

15.4 Object store

The persistent object-store prototype (`charlotte-protocol-objstore`) provides dynamically sized blob storage on top of a block device. Ordinary objects receive monotonically increasing 64-bit IDs; service-owned objects can use stable IDs through `OP_CREATE_AT`. Stable IDs let a replacement Raft process reopen state derived from its cluster and node identity.

15.4.1 On-disk layout

```
LBA 0--1:  checksummed superblock slots
next:      device-sized allocation bitmap
next:      two device-scaled directory banks:
             id | flags | generation | header_lba |
             header_blocks | crc32
data:      object headers and up to 16 data extents per object
```

15.4.2 Current durability boundary

Each object header is versioned and carries explicit header length and block count, object generation, exact and allocated data lengths, data offset, extent information, and a mandatory FNV-1a content hash. The header can grow without moving the data offset contract. The current format records up to 16 extents, so fragmented free space does not require one large contiguous run.

Replacement is copy-on-write. New data, header, and allocation state are flushed before the directory locator changes, and the old allocation is freed afterward. Mount reconstructs the bitmap from validated reachable headers, reclaiming pre-commit allocations. Whole-object transfers use multi-page memory objects and are divided into block requests of at most 512 KiB, so the NVMe request limit is not an object or file limit. The current service API's 32-bit length limits an individual object to 4 GiB and available disk space. Directory records are checksummed and mirrored. Each update goes to the generation-selected bank, and mount chooses the newest valid copy per record; versioned tombstones prevent deleted records from reappearing. This recovers from one torn directory-sector write. Full device-level sudden-power-loss guarantees still depend on truthful flush/FUA behaviour. FNV-1a detects accidental corruption, but is not cryptographic authentication.

15.4.3 Operations

- **CREATE** — allocate a directory slot, return a new object ID.
- **CREATE_AT** — create a caller-selected stable object ID, or report that it already exists.
- **WRITE** — replace the object's content. The caller moves a suitably sized memory object; the store writes it in bounded chunks.
- **SET_SIZE** — set the exact byte length consumed by the next whole-object write.
- **READ** — return the object's data as a moved memory object.
- **DELETE** — mark the directory entry as free, return the extent blocks to the free bitmap.
- **FLUSH** — flush the underlying block device.

15.4.4 The initial service-artifact store

The kernel no longer embeds every EL0 program. The host build produces an architecture-qualified service bundle, blesses every ELF according to `crates/catten-services/artifact-policy.tsv`, and `scripts/make-nvme-image.py` writes one object per logical service into an initial version-3 object-store image. The kernel embeds only the bootstrap set needed to reach storage (name service, NVMe, object store, UART, and the system observer). Other services are resolved by logical name, read through the object-store protocol on first use, identity-checked, and cached for the boot. The ordinary loader independently enforces the ELF and signature policy before mapping.

The signature record is CLS2, an identity-bearing ELF note. Besides signing the complete bytes, it binds the logical service name, artifact class, release and rollback counter, policy flags, signing-key id, and an optional provenance digest. Consequently a validly signed `dns` ELF cannot occupy the `raft` slot. A deployment generation also carries the complete ELF SHA-256, preventing an object replaced under the same logical name from changing already-committed code.

The runner fingerprints the whole blessed bundle. It rebuilds an instance's initial NVMe image when that fingerprint changes, while `-reuse-storage` deliberately retains an older store and emits a staleness warning. `-fresh-storage` always recreates it. This separates development reproducibility from explicit persistence experiments. The image producer refuses an object-id collision rather than silently replacing an earlier service.

15.5 Raft persistence

The Raft service can use in-memory stores, optionally try persistent stores, or require the object store. Required mode waits for `"obj"` through the name service; it does not use a boot-time polling range. Four stable objects per cluster/node namespace contain term and vote state, snapshot metadata, snapshot data, and serialized log entries. Mutations write the object and flush the underlying NVMe device before returning.

The ordinary two-node local election test deliberately uses memory stores, so the generic consensus service remains testable on a multicore machine without NVMe. A separate storage test starts a single voter with required persistence, records its elected term, tears down that process, restarts the same identity, and requires the new election term to be greater. A repeated term 1 would fail, so the test distinguishes recovery from a fresh in-memory start.

Launch configuration—node identity, peers, election timeout, cluster name, and storage policy—remains in the supervisor-provided manifest. The persistent objects hold Raft protocol state, log entries, and snapshots.

Cross-machine catalog replication is provided separately by the `dns` state machine described in Chapters 14 and 17; this generic Raft persistence test remains a single-voter restart test.

15.6 Native filesystem

The filesystem (`charlotte-protocol-fs`) builds hierarchical naming on top of the object store. Directories and files are stored as objects; the root directory is a fixed object ID (100).

15.6.1 Directory encoding

Each directory object stores entries as a packed sequence:

```
[name_len: u32 LE][name: UTF-8]
[file_id: u64 LE][flags: u32 LE][size: u64 LE]
```

A `name_len = 0` marker terminates the list. Flags distinguish directories (bit 0 set) from files (bit 0 clear).

15.6.2 Path resolution

Paths are walked client-side by chaining `OP_LOOKUP` calls. The filesystem service does not understand path strings — it only resolves single-component lookups within a given directory. This keeps the filesystem protocol minimal: six operations (LOOKUP, CREATE, READ, WRITE, DELETE, LIST) plus FLUSH.

15.6.3 Host inspection

A Python tool (`scripts/fs-inspect.py`) reads the raw disk image and walks the object store and filesystem on-disk structures. It understands both v3 directory banks and multi-extent headers, selects the newest valid directory record, checks CRCs and allocation overlap, compares reachable blocks with the bitmap, and verifies object contents against their FNV-1a hashes. It provides tree, dump, raw, cat, object-list, metadata, information, and format commands, enabling post-mortem debugging without booting CharlotteOS:

```
scripts/fs-inspect.py <image> info
scripts/fs-inspect.py <image> objects
scripts/fs-inspect.py <image> metadata
scripts/fs-inspect.py <image> metadata OBJECT_ID
scripts/fs-inspect.py <image> metadata /path/to/file
scripts/fs-inspect.py <image> tree
scripts/fs-inspect.py <image> cat /raft/state
```

Metadata output includes numeric and symbolic object/filesystem flags, generation, header location, logical and allocated lengths, integrity hash, and every extent. Unknown flag bits remain visible. The `format` command creates a blank v3 store and destructively replaces the image contents.

15.7 Why this matters for critical software

1. Storage policy lives in userspace — a bug in the NVMe driver, the object store, or the filesystem corrupts that service's address space, not the kernel. The kernel grants MMIO and interrupts; everything else is userspace.
2. Device-independent protocol — the block protocol is intended to let the object store and filesystem work over other block drivers. Those alternative drivers have not yet been implemented.
3. Crash-safety mechanism under development — process-restart persistence is boot-validated. Copy-on-write object replacement and mirrored directory records recover from one torn directory-record write. Broader power-loss guarantees still require validation of device atomicity and truthful flush/FUA ordering.
4. Capability-based access — a client that holds a connection to `"blk0"` can read and write blocks. A client that does not hold that connection cannot. There is no ambient filesystem namespace to escape.
5. Host-inspectable — the included Python inspector reads the raw disk image and reports its documented object and filesystem structures.

16 Live Service Upgrade

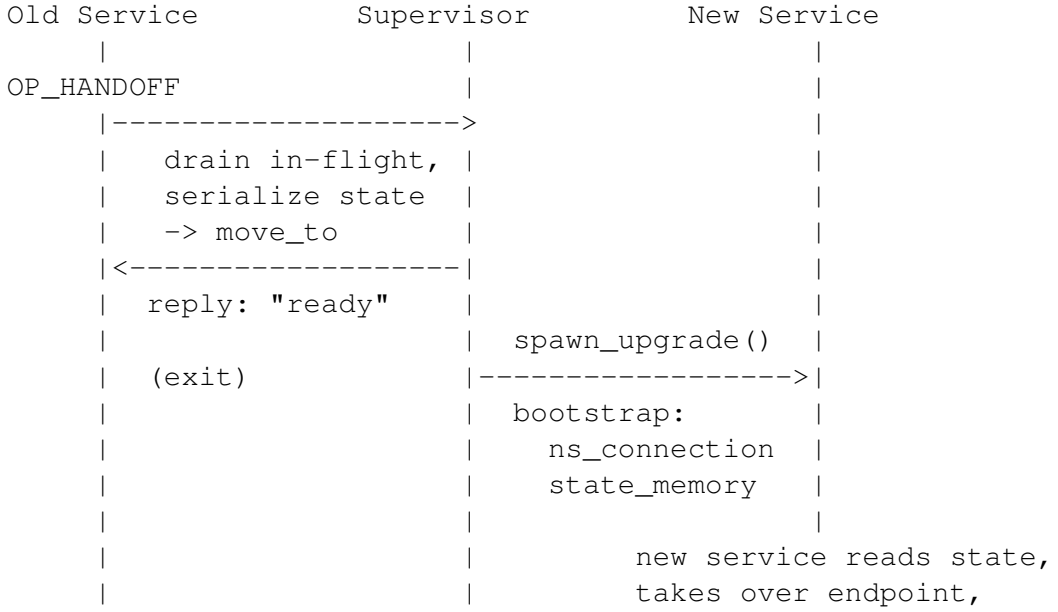
CharlotteOS has a tested prototype for replacing a reference service while transferring its state. It is **restart-with-state**, not zero-downtime: in-flight requests may fail during handoff and clients must re-lookup and retry.

16.1 The four primitives

The prototype builds on four primitives implemented in the kernel and name service:

1. Memory-object ownership transfer — the kernel's `memory::object::move_to` moves a page-backed memory object from one address space to another. The sender loses access; the receiver gains it. This is the vehicle for service state.
2. Generation tracking — the userspace name service records an instance generation that increments on restart. A stale connection returns `EndpointClosed`; a new lookup returns the replacement generation and connection.
3. EndpointClose and Cancelled — when a service domain exits, the kernel closes its endpoint. Queued requests that were never delivered complete as `EndpointClosed`. Delivered calls whose server exited before replying complete as `Cancelled`. Clients see a deterministic failure, not a hung request.
4. Bootstrap capability delivery — the supervisor writes initial capabilities to the config page before a domain starts. For a live upgrade, the bootstrap contract is extended: in addition to the name service connection, the new domain receives the old service's state as a memory object. The current reference replacement creates and registers a fresh endpoint.

16.2 The handoff protocol



```
          |                    |         registers (new generation)
          |                    |                   |
                    clients see EndpointClosed
                    -> re-lookup -> fresh connection
                    -> resume from where they left off
```

The old service receives an `OP_HANDOFF` request (typically from the supervisor or a service manager). It drains any in-flight work, serialises its mutable state into memory objects, moves those objects to the supervisor's address space, and replies "ready" before exiting.

The supervisor then spawns the new service instance with an extended bootstrap contract containing the name service connection and handoff state. The new service deserialises the state, creates a fresh endpoint, and re-registers under the same name. The name service bumps the generation. Teardown of the stopped old domain invalidates its stale client connections.

16.3 Executable source

Embedded service images are bootstrap and recovery inputs. They are not the only executable source. The service manager first tries the object store and reads the well-known executable object for the target service. The returned multi-page memory-object capability and exact byte length are passed to `spawn_upgrade`. If storage is absent or the object is not installed, the manager selects the embedded recovery image.

The kernel snapshots the capability-backed bytes into kernel-owned memory before parsing them, then checks the ELF class, architecture, program-header bounds, segment bounds, page congruence, W^X ("write xor execute")[1], integer overflow, and that the entry point lies in an executable load segment. This prevents a mutable userspace image from changing between validation and mapping.

Executable artifacts carry an Ed25519 CLS2 note that binds the complete ELF to its logical service name, artifact class, release metadata, rollback counter, policy flags, and signing-key identifier. The loader snapshots the bytes and verifies the note before mapping. The FNV-1a object checksum remains corruption detection, not executable authority. Rollback policy, key rotation, and measured boot remain future policy work.

16.4 What the supervisor prototype provides

```
pub struct UpgradeGrant {
    pub state_caps: Vec<MemoryObjectCap>,
}

pub fn spawn_upgrade(
    image: &[u8],
    name_service: &NameServiceHandle,
    old_domain: ServiceDomain,
    grant: UpgradeGrant,
) -> ServiceDomain;
```

[1] a memory page may be writable or executable, but never both simultaneously

`spawn_upgrade` is a privileged kernel operation invoked by the userspace service manager. It validates that the manager holds a callable connection to the target, loads the new service ELF, moves the state memory object to the new domain, writes the bootstrap config page, and starts the new domain. The replacement registers a fresh endpoint and the name service atomically publishes the new connection and generation before retiring the old connection or releasing deferred lookups. The reference path supports both a persistent signed echo image and its embedded fallback, plus one state object. The kernel handoff helper can transfer multiple state objects and rolls a partial transfer back if a later move or endpoint delegation fails. The current reference manager still exercises one object. Rollback policy and manager-owned teardown remain future work.

16.5 What the service must implement

The old service must handle `OP_HANDOFF`:

```
OP_HANDOFF => {
    // Stop accepting new work.
    // Serialize all mutable state into memory objects.
    // Move those objects to the supervisor's address space
    //   (the supervisor passes its ASID in the handoff call).
    // Reply "ready", then exit.
}
```

The new service reads the extended bootstrap contract:

```
fn main(ctx: Context) -> ! {
    let ns_connection = ctx.bootstrap_cap();
    let state_count = ctx.handoff_count();
    let state = ctx.handoff_state_cap();

    // Deserialize state, create and register a fresh endpoint
    // under the same name (NS bumps generation), start serving.
}
```

16.6 What clients see

A client holding a stale connection from the previous generation sees a clean failure and retries:

```
// Before upgrade:
let conn = ns.lookup("payroll")?;       // generation 3
payroll.call(OP_CALCULATE, employee_id); // in-flight

// During upgrade: call completes as EndpointClosed.
// Client retries:
let conn = ns.lookup("payroll")?;       // generation 4
payroll.call(OP_CALCULATE, employee_id); // reaches new instance
```

The new instance receives the old instance's state. The client observes a failed request, performs a new lookup, and retries. Protocols must decide when a retry is safe.

16.7 Restart-with-state vs. zero-downtime

This design is **restart-with-state**: in-flight requests to the old instance fail during the handoff window, and clients must retry. It is potentially suitable for services whose protocols define retry and state recovery. No general latency bound has been measured, and clients must explicitly handle transient failures through name-service re-lookup.

True zero-downtime would require one of:

- Queue migration — the old service's endpoint queue would need to be transferred to the new service atomically, so no request is ever rejected. This is a future kernel feature.
- Sidecar model — the new service starts alongside the old one, both sharing the same endpoint, and the old service stops accepting new work. The kernel does not currently support multiple owners for one endpoint.

16.8 Relationship to crash recovery

The live upgrade path is the graceful variant of the crash-recovery path already exercised by the UART driver test (Chapter 12). In both cases, the supervisor reclaims device authority, invalidates stale connections, and spawns a new instance with a bumped generation. The difference is that live upgrade transfers state explicitly through memory objects before the old instance exits, rather than starting the new instance from scratch.

16.9 Why this matters for critical software

1. The reference path transfers service state — the self-test demonstrates handoff for the reference service. Filesystems, network stacks, and consensus nodes would each need protocol-specific serialization, compatibility, and recovery work.
2. The upgrade sequence is explicit — the old service drains in-flight work, serialises state, and exits. The new service starts and registers. Clients retry. A hard time bound is not currently enforced or benchmarked.
3. Failure recovery has an ownership model — if the new service fails to start, a future supervisor could restart the old service or a fallback. The state memory objects are owned by the supervisor during the transition, so they are not lost.
4. The building blocks are covered by self-tests — memory-object transfer, generation tracking, deterministic teardown, and bootstrap delivery are exercised by the existing self-test suite. This evidence does not establish crash consistency, arbitrary-service compatibility, or production readiness.

17 Server-Class Cluster Vision

This chapter separates the implemented cluster mechanisms from the larger server-class target. The implemented boundary is a two-node QEMU cluster with replicated naming and deployment state, signed ELF artifacts, per-node NVMe object stores, deployment agents, remote invocation, and a stateless assignment handoff. Discovery-bridged membership admission is now demonstrated for a generic Raft group, but the production DNS voter set is still statically launched. Integrating that admission with the durable shared-name-service identity remains future work, as do shared artifact storage, stateful cross-node migration, and automatic placement.

17.1 The vision: interchangeable compute

The target treats servers as replaceable compute units over shared storage. As nodes boot, they join a cluster; deployment and routing address the cluster rather than a particular machine.

This inverts the familiar software deployment model. Instead of deploying software to one or more named servers, software is deployed to a named cluster, and the cluster selects an optimal placement of load among its nodes based on how the deployed components interoperate. Components that interoperate heavily should land on the same cluster node; components that barely talk may be spread arbitrarily. Placement is not a one-component/one-node map. A component explicitly blessed as safe for parallel execution may have several instances, and an affinity group may place instances beside tightly dependent components. Replica anti-affinity and minimum failure-domain rules remain independent: co-location for latency must not accidentally defeat availability. The placement decision is made in and by the cluster itself, from rules it holds in its shared state, not by an operator manually mapping software to machines.

The same idea appears one level down: the shard-per-core model (Chapter 9) treats the cores of one machine as interchangeable executors over owned state, and the scheduler (Chapter 10) moves work between them. This chapter scales that notion to a cluster of machines. A cluster node plays the role of a core; the shared object store plays the role of owned state; and the cluster as a whole is the shared-nothing machine.

17.2 Implemented foundation

The following implemented and verified mechanisms form the foundation:

- Cross-machine consensus — the Graft core replicates the `dns` catalog state machine across two QEMU guests on a stream Ethernet segment, serving remote invocation through `dns::OP_CALL` (Chapter 14). The same replicated catalog now carries the set-once cluster signing key and deployment assignments. Placement rules are not yet part of that state machine.
- Reliable cluster messaging — messages up to 64 KiB are fragmented across Ethernet frames and reassembled at the receiver (Chapter 13). Remote name-service calls, deployment coordination, and the demonstrated assignment handoff run over this transport.
- Persistent per-node object stores — the NVMe-backed object store with generation-selected directory records and copy-on-write replacement (Chapter 15) stores service artifacts and persistent Raft state. The current image producer places the same signed bundle on each node; a networked, interchangeable artifact store is not yet implemented.

- Local stateful replacement — live service upgrade transfers a service's state as a memory object, tracks generations, invalidates stale connections, and re-registers the new instance through the name service (Chapter 16). The two-node cluster test reassigns and restarts a stateless service while preserving its name; transferring mutable service state over the network remains future work.
- Service isolation and routing — services live in separate address spaces with delegated capabilities (Chapter 11), and the name service — replicated across nodes — is the routing table for "somewhere in the cluster".
- Certified work migration — the scheduler migrates threads between LPs with generation-validated handoff and sustained-window load rebalancing (Chapter 10). The per-node precedent for the cluster rebalancer.
- Observation substrate — the sitas executor exposes task counts, poll counts, queue depth, and shard snapshots (Chapter 9). Shard-to-shard message flow can be measured; aggregating those measurements across nodes is the runtime input to placement.
- Signed binaries at the boundary — the EL0 loader validates ELF structure, rejects writable-executable segments, maps images with page-level permissions, and now requires a valid Ed25519 CLS2 note bound to the requested service name (Chapters 15 and 16). Store lookup, administration ingress, and deployment recheck that identity.

17.3 Target node model and current bootstrap

The implemented kernel embeds only five signed bootstrap services: `ns`, `nvme`, `objstore`, `uart`, and `observe`. They are sufficient to establish naming, reach persistent storage, and expose the early console and observation paths. All other staged services are loaded on first use from the signed bundle in the initial NVMe object-store image and cached in kernel memory. Thus service binaries no longer generally have to be linked into the kernel image.

The production target remains a node with no shell, no per-node package manager, and no manually curated software repository. The current `-deploy-test` serial command loop is a kernel-side test and administration shim, not that production management plane.

The current two-node DNS test starts a statically described pair of voters. A separate boot test now drives a discovery-bridged admission into a generic Raft group through replicated `JOIN`, joint consensus, and `FINALIZE`. A production node joining the DNS/shared-state group must connect that demonstrated mechanism to durable node identity and complete three steps:

1. Auto-discovery — the node contacts the cluster through broadcast discovery or a configured seed, following the discovery mechanisms discussed for Raft peers (Chapter 14). Discovery exists as a service, supplies MAC routes and cluster posture, and the generic Raft admission test uses it to locate the anchor. Discovery does not itself grant voting authority.
2. Receiving cluster identity — the replicated catalog can distribute the set-once public signing key. Today that key must equal the build-time bootstrap anchor; blank-start establishment and key rotation are not implemented.
3. Joining the shared state — the node becomes a member of the replicated state machine and begins to receive deployment assignments. The test agents already consume assignments, but no automatic placement controller currently produces them.

The production target may begin without cluster key material. Discovery, elections, and replication can proceed, but executable artifacts cannot yet be validated. An explicit administrative operation would establish

the key through the leader and replicate it as cluster state. The current prototype instead uses a build-time bootstrap anchor; blank-start enrollment and rotation are not implemented.

17.4 Implemented deployment pipeline and production target

Software is an artifact in the object store, and deployment is the act of assigning artifacts to cluster nodes. The demonstrated pipeline implements three stages, with the storage and trust-bootstrap limitations stated below:

1. Blessing and signing — a version-controlled policy admits the artifact and it is signed centrally with the cluster key. Signed metadata binds its logical name, class, release and rollback counter, parallel-instance permission, and an optional SHA-256 link to retained SBOM/build-provenance evidence. The host-side build and `cluster-sign` tooling implement this step; nodes never receive the private key.
2. Validation — each node validates every loaded ELF against the public bootstrap anchor and, in the deployment path, the matching replicated key. The loader, object-store resolver, administration ingress, and agent enforce signature, logical-name, and digest checks. The current set-once ceremony distributes the build-time anchor; it is not yet a general key-enrolment protocol.
3. Assignment — a Raft-committed record instructs a named node to load a specific object id and immutable ELF digest. Its agent fetches, validates, and runs the artifact. At present the object is fetched from that node's locally staged NVMe store, not through a cluster-wide artifact-fetch protocol.

Deployment targets the cluster, and the cluster's routing table — the replicated name service — determines how a deployed program is reached. The implemented catalog has one active owner and one explicit node assignment per artifact. The policy contract can express "somewhere", replicas, and "every eligible node", but realizing those choices and routing among multiple owners remain placement-controller work.

This does not make third-party dependencies disappear. It moves the trust boundary to controlled ingress. A build may contain third-party code, but the resulting self-contained ELF, policy, and provenance reference are blessed once. Nodes trade that immutable artifact inside the cluster; they do not run package managers or fetch executable dependencies from the Internet during placement. Reproducible builds, vulnerability policy, SBOM production, and source attestation remain supply-chain responsibilities outside the kernel.

17.5 Placement: affinity, observation, migration

Placement is the cluster's answer to "which node runs this component?" One part is implemented as a shared validation contract; the controller and runtime feedback loop remain vision.

17.5.1 Declared policy: implemented contract

The intended first controller will use declared interoperation: components that communicate heavily should share an affinity group and be co-located when availability constraints permit. This mirrors shard placement that keeps heavily communicating tasks on the same core.

The shared `PlacementPolicy` contract makes the declaration precise: desired replicas, maximum instances per node, minimum distinct nodes, "every eligible node", affinity group, and anti-affinity group. A policy requesting concurrent replicas is invalid unless the artifact's signed CLS2 metadata includes

`PARALLEL_INSTANCES`. This type and its validation are implemented and host-tested. The policy is not yet stored in the replicated manifest or consumed by a controller: the present state machine still commits one explicit assignment per artifact.

17.5.2 Observation

Declared affinity is a guess. The cluster should observe actual inter-dependency at runtime: shard-to-shard message counts and byte flows are already visible at the executor level (Chapter 9), and the distributed name service can aggregate which components actually call which. Observed behaviour overrides the declared guess as evidence accumulates, in the same way the scheduler's sustained-window rebalancing replaces a static per-LP assignment with a load-observed one (Chapter 10).

17.5.3 Demonstrated assignment handoff and future state transfer

The two-node test already demonstrates a stateless assignment handoff: it starts the signed `greet` artifact on one node, invokes it remotely, reassigns it to the other node, retires the old instance, and verifies that the name remains reachable. Generation fencing prevents the retiring owner from unregistering its replacement.

A stateful cross-node migration will have to extend that protocol with the live-upgrade state handoff from Chapter 16:

1. The cluster instructs the target node to fetch, validate, and prepare the component in its own address space.
2. The old instance exports explicitly owned mutable state and that state is transferred to and accepted by the prepared instance.
3. The new instance registers in the distributed name service, which increments the generation for that name.
4. Clients that resolve the name now reach the new instance; clients holding the old connection observe `EndpointClosed` and re-resolve.
5. Once the name service points at the new instance, the cluster closes the component down on the previous node, reclaiming its resources.

Generation tracking, endpoint invalidation, name-service re-registration, and local memory-object state transfer are verified primitives. Network transport and ownership fencing for mutable state are not yet implemented; the existing cluster handoff starts the same immutable artifact with fresh service state.

17.6 Remaining implementation boundary

The gap between the vision and the current implementation is a set of coordination pieces, all of which sit on top of mechanisms that already exist:

- Dynamic membership integration — the generic Raft service now performs discovery-bridged admission through replicated `JOIN`, joint consensus, and `FINALIZE`. Wiring that protocol into the durable DNS voter set, provisioning the joining node's durable cluster identity, and validating partition/restart and three-voter behavior remain to be built.
- Key injection into cluster state — the current ceremony commits only the build-time anchor and is set-once and idempotent. A genuinely blank start and authenticated key rotation remain.

- Shared artifact availability — the manifest is replicated, but artifact bytes are currently pre-staged independently on each node. Network fetch, repair, and replication policy remain.
- Cluster-wide metrics aggregation — collecting the shard-level observation data from every node into the cluster state, so placement decisions are made from cluster-wide evidence.
- The placement controller — the component that turns declared affinity, observed interoperation, and node capacity into assignments, and that initiates cross-node migration when the observed behaviour justifies it.
- Replica-set realisation and routing — multiple placements, failure-domain enforcement, and selection among several active owners must be added before the parallel-instance policy has runtime effect.
- Stateful cross-node migration — the current handoff proves reassignment, retirement, and name continuity for a stateless service; transferring mutable state with explicit ownership remains.
- Production administration authority — the userspace `clusterctl` protocol exists, but its raw mutation endpoint and kernel serial shim do not yet enforce the intended separately delegated cluster-administrator capability.

Most remaining pieces are userspace coordination. The implemented slice adds one narrow kernel mechanism: only the supervisor-designated deployment agent may start or retire a verified, name-bound ELF memory object. Future work must preserve the same discipline as the current path: explicit ownership, bounded communication, and replicated decisions rather than per-machine configuration.

17.7 Demonstrated two-node deployment slice

A first end-to-end slice is boot-tested on a two-guest cluster through the deployment-test QEMU flow: a signed artifact is deployed to a named node through the replicated manifest, the remote agent picks it up, verifies it, serves it across the network, and the artifact assignment is handed to the other node without losing its name. Both guests finish with zero failed or pending self-tests. This section distinguishes implemented behavior from the simulated administration path.

17.7.1 The replicated deployment manifest

The distributed name service's Raft catalog (Chapter 14) now also carries the **deployment manifest**: an `artifact → {object_id, artifact_sha256, node_key, generation}` map committed through the same log as name registrations. A deployment is therefore ordered and replicated cluster state rather than per-node configuration, and it is generation-fenced like the name catalog. The record carries no separate artifact signature: Raft provides agreement on the record, not caller authorization. The raw mutation endpoint still needs a separately delegated cluster-administrator capability before it is a production plane. `OP_DEPLOY` submits a record; `OP_DEPLOY_QUERY` reads the local replica's applied state.

17.7.2 Cross-node hosting

Hosting a service on a node other than the Raft leader required one new piece: the **remote-register relay**. A service hosted on the follower is registered through the leader (the only node allowed to commit catalog entries), and the leader relays the committed generation back. The two-phase register/activate and the generation-fenced endpoint-death unregister are preserved across the relay, so a retiring host can never clobber a replacement's registration.

17.7.3 The agent

Each node runs an `agent`: it polls the manifest for assignments addressed to its node key, fetches the artifact from the object store, verifies that it is exactly the assigned artifact (SHA-256 identity) and that its signature note validates both the bytes and requested logical name, then invokes a deployment-authority syscall. The kernel snapshots, revalidates, maps, and starts that exact ELF in a fresh address space. The ELF registers its own local endpoint; the agent only publishes it in the distributed catalog and supervises the assignment. When the cluster re-assigns the artifact elsewhere, the agent retires and exits; its endpoint close is observed by the dns, which generation-fences the unregister. Cross-node invocation of the deployed service rides the existing `OP_CALL` relay, so a client on either node reaches the artifact wherever it currently lives.

17.7.4 clusterctl and the admin console

`clusterctl` is the "outside" administration interface. It wraps the dns manifest ops and the object store behind three operations: `upload` (store a note-signed artifact as-is under the artifact's derived cluster-wide id after checking that CLS2 blesses the requested name), `deploy` (hash the stored bytes and submit the pinned assignment), and `status` (query the committed record). A kernel serial console — `-deploy-test` run interactively, without a runner timeout — lets an operator type `upload`, `deploy`, and `status` commands at the QEMU terminal and watch them execute against the live cluster.

17.7.5 Naming

Artifact names are bare cluster-global names ("`greet`"); the object-store id is *derived* from the name (a `0xfffe...` tag over FNV-1a), so every node stores the same artifact under the same id, and the node dimension appears only in the deployment record. This naming scheme may be replaced later.

The host image producer detects and refuses collisions in the complete bundle; production should move from a truncated 48-bit name hash to a collision-resolving or fully content-addressed namespace.

17.7.6 Cryptography: real signatures, honest boundaries

Artifact validation uses real Ed25519 signatures embedded in the ELF itself. The cluster's *private* key never enters the cluster: it is held off-cluster (the `tools/cluster-sign` host tool), and the deployed binary is signed with it before upload. The signature lives in a standard ELF `SHT_NOTE` section (`.note.charlotte-sig`, note type `"CLS2"`). Its descriptor contains the logical name, artifact class and flags, release, rollback counter, signing-key id, optional provenance digest, and the 64-byte Ed25519 signature. The signature covers the SHA-256 of the whole file with only the signature field treated as zeros — a signature cannot cover itself, the same trick PE Authenticode uses — so the signed bytes are deterministic. The matching *public* key is injected into the cluster two ways: it is embedded in the kernel image at build time (the bootstrap trust anchor, handed to services through the launch manifest), and it is committed to the replicated state by the key ceremony (`OP_KEYCEREMONY` → `OP_SET_KEY`). This demonstrates the replicated distribution path that a dynamically joining node can use. The generic Raft admission test is dynamic, while the DNS test voter set remains static and the replicated key must equal the build-time anchor.

The kernel's EL0 loader verifies the note before mapping any domain, and signing is *mandatory*: an image that is unsigned or invalidly signed is refused outright (the live-upgrade syscall fails gracefully; every other load path panics). The build pipeline signs every staged service ELF with the cluster's private key — by

default the version-controlled *development* keypair in `tools/cluster-sign/dev-key.hex`, whose public half is the embedded `CLUSTER_PUBLIC_KEY`; a live key can be substituted at build time via `CLUSTER_SIGN_PRIVATE_KEY` (rebuilding the kernel with its public half embedded). Store resolution and deployment additionally verify the signed name, and the manifest pins the complete ELF SHA-256. The deployed artifact is a real binary — the `greet` service ELF, signed by the build pipeline and staged into the object store — fetched, verified, executed, served across the network, and reassigned through the demonstrated stateless handoff. The deployment record is agreed by Raft, but that agreement is not an authorization proof; the production mutation path still requires the administrator authority described above.

The slice still simulates the hard parts of the vision, and the simulations are the next work items:

- Per-node object store — storage is still per-node: the host image producer stages the blessed bundle on each initial NVMe volume rather than fetched from a cluster-wide store. A networked artifact-fetch path is not implemented.
- Bootstrap-key rotation is absent — establishment now accepts only the build-time anchor and replicated state is set-once. Blank-start trust establishment and authenticated overlapping key rotation are not implemented.
- The development key is version-controlled — every staged ELF is signed with the dev keypair in the repository; the real production key lives at the IT department and would be substituted at build time. The NVMe-ELF upgrade path remains identity- and signature-constrained (Chapter 16).
- The admin is kernel-side — the console is a kernel thread, not a real out-of-band management plane. Its raw mutation path also lacks the intended separately delegated cluster-administrator capability.
- Replica placement is a contract, not a controller — the policy representation and signed parallel-safety gate exist, but the current manifest has one node assignment per artifact and name routing has one active owner. Multiple placements and load-balanced routing remain to be implemented.

17.7.7 What the slice proves

Within these limits, the slice shows the main pieces working together: an explicit node assignment is replicated cluster state; services are hosted cross-node and stay reachable through the distributed name service; generation fencing makes a stateless assignment handoff safe; and deployment and administration exist as protocols. It does not yet prove automatic placement, stateful migration, dynamic reconfiguration of the durable DNS/shared-state group, or production authorisation. It does prove the narrower generic-Raft admission path.

17.8 Related work

The vision draws on historical distributed operating systems, cluster managers, and artifact-trust systems. The comparisons below identify relevant ideas; they are not claims of equivalent guarantees or implementation maturity.

17.8.1 Historical distributed operating systems

- Amoeba (Vrije Universiteit Amsterdam) is the closest ancestor for the "pool of processors" model: terminals submit work that runs on any processor in the pool, dedicated machines act as file and directory servers, and threads communicate by network-wide RPC with capability-based access. Amoeba deliberately did not migrate processes; cross-node migration is this vision's extension of that model.

- Plan 9 (Bell Labs) splits the machine into terminals, CPU servers, and file servers, and centralizes authentication and key management in a dedicated daemon (`factotum`) — an early instance of "the cluster holds the keys, not the node."
- Inferno (Bell Labs) continues Plan 9's model with a portable virtual machine and code distributed as data over the network.
- Jini / JavaSpaces (Sun, later Apache River) contribute the discovery pattern: services find a lookup service by multicast, register, and hand clients a proxy; `JavaSpaces` is a tuple space in which objects are written, taken, and picked up — close to "software uploaded to the object store and picked up by whichever node it is assigned to."
- MOSIX is the single-system-image cluster: automatic resource discovery, transparent process migration, and no need to know where a program runs. Its research program includes communication-aware placement (Keren and Barak, "Opportunity Cost Algorithms for Reduction of I/O and Interprocess Communication Overhead in a Computing Cluster," IEEE TPDS 2003), the earliest an example of communication-aware placement.

17.8.2 Cluster managers and placement products

- Borg / Omega / Kubernetes (Burns, Grant, Oppenheimer, Brewer, and Wilkes, "Borg, Omega, and Kubernetes: Lessons from Three Decades of Platform-as-a-Service," ACM Queue 2016) established the modern shape: software is deployed to a cluster of machines, placement is a cluster decision made from constraints and packing, and machines are interchangeable.
- HashiCorp Nomad and Consul pair cluster scheduling with affinity and anti-affinity constraints and gossip-based membership.
- VMware DRS and vMotion implement the full placement arc: declared affinity rules, load-observed rebalancing, and live migration with a network-level switchover.
- Erlang/OTP distribution provides node discovery, hot code loading, and the "load, register, redirect, retire" upgrade pattern.

17.8.3 Trust, signing, and bootstrap

- TUF (The Update Framework) defines signed metadata that describes which keys are trusted, with versioning and expiration; its root-key ceremony is the model for injecting key material into a blank-start cluster.
- Uptane (automotive software update security) shows the offline and build-time key provisioning patterns — the "bake a secret into the binary at build time, but allow a blank start" option.
- Sigstore / Cosign / Notary provide related artifact-signing approaches for deployment systems.
- Remote attestation and measured boot (TPM, ARM CCA, SEV-SNP) complement cluster-level validation with a hardware root of trust for the node itself.

17.8.4 Membership and artifact stores

- SWIM (Gupta, Birman, and van Renesse, DSN 2002) and its production form **memberlist** provide gossip membership and failure detection for the auto-discovery step.
- Nix is the content-addressed, hash-verified software store: the artifact-identity half of "software lives in the object store."
- OCI container registries with signed manifests are the deployment-time equivalent.

CharlotteOS could use shard-level message flow and executor snapshots (Chapter 9) to inform placement. That controller remains future work.

18 Programming Model

This chapter summarizes the kernel and userspace rules for the asynchronous, shard-per-core model.

18.1 Invariants

The design relies on these rules:

18.1.1 Only the owning LP mutates its state

Per-LP data is accessed only from the LP that owns it. Cross-LP mutation goes through message dispatch (`ShardMailbox<M>`, IPI closure, or IPI RPC), never through a shared lock acquired from a foreign LP.

18.1.2 Messages are owned values

Everything that crosses a shard boundary is an owned value — a typed `M` in a `ShardSender<M>`, an IPI closure, or a memory object. No shared references, no `Arc<Mutex<...>>` as the normal programming model. `Send + 'static` boundaries enforce this at compile time.

18.1.3 References to shard-local state must not escape

`ShardLocal<T>::with(fn)` provides `&mut T` inside a closure. The closure must not store the reference, send it to another thread, or cross an `.await` point. The borrow is verified at runtime (owner-check plus re-entrancy flag).

18.1.4 Work stays on its assigned LP

A thread spawned on LP N stays on LP N. To submit work to another LP, use `try_run_on_lp(target, closure)` or `ShardSender::try_send`.

18.1.5 Observability returns owned snapshots

Never expose borrowed references into runtime internals. Diagnostic and introspection code captures state by value. Self-tests and the demo illustrate this pattern.

18.2 Kernel-side primitives

18.2.1 `PerLp<T>`

Lock-wrapped per-LP container. Use for data accessed from interrupt handlers, IPI handlers, or multiple LPs:

```
// Declaration:
static SCHEDULER: PerLp<RoundRobin> = PerLp::new();

// Access:
SCHEDULER.with(|scheduler| {
    scheduler.submit_ready_thread(tid);
});
```

The lock masks interrupts during the critical section on architectures that require it.

18.2.2 ShardLocal<T>

Lock-free per-LP container for single-LP, thread-only data:

```
// Declaration:
static MAILBOX: ShardLocal<ShardMailbox<Command>> = ShardLocal::new();

// Access (only from owning LP, not from interrupt context):
MAILBOX.with(|mb| {
    mb.try_send(Command::Ping)?;
});
```

Panics on wrong-LP access or re-entrant access.

18.2.3 ShardMailbox<M>

A typed, bounded per-LP queue has cloneable `ShardSender<M>` producers and one `ShardReceiver<M>` consumer per LP, accessed through `ShardLocal`:

```
let sender: ShardSender<Command> = mailbox.sender();
// Returns Err(data) if full.
sender.try_send(Command::Process { data })?;
```

18.2.4 IPI closures

`try_run_on_lp(target_lp, || { ... })`. Boxes arbitrary work and delivers it to the target LP's IPI queue. Returns the closure on full queue (backpressure). Use for kernel-level cross-LP work where typed mailboxes are not appropriate (e.g., TLB shootdown, scheduler commands).

18.2.5 Completion API

The API provides `submit`, `submit_worker`, `complete`, `poll`, `wait`, `cancel`, and `close`. The exit-observer path (`submit_worker`) is the intended default for fire-and-forget async kernel work:

```
let cap = submit_worker(|| {
    // Worker thread: do async work, then exit.
    // Completion fires automatically on exit.
});
let result = wait(cap)?; // Block until worker exits.
```

18.2.6 CQ ring

`CompletionQueueRing` for zero-syscall completion draining. Kernel side: `write(cap, result)`. Consumer side: `read() -> Option<CqEntry>`.

18.3 Userspace programming model

18.3.1 The launch contract

A CharlotteOS userspace program is a `no_std` ELF binary. It uses the `catten-rt` crate (the CRT, or C runtime) and the `entry!` macro:

```
#![no_std]
#![no_main]

use catten_rt::entry;

entry!(main);

fn main(ctx: Context) -> ! {
    // ctx provides:
    // - A connection to the name service
    // - The default CQ handle
    // - Heap configuration
    // - Bootstrap capabilities (device grants, etc.)

    let name_service = ctx.name_service();
    let nic_driver = name_service.lookup("driver.nic.v1")?;

    // ... service logic ...

    loop {
        // Poll tasks, drain CQ, wait for events.
    }
}
```

The program does not define `_start`, a panic handler, or an allocator. These are provided by `catten-rt`. The `entry!` macro wires them together, validates the launch ABI version, sets up the heap, and calls `main(ctx)`. A panic invokes the domain-abort syscall: every thread in the address space is retired, so no sibling can continue after shared Rust state or a userspace lock has been left inconsistent.

18.3.2 Resource ownership in Rust

The register-level syscall ABI deliberately exposes capability handles as integers. Application code should use the linear wrappers in `catten_rt::owned`:

- `OwnedMemory` closes its capability on drop and is neither `Copy` nor `Clone`.
- `MappedMemory<ReadOnly>` and `MappedMemory<Writable>` own both the mapping and capability; only the writable state can produce `&mut [u8]`.
- `DmaTransfer` is created only from unmapped `OwnedMemory` and returns CPU ownership after synchronous DMA unmapping.
- `ReadOperation<'a>` retains its `&'a mut [u8]` until completion, including the cancel-and-wait path in its destructor.
- `Completion` cancels, waits, and closes on drop; timeout does not consume the handle unless a terminal result was observed.
- `Endpoint`, `Connection`, and `PendingCall` close exactly once. A pending call retains any memory loan until reply observation or cancellation.
- `CapabilityVector` owns moved entries and retains read/write loans across a vector call. Submission failure returns the builder intact, including every moved object.
- `MmioRegion`, `MappedMmio`, and `Interrupt` provide single-close device ownership. Register access remains unsafe because width, alignment, volatility, and ordering are device-specific.
- `ThreadHandle` stores both the recyclable TID and the spawn-time generation. Joining passes both values back to the kernel, so a replacement thread cannot satisfy a delayed join.

The raw `catten-syscall` interface remains available to runtimes and drivers. Coherent `dma_map` is explicitly unsafe because the driver must synchronize CPU references and device access itself. Mixing raw operations with an adopted ownership wrapper violates the wrapper's safety contract.

These wrappers have host-side failure injection for unmap, DMA/MMIO release, completion cancellation, IPC move rollback, vector descriptor rollback, and loan cancellation. The no-std `charlotte-lifecycle` crate supplies the pure generation classifiers used by both the kernel and runtime and exercises their bounded input spaces in host tests.

18.3.3 Bootstrap authority

The `Context` carries the domain's bootstrap capabilities:

- `name_service()` — a connection to the name service, for looking up other services.
- `default_cq()` — the CQ handle for submission and waiting.
- `device_grants()` — any `MmioRegion` or `Interrupt` capabilities granted to this domain.
- `heap_config()` — memory available for the allocator.

The `Context` is the only way a userspace program acquires capabilities at startup. There is no ambient authority, no global filesystem, no `/dev` to explore.

18.3.4 Caller identity is kernel authority

Syscalls derive the caller's address space from trusted thread and trap context. Userspace cannot select another domain's address space or capability table.

18.3.5 Backpressure is normal

Bounded queues and capability tables return `WouldBlock`. The correct response is to retry, yield, or propagate backpressure upstream. Do not assume an unbounded queue.

18.3.6 Sitas integration

When writing code that uses sitas on CharlotteOS:

- A sitas shard is an LP-affine CharlotteOS user thread.
- `CharlotteReactor::new(lp_id)` binds a reactor to the calling shard's LP.
- Async operations return kernel-owned completion handles. A completion handle is opaque: wait on it, poll it, close it, or pass it through a typed channel.
- Cross-shard work is an owned message. Prefer sitas channels and futures over shared mutable state.
- The executor parks on `CQ_WAIT` (via `ShardParker`), not on a busy-loop.

18.4 Rule-of-thumb guidance

- **Choose the right container:** `PerLp<T>` for interrupt-accessed or cross-LP data; `ShardLocal<T>` for single-LP, thread-only data.
- **Choose the right channel:** `ShardMailbox<M>` for typed cross-LP communication within the kernel. `ShardSender/Receiver` for typed cross-shard communication within userspace. Endpoint IPC for cross-domain communication.
- **Use `submit_worker` for fire-and-forget async work.** When the worker returns, the exit-observer completes the capability.
- **Register a CQ ring at address-space creation** so completions are visible to userspace without per-entry syscalls.
- **Accept backpressure.** `WouldBlock` or `Err(m)` means the receiver is slow, not that the system is broken. Retry or propagate upstream.
- **Buffers have one owner.** While an async operation owns a buffer, userspace must not mutate or free it. Cancellation still ends in a terminal completion.
- **Never accept ASID or page-table roots from userspace.** Caller identity comes from the trap frame. This is the security invariant.
- **Do not smuggle shared kernel state into userspace.** Shared pages (CQ ring, config page) are data queues with narrow layouts, not references to kernel objects.

18.5 The syscall ABI

The userspace–kernel boundary exposes these operations:

`submit(op_code, buffer) -> OperationId` Start an async operation. Returns immediately. Buffer ownership transfers to the kernel.

`wait(cq_id) -> Vec<CompletionEntry>` Block until a completion arrives on the given CQ. Returns all pending entries.

`poll(op_id) -> Option<Completed>` Non-blocking check. Returns `None` while in flight.

`cancel(op_id) -> CancelState` Request cancellation. May complete asynchronously.

`close(op_id)` Release an operation ID after its terminal completion was observed.

For IPC:

```
endpoint_create(interface, capacity) -> EndpointCap
connection_mint(endpoint, rights) -> ConnectionCap
scalar_send(connection, opcode, arg)
scalar_call(connection, opcode, arg) -> ReplyToken
memory_send(connection, opcode, attachments)
reply(token, result)
receive(endpoint) -> Message
```

The full ABI is described in `docs/architecture/async-syscall-abi.md` and implemented in `crates/catten/src/syscall/mod.rs`.

18.6 Why the programming model matters for critical software

1. **Invariants have enforcement points.** `ShardLocal<T>` rejects wrong-LP and re-entrant access; `ShardSender<M>` moves values across the typed boundary. Unsafe code and incorrect primitive use remain review obligations.
2. **The launch contract is minimal.** A userspace program receives exactly what it needs and nothing more. There is no global state, no inherited file descriptors, no ambient filesystem. The bootstrap capabilities are the program's entire universe of authority.
3. **Messages, not shared memory, are the default.** The safe, preferred path is to send an owned value through a bounded channel. Shared memory with locks is the exception and not the rule. This reduces the number of places where concurrency bugs can hide.
4. **The ABI is auditable.** Userspace wrappers and the kernel dispatcher share the typed `SyscallNumber` enum, and dispatch is exhaustive. The source enum, rather than a duplicated count here, defines the current surface.
5. **The programming model is testable.** Self-tests exercise owner checks, re-entrancy guards, selected backpressure paths, and cancellation behavior. Coverage is substantial but not exhaustive.

19 Development Workflow

This chapter gives the current build, boot, test, and code-reading workflow.

19.1 Prerequisites

- **Rust nightly**, installed via `rustup`. The exact toolchain is pinned in the repository's `rust-toolchain.toml`; do not copy a version from this manual.
- **QEMU** for AArch64 (`qemu-system-aarch64`) or x86-64 (`qemu-system-x86_64`). On macOS, TCG is the default and HVF is opt-in. The virtio-net smoke test runs under TCG; HVF is unsuitable for its direct EL0 device-MMIO path. KVM is an optional Linux accelerator, not a networking requirement. **HVF also does not honor hardware ASIDs** — see the *AArch64 Port Status* write-up — so HVF builds rely on the `hvf_compat` whole-TLB-flush-on-switch fallback. Do not assume ASID-based TLB isolation works when debugging under `-hvf`.
- **Build dependencies:** `cargo`, `just` (optional, for convenience targets in the `Justfile`).

19.2 Build targets

CharlotteOS supports three architectures via custom target JSON specs in `target_specs`:

Target spec	Status
AArch64 custom target	Primary development target; all features and self-tests pass
x86-64 custom target	Boots, core self-tests pass; ring-3 userspace implemented but not yet exercised
RISC-V custom target	Planned, not actively maintained

19.2.1 Kernel build

The commands below use the AArch64 target specification and the x86-64 target specification.

```
# AArch64 (headless, serial console only):
cargo build --package catten \
  --target target_specs/aarch64-unknown-none-catten.json \
  --no-default-features --features acpi

# x86-64:
cargo build --package catten \
  --target target_specs/x86_64-unknown-none-catten.json \
  --no-default-features --features acpi
```

The `display` feature requires a C library for the flanterm framebuffer terminal. Headless boot uses serial output only (AArch64: PL011 UART; x86-64: 16550 COM1).

19.2.2 Userspace service programs

Service binaries are built separately because they target `no_std` / `no_main` with custom EL0 targets that depend on build-std:

```
# EL0 service programs:
cargo build --release --package catten-services \
  --target aarch64-unknown-none.json \
  -Zbuild-std=core,alloc

# Sitas user binary (requires external sitas crates):
cargo +nightly build --package catten-user \
  --target aarch64-unknown-none.json \
  -Zbuild-std=core,alloc
```

19.3 Booting in QEMU

19.3.1 AArch64

The boot entry points are `scripts/boot-smp1.sh`, `scripts/boot-smp2.sh`, and `scripts/run-aarch64.sh`.

```
# Single-LP self-test boot:
./scripts/boot-smp1.sh

# Two-LP self-test boot:
./scripts/boot-smp2.sh

# Or choose the profile, LP count, and timeout directly:
./scripts/run-aarch64.sh debug --smp 4 --timeout 180

# Apple Silicon: request HVF explicitly if suitable:
./scripts/run-aarch64.sh debug --hvf --smp 4 --timeout 180
```

19.3.2 x86-64

The x86-64 entry point is `scripts/boot-x86.sh`.

```
# Requires -cpu max for RDTSCP/FSGSBASE support:
./scripts/boot-x86.sh
```

19.3.3 Expected output

```
CharlotteOS booting...
[init] BSP initialization complete.
[init] Secondary CPUs brought online.
[init] Starting scheduler...
[async] coordinator: submitted worker, awaiting completion...
[async] worker on LP1: sleeping 50ms
[async] worker on LP1: exiting
[async] COMPLETION RECEIVED, result Ok(42)
[async] SUCCESS: async syscall round-trip complete
Testing Complete. All Tests Passed!
```

19.4 Self-tests

The repository combines host-side library tests with boot-time kernel and EL0 self-tests. Host suites cover components such as `catten-graft`, `charlotte-protocol-msg`, and `charlotte-smoltcp`. Boot tests run from `self_test::run_self_tests()` after initialization; assertion or verifier failure fails the boot suite.

Deferred tests register an expected result bit and spawn their verifier through `results::spawn_verifier`. The result subsystem associates that single verifier thread with the test in a fixed atomic table. If the verifier panics, the panic handler uses a best-effort non-blocking scheduler lookup and marks the associated test failed before emitting the panic. Consequently the authoritative reporter names the failed test instead of leaving it pending until the runner timeout. The panic path neither allocates nor waits for a scheduler lock.

Representative boot suites include:

- `completion.rs` — buffer ownership, cancel/deferred-reclaim, backpressure.
- `cq.rs` — CQ ring write/drain/overflow.
- `cq_completion.rs` — CQ ring integrated with completion subsystem.
- `syscall.rs` — dispatch table routing via synthetic TrapFrame.
- `ipi.rs` — bounded queue, backpressure, closure dispatch.
- `shard.rs` — `ShardLocal<T>` owner-check / re-entrancy guard, `ShardMailbox<M>` send/recv.
- `el0.rs` — real-EL0 user thread (AArch64 only).
- `el0_demo.rs` — EL0 userspace smoke test.
- `el0_ipc.rs` — EL0 endpoint IPC between domains.
- `el0_net.rs` — EL0 networking test.
- `el0_pingpong.rs` — bidirectional EL0 IPC.
- `el0_raft.rs` — two-node Raft election over local IPC.
- `el0_service.rs` — EL0 service lifecycle (spawn, IPC, teardown).
- `el0_uart.rs` — userspace UART driver (delegated MMIO + IRQ, crash recovery).
- `el0_sitas.rs` — sitas executor integration: sharded KV, the mailbox index demo, and the thread-join probe (Section 9.8).
- `scheduler_lifecycle.rs` — spawn, block, wake, abort, migration-safe migration, timer-wait affinity.
- `ipc.rs` — IPC path tests (scalar, memory, delegation, reply tokens).

- memory/allocator.rs — heap allocator, stack allocator.
- memory/object.rs — memory object lifecycle tests.
- adversarial.rs — cancellation races, misuse paths, error-path testing.

19.5 The async-syscall demo

crates/catten/src/demo.rs is an end-to-end test of the async model. It:

1. Spawns a coordinator and worker from bsp_main.
2. The coordinator calls submit_worker, which spawns a worker thread and returns a capability that fires when the worker exits.
3. The worker sleeps 50ms (exercising the timer and block-yield-wake cycle).
4. The worker exits; the exit-observer fires; the capability completes.
5. The coordinator (blocked on wait(cap)) wakes and reads the result.

This exercises thread creation, scheduling, timer events, the block–yield–wake cycle, exit observation, and the wait/poll ABI. It is evidence for that path, not a substitute for the wider self-test suite.

19.6 CI pipeline

GitHub Actions (.github/workflows/ci.yml):

1. Checks formatting for the workspace and standalone EL0/tool crates.
2. Runs warning-denied Clippy for the workspace, AArch64 services, and signing tool.
3. Runs host-testable library suites and the signing self-test.
4. Builds AArch64 and x86-64 kernels and the AArch64 service bundle.
5. On pushes to main, boots the four-LP AArch64 QEMU suite and verifies its required success markers.

The workflow runs on pushes and pull requests for its configured branches; the QEMU boot job is restricted to pushes to main.

19.7 Codebase reading guide

For new contributors, the recommended reading order:

1. crates/catten/src/main.rs — kernel entry point, init sequence.
2. crates/catten/src/completion/mod.rs — the completion subsystem: capability table, submit/complete/poll/wait/cancel/close, submit_worker, exit-observer.
3. crates/catten/src/completion/cq.rs — the CQ ring buffer.
4. crates/catten/src/syscall/mod.rs — syscall dispatch table.
5. crates/catten/src/demo.rs — the end-to-end async demo.
6. crates/catten/src/cpu/scheduler — threads, wakers, RoundRobin, block_thread.

7. crates/catten/src/cpu/multiprocessor — SMP: IPI, PerLp<T>, ShardLocal<T>, shard mailboxes.
8. crates/catten/src/cpu/isa — ISA abstraction layer. Start with interface/ (traits), then aarch64/ (reference implementation).
9. crates/catten/src/service — supervisor, ELF loader, bootstrap.
10. crates/catten/src/device_management — device driver framework.
11. crates/catten-services/src/bin — reference EL0 services.
12. crates/catten-rt/src/lib.rs — userspace runtime.

19.8 Troubleshooting

Triple fault during boot Check that the panic handler is working. Early faults before heap initialization rely on early_logln! (direct serial writes). Ensure the serial port is configured correctly.

Blocked thread The block–yield–wake cycle requires that block_thread not clear current_handle before yield_lp saves the thread's context. Verify that the blocked thread is still the current thread at the point of the yield.

ASID-0 collision The first user address space must not receive ASID 0 (which equals KERNEL_ASID). The address-space table must be pre-populated with the kernel AS at index 0.

x86-64 LA57 mismatch Derive the address width from CR4.LA57 (the active paging mode), not from CPUID capability. Limine boots with 4-level paging even on CPUs that support 5-level.

20 Formal Safeguards with TLA+

CharlotteOS uses executable TLA+ models to review stateful functionality. The models state each safety contract, explore action orderings in a finite instance, retain known-bad designs as counterexamples, and map abstract transitions back to Rust.

This is stronger than relying on selected test schedules, but narrower than a proof of the complete operating system. TLC proves invariants of the modeled state machine for the configured finite bounds. The repository supplements that result with an action-level conformance map, Rust regression tests, target builds, Clippy, and boot tests. Each layer answers a different question:

TLA+ model Is the protocol design safe under every modeled action ordering, crash point, retry, and stale event within the configured bounds?
Conformance map Which concrete Rust state and linearization point implement each abstract action?
Rust and boot tests Does the implementation exercise the intended transition on the host and on CharlotteOS targets?
Build and lint gates Does the integrated implementation remain type-correct and warning-free for the actual target?

The models and their configurations live in `docs/tla`. The concise model inventory is in `docs/tla/README.md`; the concrete-to-abstract mapping is in `docs/tla/CONFORMANCE.md`. Those files are maintained with the implementation and are authoritative over the overview in this chapter.

20.1 Why model functionality as a state machine?

Most failures addressed by these models are not computation errors. They are ordering errors: a stale cleanup arrives after a replacement, a thread is removed while it can still execute, a session identifier is reused after a restart, or a joiner forgets which cluster has authority over it. A unit test can exercise one such interleaving. A state-machine model asks TLC to enumerate all reachable interleavings in a deliberately small world.

A TLA+ safety specification has four essential parts:

1. **Variables** record only the state relevant to the property, such as an owner, generation, durable term, queue, or admission anchor.
2. **Init** defines every allowed initial state.
3. **Next** is a disjunction of named actions. Each action defines its enabling condition and complete next-state update.
4. **Invariants** state what must be true in every reachable state.

The common shape is:

```
VARIABLES phase, generation, owner
vars == <<phase, generation, owner>>

Init == ...
```

```
Next == Start \/ Replace \/ Stop \/ Crash \/ Restart
Spec == Init /\ [][Next]_vars

TypeOK == ...
Safety == ...
Invariants == TypeOK /\ Safety
```

The square temporal operator means that every step must be a `Next` step or a stuttering step and that the invariant must hold throughout the behavior. TLC constructs the reachable state graph rather than sampling it. Constants in a `.cfg` file bound identities, queue capacities, generations, terms, and log indices so that this graph is finite.

Small bounds are intentional. Many protocol violations need only two owners, two generations, or three Raft nodes. Exhaustive exploration of that small instance is often more revealing than a large randomized run because TLC systematically covers the orderings rather than hoping to encounter them.

20.2 The safeguarding loop

CharlotteOS applies the models as a feedback loop between design, implementation, and validation:

1. **Choose a safety boundary.** Identify the authority or resource whose invalid transition would be harmful: reply authority, memory ownership, an executing address space, a committed log entry, or a service generation.
2. **Project concrete state.** Remove bytes, addresses, and policy details that cannot affect the selected invariant. Preserve identity, ownership, durability, ordering, and lifecycle phases.
3. **Name the transitions.** Actions use implementation language. Names such as `Receive`, `Reap`, and `Restart` make review against Rust possible.
4. **State the invariant positively.** Examples are "a reaped thread is off-CPU," "a committed entry agrees at every replica," and "a stale generation cannot remove its replacement."
5. **Model crashes and stale work.** Durable and volatile state are separated where restart matters. Delayed messages, old handles, and retries are actions rather than assumptions that the environment is well behaved.
6. **Retain a negative specification.** When a faulty transition is understood, an `Unsafe...` action and configuration are kept. The check suite requires TLC to find the named invariant violation. This proves that the invariant and test harness are capable of seeing the regression rather than passing vacuously.
7. **Map back to code.** `docs/tla/CONFORMANCE.md` records the concrete function, persisted field, and linearization point for every modeled action. Disagreement is resolved by changing the model, changing CharlotteOS, or documenting behavior outside the abstraction.
8. **Validate both levels.** Run TLC, focused Rust tests, target builds and Clippy, and relevant boot or multi-node tests.

The model is an executable safety contract, not a diagram added after implementation. A change is incomplete if the model claims an atomic or durable transition that the code does not provide.

20.3 What is modeled

The current suite contains twenty safe specifications. They are divided by state ownership and failure boundary so that each model remains reviewable. The Raft models are deliberately layered: election, log replication, membership, pre-membership admission, and snapshots have different state and different proof obligations.

Model	Functional boundary	Principal safety properties
`CharlotteIPC`	Endpoint calls, reply tokens, memory move/copy/borrow, cancellation, endpoint close, domain teardown	Bounded queues; unique receiver-visible reply authority; exclusive write borrow; every remaining borrow backed by a live token; no dangling moved-memory capability.
`Charlotte EndpointObservers`	Readiness observers and connection-close watches	Ordinary message arrival cannot falsely signal endpoint closure; closure wakes the appropriate observers.
`CharlotteCQ`	Completion operations, CQ ring/backlog, wait/wake, cancellation, timers	Non-lossy completion tracking; bounded visible ring; no lost wakeup; timer completion remains tied to its operation; a buffered operation retains its kernel loan through cancellation until terminal completion.
`CharlotteTimedWait`	Work-generation publication, delayed wake delivery, and watchdog expiry	A timer cannot report timeout after a new work generation has already been published.
`CharlotteScheduler`	Spawn, admission, execution, blocking, migration, cross-LP abort, reaping, address-space destruction	At most one execution per LP; only the owner LP retires an executing thread; reaping occurs off-CPU; domain abort prevents late sibling publication; destroyed address spaces have no live threads.
`CharlotteThreadJoin`	Recyclable TIDs, spawn generations, delayed exit-observer registration	A captured handle observes only its exact generation; a missing or recycled slot completes immediately.
`Charlotte AddressSpace`	Reusable numeric ASIDs and software-generation handles	A stale handle cannot close a replacement that reused the same numeric ASID.
`Charlotte HardwareAsid`	Hardware-ASID allocation, retirement, cached translations, invalidation	A hardware tag is not reused while stale translations for its former owner remain.
`Charlotte InterruptRoute`	Interrupt binding, queued wake, unbinding, deferred drain	A queued wake cannot be delivered through a later route generation to the wrong owner.
`CharlotteCapability`	Unified per-address-space capability namespace and move rollback	Handles are fresh and authoritative; move revokes the source; rollback restores only the exact transaction; a closed namespace is empty.
`Charlotte Authorization`	Versioned subject/service policy, generation-fenced service bindings, decisions and capability issuance	Only authorized roles mutate policy or publication; decisions are bound to a subject, policy version, service generation and rights; redemption is single-use and cannot amplify authority.

Model	Functional boundary	Principal safety properties
`CharlotteDMA`	CPU mapping, IPC loans, coherent/exclusive SMMU mapping, invalidation, quarantine, driver exit, pin reclamation	Unique stream/domain ownership; exclusive DMA has no overlapping CPU authority, loan, or second pin; mapped memory stays pinned; failed hardware acknowledgement retains rather than prematurely frees authority.
`Charlotte ServiceLifecycle`	Signed artifact staging, loading, start, two-phase publication, lookup, fenced unregister, exit and teardown	Untrusted images never load; only activated generations resolve; stale activation/unregister cannot affect a replacement; teardown follows domain-wide reaping after cooperative exit or domain abort.
`CharlotteRaft`	Terms, durable votes, election, higher-term step-down, crash/restart	A voter records at most one durable vote per term and at most one leader can hold a term.
`CharlotteRaftLog`	Append, conflict repair, replication, commit, crash/restart	Log matching, committed-entry agreement, commit within the log, and leader completeness.
`Charlotte RaftMembership`	Voters/learners, joint consensus, finalization, decommissioning	Both voter majorities are required while joint; all proposed peers reach the joint fence before finalization; leadership and decommissioning follow active membership.
`CharlotteRaftJoin`	Selected-anchor admission before membership, JOIN fence, catch-up, crash/restart	A joiner accepts only the chosen anchor, cannot campaign, is promoted only after catch-up, and restores its durable admission fence after restart.
`Charlotte RaftSnapshot`	Chunk reception, durable installation, matching suffix, membership recovery	Stale snapshots cannot regress progress; the installed state and retained suffix agree; restart restores membership and application state before use.
`CharlotteRemoteCall`	Call identity, target generation, duplicate suppression, timeout uncertainty, result eviction	At most one execution while tracked; stale targets never execute; uncertain outcomes are not reported as success; deduplication evidence remains until transport settlement.
`Charlotte ReliableMessage`	Wire sessions across retry abandonment and unilateral service restart	Logical epochs have unique ordered wire identities and delayed sessions cannot move receive state backwards.

This decomposition also states what is *not* silently being claimed. For example, Ethernet fragmentation and retry timing are not part of the Raft election proof; protobuf bytes and peer addresses are not part of joint consensus; cryptographic correctness is not part of the service-lifecycle trust-bit abstraction. Such omissions are recorded in the conformance map.

20.4 How model reasoning changes CharlotteOS

The most valuable result is often a mismatch discovered while constructing the model. The correct response depends on which side was wrong: sometimes the manual or model overstated functionality; sometimes the code lacked a safety fence.

20.4.1 Authorization is controlled capability issuance

The current name service has an optional reusable 64-bit access-key gate. It does not identify a stable principal, express different rights per caller, separate policy administration from service publication, or version a policy decision. It is therefore not the production policy service suggested by the architecture.

The authorization model makes the intended boundary explicit. The catalog publishes a service generation and the maximum rights that may be delegated. A separate logical policy role maps a stable principal and service identity to an allowed-rights set and policy version. A decision is bound to the kernel-authenticated subject, requested rights, current service generation and policy version, and can be redeemed once for an attenuated connection. A co-located name/policy process can perform decision and redemption as one serialized transition; a later split service needs an unforgeable single-use grant and must revalidate every binding at redemption.

The model also prevents an attractive but false claim. A policy update blocks new issuance and invalidates unredeemed decisions, but cannot selectively retract a direct connection already delegated by the kernel. Hard revocation requires endpoint closure, a revocable proxy, or a future kernel lease object. This prospective revocation contract is safer than documenting authority that the implementation cannot enforce. The practical design is recorded in `docs/architecture/authorization-policy.md`.

20.4.2 Two-phase service publication

An earlier lifecycle abstraction treated replacement publication as one atomic swap. The implemented distributed catalog instead performs three observable steps: commit an inactive prepared generation, publish the local connection, and commit activation. Lookup intentionally excludes the inactive generation, so replacement has a temporary unavailable interval.

The model was changed to describe that real contract rather than claim continuous availability. It also exposed the code-level obligation that generation allocation must fail closed. Distributed generation saturation and local unchecked increment were replaced with checked allocation that returns an error without mutating the catalog. A delayed unregister is accepted only when both owner and generation still match, which preserves a replacement.

Formal safeguarding does not promise every desired availability property. Here it prevents the documentation from claiming atomic replacement and protects the generation safety that the implementation does provide.

20.4.3 Reliable-message session identity

The reliable-message service formerly initialized a wire session from the service generation and incremented that number when abandoning a retry epoch. Generation N after one abandonment could therefore use the same identity as generation $N + 1$ before abandonment. A bounded retired-session window also allowed a sufficiently old delayed SYN to replace newer receive state after it fell out of the window.

The session model represents a session as the ordered pair $(serviceGeneration, retryEpoch)$. The repaired wire encoding packs those values into disjoint 32-bit fields, accepts a replacement session only when the packed identity is well formed and strictly newer, and disables sends rather than wrapping at exhaustion. Two unsafe configurations retain the old flat-identity collision and receive-regression behaviors as required counterexamples.

20.4.4 Restart-safe Raft admission

Dynamic admission originally kept `joining` and `joining_from` only in process memory. A node could select an existing cluster as its authority, crash before membership became durable, restart from its launch-time singleton configuration, and campaign. The gap represented real CharlotteOS functionality, not merely a missing model action.

The repaired implementation persists a `JoinAdmission` record containing the selected anchor and the snapshot index that existed before admission. Construction restores the joining posture and disables elections. When the joint membership commits, the node first writes a newer membership-bearing snapshot and only then clears the durable admission record. This ordering covers both crash edges:

- before the membership snapshot is durable, restart retains the anchor fence and the node cannot campaign;
- after the snapshot is durable but before the fence-clear write, restart recognizes the strictly newer membership snapshot as completion evidence and safely clears the stale admission record.

Auto-join is disabled when the generic Raft service falls back to volatile storage. That is an explicit safety-over-availability decision: without stable state, a process restart cannot prove which cluster has authority. The safe TLA+ specification includes crash and restart; an unsafe restart that forgets admission must violate `RestartPreservesAdmission`.

20.4.5 Retained regression patterns

Other negative specifications preserve compact versions of repaired faults:

Unsafe action	Failure represented	Required violation
`UnsafeMessageWake`	Message readiness and endpoint-close observation share one observer queue.	`CloseSignalImpliesClosed`
`UnsafeRemoteAbort`	A remote LP removes and reaps a context that may still execute.	`ReapOnlyOffCpu`
`UnsafeSpawn DuringAbort`	A sibling publishes after the domain-abort snapshot.	`AbortingThreadsDoomed`
`UnsafeCancelReleases Buffer`	Cancellation releases a mutable buffer before the operation becomes terminal.	`NonTerminalBuffer RemainsLoaned`
`UnsafeTimerFire`	A watchdog reports timeout without rechecking a concurrently published work generation.	`TimeoutObservedNoWork`
`UnsafeObserveJoin`	A delayed join attaches its old handle to the replacement occupying the same TID.	`ObserverMatches CapturedHandle`
`UnsafeBeginExclusive Map`	Exclusive DMA begins while CPU or other transfer authority still exists.	`ExclusiveDmaHasNo CpuAuthority`
`UnsafeStaleClose`	A recycled numeric ASID is closed through an old generation handle.	`ReplacementSurvives StaleHandle`

Unsafe action	Failure represented	Required violation
`UnsafeAllocate`	A hardware ASID is reassigned before stale translations are invalidated.	`NoDirtyTagReuse`
`DrainUnsafe`	A deferred interrupt wake ignores its route generation.	`NoStaleWakeDelivery`
`UnsafeStaleUnregister`	Cleanup from an old service generation removes its replacement.	`ReplacementSurvives StaleUnregister`
`UnsafeReplicate ToJoiner`	A non-selected peer supplies authority to a joining node.	`JoiningAcceptsOnly SelectedAnchor`
`UnsafeRestartForgets Admission`	Restart makes an incomplete joiner election-eligible.	`RestartPreservesAdmission`
Flat session allocation	Retry and restart epochs alias one wire session.	`SessionIdentityUnique`
`AcceptDelayedSession`	An older delayed SYN regresses receive state.	`ReceiveSessionMonotonic`
`UnsafeSetPolicy`	A non-administrator changes a policy rule.	`PolicyMutationAuthorized`
`UnsafeRedeemOther Principal`	A decision for one subject is redeemed for another.	`MintBoundToPrincipal`
`UnsafeRedeemStale Policy`	A decision survives a policy-version change.	`MintUsesCurrentPolicy`
`UnsafeRedeemStale Binding`	A decision for an old service generation reaches its replacement.	`MintTargetsCurrentBinding`
`UnsafeAmplifyRights`	Redemption mints rights absent from the decision.	`NoRightsAmplification`

The check suite expects all twenty negative configurations to fail for the named reason. If one unexpectedly satisfies its invariant, the regression harness has lost sensitivity and the suite fails.

20.5 Concrete conformance and atomicity

TLA+ actions are atomic. Rust code usually crosses several structures, locks, or persistent writes. The conformance review must therefore identify a linearization point or represent the concrete intermediate states explicitly.

The mapping uses two correspondence classes:

Direct The abstract action follows one concrete transition while omitting data irrelevant to the invariant. For example, a successful fenced unregister directly checks owner and generation.

Abstract Several concrete operations are collapsed. This is valid only when intermediate states are unobservable or separately guarded. For example, scheduler retirement spans multiple tables, so the concrete `RETIREMENTS_IN_FLIGHT` state prevents address-space teardown during that interval.

Persistent ordering deserves the same care. In Raft admission, “snapshot membership and clear join fence” is one model transition, but two durable writes in Rust. Safety depends on snapshot flush preceding fence removal. A crash between those writes must map to a modeled state that remains safe.

The conformance document is not yet a machine-checked refinement proof. It is a reviewable refinement *obligation*: it names the functions and state that must be re-examined when either side changes. Code review should reject a model update that merely hides a new concrete intermediate state.

20.6 Validation

20.6.1 Running TLC

The repository wrapper runs the complete bounded suite:

```
docs/tla/check.sh /path/to/tla2tools.jar
```

Alternatively, `TLA2TOOLS_JAR` may name the tools archive. The script:

- runs all twenty safe configurations;
- enables action coverage and requires selected actions to have a positive successor count;
- rejects structural TLC warnings about variables, `EXCEPT` expressions, or incomplete successor states;
- runs twenty negative configurations and requires each one to fail on the named invariant after exercising the named unsafe action; and
- writes checkpoints and traces into a temporary directory so generated state does not become part of the source tree.

Action coverage is important. An invariant can pass because a transition is accidentally disabled. Requiring coverage does not prove that an action is correct, but it detects this common vacuity.

At the 15 August 2026 implementation synchronization point, representative fast configurations produced the following complete state graphs:

Model	Generated	Distinct	Depth
`CharlotteIPC`	6,118,645	1,254,153	13
`CharlotteCQ`	1,586,249	302,659	12
`CharlotteTimedWait`	22	15	8
`CharlotteScheduler`	4,308,577	819,972	22
`CharlotteThreadJoin`	37	32	9
`Charlotte ServiceLifecycle`	689,321	74,232	37
`CharlotteDMA`	19,331,379	2,092,882	26
`CharlotteAuthorization`	144,598	54,705	12
`CharlotteRaftJoin`	2,029	520	13
`Charlotte ReliableMessage`	30	21	5

State counts are evidence about one configuration, not a quality score. A small model may encode the decisive identity argument, while a large model may contain substantial independent bookkeeping. The invariant, abstraction, and coverage matter more than graph size.

20.6.2 Implementation validation

TLC is followed by validation proportional to the affected code. The August synchronization work, including the 15 August domain-abort and memory-ownership repair, included:

- host tests for `catten-graft`, including restart before durable membership and restart between membership persistence and fence clear;
- host tests for `charlotte-protocol-msg`, including disjoint retry/restart sessions and fail-closed exhaustion;
- host tests for ownership-aware runtime cleanup and the shared `charlotte-lifecycle` thread-join and timed-wait classifiers;
- AArch64 release builds and warning-denied Clippy for the changed `raft`, `relmsg`, and `ns` services;
- AArch64 and x86-64 kernel checks, plus warning-denied AArch64 Clippy, for the domain-abort publication fence and memory-ownership paths;
- a four-LP AArch64 QEMU boot whose authoritative result reported all eighteen registered self-tests passing, including owner-LP retirement, DMA/SMMU, completion cancellation, and service crash/restart; and
- formatting, shell syntax, and diff-whitespace checks.

Boot and multi-guest tests remain necessary when a change affects the concrete scheduler, device, storage, or network integration. A TLC result cannot show that QEMU delivers an interrupt, that an object-store flush reaches the emulated disk, or that a manifest starts the intended services.

20.7 What a successful check means

A successful safe configuration means that TLC found no reachable violation of the declared invariants within that specification and its finite constants. It does *not* by itself establish:

- a proof that arbitrary larger bounds preserve the result;
- a mechanically checked refinement from Rust to TLA+;
- liveness, fairness, bounded latency, or eventual leader election unless explicitly expressed as temporal properties;
- correctness of omitted bytes, parsers, cryptography, memory mappings, hardware programming, or transport framing;
- correctness under storage failure modes below the modeled durable write interface; or
- freedom from ordinary implementation bugs such as arithmetic, aliasing, or unsafe-memory errors outside the modeled projection.

Conversely, a gap between code and model is actionable information. It can mean one of three things:

1. **Functionality is unsafe.** The Raft admission restart gap was of this kind and required a CharlotteOS change.
2. **The model overclaims.** Atomic service replacement was of this kind; the model was corrected to admit temporary unavailability.
3. **The behavior is outside scope.** It must be documented rather than silently attributed to the checked invariants.

Therefore a green TLC run is not a blanket "formally verified" label for CharlotteOS.

20.8 Workflow for changes

Any change to ownership, generations, persistence, retry identity, lifecycle, or distributed membership should be checked against this chapter's method. A practical review checklist is:

1. Search `docs/tla/CONFORMANCE.md` for the changed Rust function or state field.
2. Decide whether the change adds an action, changes an enabling condition, moves a linearization point, or changes durable/volatile classification.
3. Update the safe model and bounds. If repairing a discovered fault, retain the old behavior as an unsafe action and named negative configuration.
4. Add the new action to the required-coverage list in `docs/tla/check.sh`.
5. Run the entire TLC suite, not only the edited module, because shared documentation claims and layered Raft assumptions can drift together.
6. Update the conformance map with exact Rust functions and ordering.
7. Add implementation tests at the crash, retry, or stale-generation edges identified by the model.
8. Run target validation and the relevant boot or multi-node suite.
9. Record deliberate omissions and avoid converting a safety result into an unsupported liveness or availability claim.

Used this way, TLA+ is part of the engineering control system for CharlotteOS. It preserves the reasoning behind a safety fence, makes regressions executable, and forces the model, manual, code, and tests to describe the same functionality.

A Glossary

AS (Address Space) A page-table context identified by an `AddressSpaceId` (0 = kernel, >0 = user). Contains the `TTBR0_EL1` (user) and `TTBR1_EL1` (kernel) pointers on AArch64, or `CR3` on x86-64.

Block (verb) To remove a running thread from the scheduler and register its `Waker` on an `Observable`. The thread yields; when the event fires the waker re-admits it.

Capability An unforgeable per-domain handle that names a kernel object and carries rights. Answers: "May I perform this operation?"

Capability table A per-AS sparse table mapping capability indices to kernel objects. Used for the thread table, address-space table, and per-domain capability table.

Completion The terminal result of an asynchronous operation. The kernel tracks operation state and delivers results through polling, waitable completion objects, or CQ entries.

Connection capability A client-side authority to send or call an endpoint. Minted from an endpoint, typically by the name service.

CQ ring (Completion Queue) A shared-memory ring buffer for zero-syscall completion delivery from the kernel to userspace.

EL0 Exception Level 0 (AArch64). Userspace execution.

EL1 Exception Level 1 (AArch64). Kernel execution.

Endpoint A bounded server-side IPC receive object. Has an owning domain, a queue capacity, and a lifecycle.

Exit-observer An `Observer` registered on a `Thread`; fires when the thread is dropped (exits). The ABI's intended completion mechanism: `submit_worker` registers the capability as the worker's exit-observer.

HHDM (Higher Half Direct Mapping) A linear mapping of all physical memory into the kernel's virtual address space, provided by the bootloader (Limine).

IdTable A sparse `Vec<Option<T>>` with ID recycling. Used for the thread table, address-space table, and capability table.

IOMMU / SMMU I/O Memory Management Unit. Hardware that enforces DMA access permissions from devices, analogous to the MMU for CPU access.

IPI (Inter-Processor Interrupt) A signal from one LP to another. The kernel's IPI queues are bounded per-LP queues carrying typed commands.

LP (Logical Processor) A schedulable hardware execution context, usually one core or hardware thread. It is distinct from a sitas shard.

Memory object A page-backed kernel object with explicit ownership and capability semantics. Can be mapped, moved, lent, or copied between domains.

MMU Memory Management Unit. Hardware that enforces page-table permissions for CPU access.

Name service A userspace service that maps human-readable names to connection capabilities. The kernel does not understand service names.

Notification "State may have changed; inspect the corresponding object or queue." May be coalesced. Is not itself a result.

Observable / Observer The event-notification pattern. An `Observable` holds `Weak<dyn Observer>` references; when it fires, it calls `Observer::notify()`.

PerLp<T> A boxed slice of per-LP cells with lock wrapping. Interrupt-safe (masks interrupts on acquire). Use for data accessed from interrupt handlers or multiple LPs.

Protection domain An address space and authority boundary. Owns a capability table, memory mappings, threads, and failure state. Typically implemented as a process.

Quantum timer The per-LP periodic timer (10 ms default) that drives preemptive context switching in the

`RoundRobin` scheduler.

Reaper The deferred-thread-cleanup mechanism: dead threads are moved to `DEAD_THREADS` and dropped from the next thread's context after the context switch.

Reply token One-shot authority to complete a blocking or deferred endpoint call. Created by the kernel, bound to the caller's pending call, consumable exactly once, invalidated by cancellation or teardown.

Shard A userspace ownership and execution domain. One executor runs its tasks; at most one task at a time mutates shard-owned state; cross-shard access goes through typed messages.

ShardLocal<T> A lock-free per-LP container using `UnsafeCell<T>` with owner-check and borrow-flag guard. sitas-derived. Use for single-LP, thread-only kernel data.

ShardMailbox<M> A typed bounded per-LP queue with `ShardSender<M>` (cloneable, any LP can send) and `ShardReceiver<M>` (single-consumer). First-class backpressure.

Supervisor Kernel-side component that creates and destroys protection domains, loads ELF images, delivers bootstrap capabilities, and handles domain teardown.

TrapFrame An architecture-specific snapshot of user register state at an exception. Syscall dispatch combines it with trusted per-thread or per-LP state to identify the caller.

Waker An `Observer` wrapping a `ThreadId`. When notified, calls `submit_ready_thread(tid)`. In userspace, a Rust `Waker` is a hint to repoll a future — it is not a kernel completion.

Yield To voluntarily give up the LP by calling `yield_lp()`. Saves the thread's context and lets the scheduler run the next ready thread.

B Implementation Status

This appendix is a snapshot of built and verified behavior. Source, tests, and repository history remain authoritative.

B.1 Implementation phases

Phase	Description	Status
1	Capability table, rights masks, object teardown	Done
2	Endpoint IPC v1: scalar send/call/receive/reply, connection delegation, reply tokens	Done
3	Userspace name/service manager, bootstrap delivery, ELF loader, supervisor, generation tracking	Done
4	First-class memory objects: allocate, map, unmap, close, ownership accounting	Done
5	Memory IPC: Copy, Move, BorrowRead, BorrowWrite, reply-bound revocation, cancellation, server-death recovery	Done
6	CQ subsystem normalization: operation IDs, detached submission, CQ_WAIT/CQ_WAKE, per-shard CQ partitioning, backlog batching, 32-byte completion records	Done
7	Sitas endpoint/CQ backend: endpoint readiness binding, unified shard wait, ShardExecutor (budgeted polling), ShardParker (spin free), per-shard CQ rings	Done
8	Userspace UART driver: delegated MMIO + IRQ, EL0 MMIO writes, interrupt-driven deferred reads, crash recovery	Done
9	Virtio-net driver: PCI discovery, MSI-X and SMMU delegation, feature negotiation, MAC/link read, virtqueue setup, and EL0 transmit submission. Smoltcp 0.13 adapter + TCP/IP service binary compile, and the frame demultiplexer routes IPv4/ARP to the tcpip service.	Two-guest TCG suite
10	Lock-free device interrupt delivery (deferred wake), idempotent scheduler wakes, sustained-window load rebalancing	Done
11	Userspace startup ABI: ELF `_start`, `entry!` macro, typed `Context`, fixed-width versioned launch header, explicit startup input	Done

B.2 Success criteria

1. Two EL0 domains communicate through a bounded endpoint — **Verified.**
2. Deferred async call while a shard continues other tasks — **Verified.**
3. Moved memory object — sender locked out until return — **Verified.**
4. Lending enforced by mappings, including death cleanup — **Verified.**
5. Single `CQ_WAIT` for endpoint + completions + device interrupts — **Verified.**
6. Terminal CQ results survive ring overflow (non-lossy backlog) — **Verified.**
7. Coalesced remote shard wakes (idempotent scheduler fix) — **Verified.**
8. Userspace UART driver with only delegated MMIO + IRQ — **Verified.**
9. Restart invalidates stale connections + device reset + op reconciliation — **Verified.**
10. Virtio-net data path with batched packet buffers — **Runtime-validated on macOS QEMU TCG.**
11. Fixed-width stable ABI representations (32-byte CQ entries) — **Implemented; not formally audited.**
12. Capability misuse and cancellation race adversarial tests — **Partial; six IPC error-path tests.**

B.3 Verified scope

- **AArch64:** boots and the synchronous self-tests pass. The EL0 service suite starts one shared node name service; Raft peers, echo, the service manager, NVMe/object store, UART, and their clients receive delegated connections to that registry. The deferred boot tests exercise registration, waitable lookup, restart, and live upgrade through `crt0/cmain`; the harness may print its synchronous success banner before those deferred tests finish. The sitas `basic_kv` binary runs with 2 shards on per-shard CQ rings.
- **x86-64:** boots with `-cpu max`, core self-tests pass. Ring-3 userspace is implemented but not yet exercised.
- **Completion subsystem:** submit, complete, poll, wait, cancel, close, `submit_worker` (exit-observer), CQ ring, CQ ring integrated with AS, non-lossy CQ overflow backlog, per-shard CQ partitioning, 32-byte entries.
- **Endpoint IPC:** bounded server queues, scalar messages, connection delegation with rights attenuation, memory-object attachments (move/lend/copy), deferred replies, reply-time connection delegation.
- **Device capabilities:** MmioRegion and Interrupt object types, user-accessible Device-nGnRnE page mapping, GICv3 SPI enable/route/mask, IRQ-to-CQ coalesced delivery (lock-free deferred wake).
- **Userspace drivers:** reference UART driver (PL011) with delegated MMIO + IRQ, interrupt-driven deferred reads, crash-test restart with device reset and operation reconciliation.
- **Syscall dispatch:** AArch64 SVC dispatch uses the shared typed `SyscallNumber` enum. Caller identity comes from trusted thread and trap context, never a userspace parameter.
- **Scheduler:** RoundRobin, quantum timer, block/yield/wake, idempotent wakes, deferred reaper, per-LP thread pinning.
- **Sitas integration:** sitas-core compiles for aarch64-none. sitas-charlotte links against the kernel's syscall ABI. Per-shard CQ rings mapped by the loader contract. ShardParker retires busy-waiting.
- **Protocol crates:** `charlotte-protocol-net`, `charlotte-protocol-msg`.
- **Networking:** smoltcp 0.13 adapter and TCP/IP service runtime-validated over the frame demultiplexer; reliable-message fragmentation to 64 KiB.
- **Raft:** two-node election over local endpoint IPC, persistent single-voter restart, discovery-led join

through joint consensus, and a two-guest replicated DNS catalog. Restart-safe admission has focused host tests and a persistent-storage boot-test manifest.

- **CI:** GitHub Actions builds both architectures with `-D warnings`, QEMU boot gate on AArch64.

B.4 Known limitations

- **HVF does not honor hardware ASIDs.** Apple's Hypervisor.framework strips the ASID bits from `TTBR0_EL1` on read and does not use them to isolate user translations, so the kernel's ASID-based TLB fast path is unsafe there. Under `hvf_compat`, `switch_ctx` flushes the whole TLB on every context switch (`tlbi vmalle1is`) and the SVC handler attributes syscalls from a per-LP tracked logical ASID rather than a `ttbr0_el1` readback. See *AArch64 Port Status* for the full diagnosis.
- **x86-64 ring-3 userspace:** SYSCALL entry trampoline exists; the EL0 test and real-EL0 user thread are AArch64-only.
- **Virtio-net runtime:** initialization and transmit submission are runtime-validated on macOS under QEMU TCG. HVF remains unsuitable for direct EL0 device MMIO. The receive/transmit data path, frame demultiplexing, reliable messages, distributed name service, and the TCP/IP service are validated; TX reclamation and batching remain incomplete.
- **General process loader:** the ELF loader is fixed-address for reference service images, not a general loader with argv/env/auxv-style setup.
- **Resource accounting:** per-domain quotas are not yet implemented; capability-table capacity is fixed.
- **Cross-machine Raft boundary:** two TCG guests share a QEMU stream LAN; the reliable-message frame transport and remote Raft peer routing are connected and validated (`-dns-test`).
- **Observability HTTP:** the httpd keyhole aggregates the `observe` snapshot and per-service status into a JSON report served over the TCP/IP service (`-http-test`).
- **IOMMU/SMMU integration:** coherent Arm SMMUv3 discovery via ACPI IORT, per-requester DMA-domain capabilities, memory pinning, IOVA map/unmap, synchronous invalidation, fault reporting, teardown, and NVMe integration are implemented and QEMU-tested. Non-coherent systems, ATS/PRI, multiple SMMUs, and other IOMMU architectures remain.
- **Upgrade:** the EL0 service manager performs handoff and invokes the capability-checked replacement spawn path. It can fetch a replacement ELF from a well-known NVMe object, pass it as a memory capability, and fall back to the embedded recovery image. Structural ELF validation, immutable kernel snapshotting, and Ed25519 CLS2 signature and logical-name checks are implemented. Rollback and key rotation policy, multiple state objects in the reference manager, and manager-owned teardown remain future work.
- **Resource bounds:** endpoint queues and rings are bounded, but the non-lossy CQ overflow backlog has no hard memory bound.

C Lessons Learned

Booting the asynchronous kernel under QEMU exposed integration faults that unit tests and static checks had missed. This appendix records the most instructive ones.

C.1 Bugs found on hardware

C.1.1 Panic handler recursed through heap-dependent logging

Symptom: any fault during early boot silently triple-faulted instead of printing a panic message. **Root cause:** the panic handler used the normal logging path, which allocated from the kernel heap. A fault before the heap was ready caused the panic handler to fault, re-entering itself, recursing until stack overflow and triple fault. **Fix:** the panic handler now uses `early_logln!` (direct serial writes, no heap dependency). This unmasked every subsequent bug.

C.1.2 Blocked threads lost their execution context

Symptom: a thread that blocked in `wait()` restarted from its entry point when woken. **Root cause:** `block_thread` cleared the scheduler's `current_handle` before `yield_lp` saved the thread's context, so the save was skipped. **Fix:** a self-blocking thread stays `current_handle` until the yield saves its context.

C.1.3 RwLock held-across-call deadlocks

Symptom: `sleep()` worked with logging but hung without it (a heisenbug). **Root cause:** an `if let` binding extended an RwLock read guard's lifetime across a call that tried to acquire the write lock. **Fix:** bind the value out of the scrutinee first, so the temporary guard drops before the re-locking call.

C.1.4 Quantum timer grew without bound

Symptom: the per-LP timer queue contained thousands of quantum events after extended operation. **Root cause:** manual yields enqueued a new quantum event each time. **Fix:** an `armed` flag keeps exactly one quantum event in flight.

C.1.5 A thread cannot free its own kernel stack

Symptom: a returning kernel thread hung during teardown. **Root cause:** `ThreadContext::Drop` deallocated the stack while the thread was still executing on it. **Fix:** a deferred reaper moves dead threads to a zombie list and drops them from the next thread's context after the switch.

C.1.6 LA57 paging-mode mismatch

Symptom: a `#GP` with a non-canonical address on x86-64. **Root cause:** the address width was derived from CPUID (57-bit capability) instead of `CR4.LA57` (the active mode, 48-bit under Limine). **Fix:** derive from the active paging mode.

C.1.7 ASID-0 collision

Symptom: the first user thread faulted at its entry point. **Root cause:** the first user address space received ASID 0, which equals `KERNEL_ASID`, so the thread was created as a kernel thread (EL1), where PXN forbade execution of user code. **Fix:** pre-populate the table with the kernel AS at index 0.

C.1.8 x86-64 trampoline halted on thread return

Symptom: the async demo's worker completed but the coordinator never resumed. **Root cause:** the x86-64 trampoline entered a `hlt` loop when a thread returned, instead of calling `abort()`. **Fix:** make the x86-64 trampoline call `abort` on return.

C.1.9 Hypervisors that strip TTBR0 ASIDs break TLB isolation

Symptom: under Apple's Hypervisor.framework, EL0 tests failed with apparently unrelated data aborts, wrong syscall results, and deadline panics; TCG remained green. **Root cause:** HVF did not preserve the ASID bits in `TTBR0_EL1`. Because context switches relied on those tags instead of a full TLB flush, low user virtual addresses could resolve through another domain's stale translation. **Fix:** `hvf_compat` performs a whole-TLB flush on each context switch and obtains syscall identity from trusted per-LP state. **Lesson:** revalidate translation-tag assumptions on every hypervisor. See *AArch64 Port Status* for the detailed diagnosis.

C.2 Engineering lessons

1. **Test transitions, not only interfaces.** The difficult failures occurred where block, yield, wake, context save, and reclamation met. Tests must cross those boundaries.
2. **Keep failure paths independent.** Panic reporting must work before the heap and scheduler; reclamation must run from a live stack.
3. **Validate platform assumptions at runtime.** CPU capability does not imply active paging mode, and a hypervisor may not preserve hardware translation tags.
4. **Treat lock lifetime as part of control flow.** Bind values out of guards before calling code that may acquire the same lock.

D Research Lineages and Their Afterlives

The systems cited throughout this manual did not share one fate. Some remain active, some became products or standards, some were absorbed into a larger platform, and some ended after demonstrating one useful mechanism. This appendix separates those outcomes from CharlotteOS's own inheritance.

A research operating system need not replace Unix or Windows to matter. Its usual afterlife is selective: a fast IPC path, an authority model, a queue discipline, or a deployment abstraction survives at a different layer while the original system and its stronger unifying thesis disappear. Conversely, the adoption of one mechanism does not validate every claim made by the original architecture.

D.1 Capability kernels and confined Unix

KeyKOS, EROS, and CapROS. KeyKOS was a commercial capability system; EROS carried its object-capability and persistence ideas into research, and CapROS continues EROS as a small open-source project. They did not displace conventional operating systems, but their account of authority—possession, delegation, attenuation, confinement, and persistence as properties of references rather than global names—remains foundational. The CapROS project explicitly identifies itself as the continuation of EROS (https://www.capros.org/).

L4 and seL4. This lineage achieved both assimilation and independent survival. Earlier L4 variants demonstrated that microkernel IPC could be fast and shipped commercially in mobile devices. seL4 added machine-checked functional correctness and security properties, became open source in 2014, and acquired a foundation and commercial high-assurance ecosystem in 2020. The project's history records research prototypes, the HACMS deployment, proof milestones, and the later ecosystem (https://sel4.systems/About/history.html).

Capsicum and CHERI. Capability mechanisms also entered conventional systems in narrower forms. Capsicum has shipped in FreeBSD since version 9.0 and treats file descriptors as attenuable capabilities inside Unix (https://man.freebsd.org/cgi/man.cgi?query=capsicum&sektion=4). CHERI moved bounds and permissions into tagged pointers and processor support. Arm's Morello board and CheriBSD established a substantial experimental stack, while CHERI-RISC-V work continues toward standardisation (https://www.cl.cam.ac.uk/research/security/ctsrd/cheri/). These are complementary outcomes: Capsicum confines named Unix resources and CHERI constrains memory references; neither is by itself a complete object-capability operating system.

D.1.1 CharlotteOS inheritance

CharlotteOS adopts the strong local authority rule: an opaque, typed object capability is authority, may carry attenuated rights, and is not a name, notification, completion, or unit of work. Reply tokens are linear authority; memory, device, DMA, observer, and bootstrap access are explicitly delegated. The present kernel does *not* retain a general derivation tree, prove confinement, or provide CHERI-style protection for arbitrary pointers. This is an EROS/seL4-inspired object model, not an seL4 refinement or a CHERI substitute.

D.2 IPC, typed channels, and ownership

Mach. Pure Mach did not become the dominant Unix organisation, but it was absorbed into Apple XNU. XNU retains Mach tasks, virtual memory, ports, and IPC while co-locating BSD, networking, filesystems, and I/O components in one kernel for performance. Apple's description is candid about this hybrid outcome (Apple's XNU architecture note). The lesson is double-edged: message-mediated authority survived, while the pure user-server decomposition did not.

Singularity. Microsoft ended the standalone research OS and evolved its design into the internal Midori project. Midori never became a commercial OS, although Microsoft reports that it ran part of the company's natural language search service (`https://www.microsoft.com/en-us/research/project/singularity/`). Typed channel contracts, software-isolated processes, manifest-checked components, and ownership transfer through an exchange heap outlived the platform as research ideas. Managed-language enforcement did not become a mainstream OS boundary, but ownership types and message-oriented runtimes became normal engineering tools.

Xous. Xous remains a working Rust microkernel for embedded devices, not merely an historical analogy. Its userspace servers and distinct scalar, send, lend, and mutable-lend messages make it CharlotteOS's closest direct IPC ancestor (`https://xous.dev/kernel/`).

D.2.1 CharlotteOS inheritance

CharlotteOS takes Xous's isolated-server model and its separation of scalar from memory-bearing IPC. It takes Singularity's ownership-moving discipline but enforces it through capabilities, page mappings, tear-down state machines, and DMA rules rather than a managed-language verifier. Copy, move, read-only borrow, and writable borrow have explicit failure, cancellation, restoration, and rollback behavior. Typed protocol crates provide channel contracts above an ABI that still validates untyped words at the kernel boundary.

D.3 Multicore systems, runtimes, and fast I/O

Barrelfish and Arrakis. Barrelfish argued that a multicore machine should be treated as a distributed system with explicit messages and topology-aware state placement. The project now identifies itself as inactive (`https://barrelfish.org/download.html`). Arrakis gave applications virtualised device queues while leaving allocation and protection policy in the kernel; it merged back into Barrelfish in 2015. The code line ended, but per-core runtimes, hardware queue virtualisation, and control/data-plane separation became widespread design patterns.

IX, DPDK, and Seastar. IX was a research dataplane OS. Its run-to-completion, per-core, batched approach continued in datacenter runtimes. DPDK succeeded by becoming a Linux Foundation userspace packet-processing project with production device support (`https://www.dpdk.org/about/`). Seastar remains an active shared-nothing, shard-per-core, futures runtime used by ScyllaDB (`https://seastar.io/`). In this family, the ideas prospered as libraries and queues on Linux more than as replacement operating systems.

Windows completion ports, BSD `kqueue`, Linux `epoll`, and Linux `io_uring` show the same selective adoption. Submission, retained completion, aggregation, and batching became normal interfaces without making notification, readiness, completion, IPC, and authority one thing.

D.3.1 CharlotteOS inheritance

Sitas is the direct shard execution discipline and is Seastar-like: one owner per shard, cooperative futures, bounded messages, and deliberate placement. Barrelfish supplies the analogy for explicit cross-LP coordination. IX, DPDK, and Arrakis inform userspace drivers, batched queues, IOMMU protection, and buffer ownership. CharlotteOS makes a different authority choice from Arrakis: devices are delegated to isolated driver services, not directly to ordinary applications. Completion queues retain terminal operation records and remain separate from endpoint IPC and Rust `Wakers`.

D.4 Service recovery

MINIX 3, CuriOS, and Pebble explored minimally privileged userspace services, drivers, supervision, and restart. Nooks and SafeDrive instead tried to contain faulty extensions while preserving compatibility with in-kernel drivers. MINIX remains a research and teaching system; the other projects are completed prototypes. Their broad result—isolate components and supervise them—is now conventional. Their harder problem did not disappear: hardware, DMA, persistent state, and externally visible effects prevent arbitrary restart from being transparent.

CharlotteOS adopts the strong containment boundary and the honest recovery contract. The supervisor can revoke capabilities and borrows, reset a device, reconcile operations and DMA, start a fresh generation, and require clients to resolve it again. A stateful upgrade transfers explicitly serialised state; it does not inherit a process image or promise uninterrupted execution. This is closer to MINIX's supervision goal than to transparent recovery.

D.5 From distributed operating systems to cluster managers

Amoeba. Amoeba's last official release was 5.3 in 1996. Its processor pools, capability-protected objects, location-independent RPC, and FLIP network did not become a maintained product platform (https://www.cs.vu.nl/pub/amoeba/manuals/usr.pdf). The component ideas did become ordinary: pooled compute, object references, RPC, directories, and replicated services. The part modern systems mostly rejected was the promise that a distributed computer should look like one failure-free machine.

Plan 9 and Inferno. Bell Labs development ended, but Plan 9 has community descendants. Its 9P protocol remains in Linux, including a Virtio transport (https://www.kernel.org/doc/html/latest/filesystems/9p.html). Per-process namespaces and protocol-served resources stayed influential while Plan 9 and Inferno themselves remained niche.

Jini and MOSIX. Sun's Jini became Apache River; Apache retired River in 2022 (https://attic.apache.org/projects/river.html). The openMosix project officially ended in 2008 (https://sourceforge.net/p/openmosix/news/2008/02/openmosix-project-ends/). Discovery, leases, tuple spaces, automatic placement, and migration were not lost. They reappeared in registries, brokers, serverless platforms, and cluster schedulers. Modern systems usually reschedule a container or actor and reconstruct service state instead of migrating an arbitrary process image.

Borg, Omega, and Kubernetes. Borg remains Google's internal production cluster manager; Omega explored a concurrent control plane; and Kubernetes directly inherited their lessons about declarative desired state, scheduling, services, labels, and self-healing. Google's own account describes that lineage (https://kubernetes.io/blog/2015/04/borg-predecessor-to-kubernetes/). The

cluster became the deployment target, but above commodity kernels and containers rather than inside a distributed OS kernel.

Focused mechanisms. Raft succeeded as a reusable replicated-state- machine protocol with many implementations (https://raft.github.io/). SWIM-style gossip survived in memberlist and Consul. Nix made hash-identified immutable software stores practical. TUF and Uptane became maintained update security frameworks; Uptane is now a versioned standard under Linux Foundation governance (https://uptane.org/learn-more/about). Sigstore, Cosign, and Notary brought signed identity and provenance into cloud artifact workflows.

D.5.1 CharlotteOS inheritance

CharlotteOS separates a human-readable name from the connection capability returned by policy-controlled lookup. Its `dns` catalog is a Raft state machine across two guests. Remote calls use request identity, generation fencing, duplicate handling, deadlines, and an explicit uncertain outcome. The cluster slice verifies Ed25519-signed ELF notes, commits placement records, and reassigns a stateless service. It is Amoeba-like in interface and Kubernetes-like in deployment intent, but remains bounded: automatic placement, a shared content-addressed artifact store, production key rotation, general stateful migration, and distributed capability revocation are open.

D.6 The synthesis and the rejected inheritances

Relationship	Source ideas	CharlotteOS status
Direct source	Xous IPC and memory messages; Sitas shard ownership and async execution	Implemented locally, combined with CharlotteOS capabilities, memory objects, completion queues, and teardown semantics
Adopted and changed	EROS/seL4 authority; Singularity ownership transfer; Barrelfish messages; MINIX recovery; completion-ring I/O	Implemented in bounded local forms; no compatibility or refinement claim
Focused protocol	Raft replication; Ed25519/SHA-256 artifact verification	Implemented and tested, including cross-guest catalog replication and signed deployment
Architectural influence	Amoeba invocation; IX/DPDK/Arrakis fast paths; Borg/Kubernetes placement; Nix/TUF/Uptane artifacts and trust	Partial cluster slice or future work
Rejected or narrowed	Failure-transparent remote calls; arbitrary process migration; language-only isolation; direct device access by ordinary apps	Remote uncertainty is visible; migration is protocol-specific; MMU/IOMMU and capabilities enforce isolation; drivers retain device authority

E Architectural Redirections

This appendix records decisions that redirected early experiments from a "universal completion-capability" model to the three-primitive architecture in this manual. Later chapters describe the resulting implementation.

E.1 Retained mechanisms

The following early mechanisms remain part of the architecture:

- AArch64 EL0/SVC path.
- Caller identity derived from the current trap/thread context.
- Per-address-space capability tables.
- Shared CQ ring.
- Non-lossy kernel CQ backlog.
- `CQ_WAIT` blocking through scheduler/observer machinery.
- Bounded queues and explicit `WouldBlock`.
- No-std sitas split and CharlotteOS backend.
- Per-shard executor model.
- Typed in-kernel `ShardMailbox<M>`.
- Closure-based `ShardLocal<T>` with restricted use.
- Segment-aware ELF loading.
- QEMU boot CI across AArch64 and x86-64.

E.2 What to reframe

These components are correct but their *role* in the architecture has shifted:

- **Completion capability work** is the **operation completion subsystem**, not the universal service IPC subsystem. Use completion queues for kernel and device operations; use endpoints for cross-domain service calls.
- **Observer/waker code** is internal scheduling machinery, not the public semantic identity of all async objects. A Waker is a hint to repoll a future; it is not a completion record.
- **IPI queue closures** are a temporary kernel work mechanism, not the long-term cross-domain messaging model. Use typed mailboxes or closed kernel enums for subsystem-specific work.
- **The earlier mailbox syscall ABI** is a smoke-test interface only. The stable ABI is endpoint IPC.

E.3 Replacements adopted

E.3.1 Stop expanding raw mailbox syscalls

A mailbox tied directly to LP identity is not a complete userspace service abstraction: it exposes placement where authority should be exposed, lacks protocol identity, conflates shard transport with domain IPC, and lacks move/lend semantics. Endpoint and connection capabilities are therefore independent of LP identity. Clients address the service connection, not a target LP.

E.3.2 Do not allocate one completion capability per service request

For userspace service calls, the natural object is a reply token. For high-rate kernel operations, per-operation capabilities create unnecessary table pressure. The design uses reply tokens for endpoint calls; operation IDs plus CQ entries for ordinary kernel operations; first-class operation capabilities only where independent authority is required.

E.3.3 Separate readiness, notification, and completion

Their contracts differ: readiness means progress may be possible; notification means inspect state; completion means an operation changed state and has result data. Keep one aggregated wait mechanism, but define distinct object semantics.

E.3.4 Replace arbitrary cross-LP closures

Boxed `FnOnce()` work transported through the IPI layer is hard to account, difficult to inspect, and brittle under backpressure. The preferred direction is closed kernel enums for mandatory kernel RPCs and typed mailbox instances for subsystem-specific work.

E.3.5 Do not equate shard wake with IPI delivery

Remote wake mapping directly to `send_ipi(target_lp)` can generate excessive interrupts. The implemented rule is to queue first, mark pending, and send a coalesced reschedule IPI only when the target cannot otherwise observe the transition.

E.3.6 Introduce memory objects before rich IPC payloads

Move/lend semantics require durable object identity rather than transient pointers. CharlotteOS introduced memory-object capabilities, mapping, ownership transfer, temporary lending, lifecycle revocation, and DMA integration before promoting rich IPC payloads.

E.3.7 Treat service discovery as userspace policy

String-based lookup remains outside the kernel. The userspace name service receives bootstrap authority and delegates connection capabilities.